

Relatório de Implementação Concorrente para o Algoritmo BM25

Carlos Eduardo Valle Rosa Filho ¹

¹ Federal University of Rio Grande do Norte, Natal, Brazil

Resumo. *Este relatório inclui a descrição e comparação de implementações concorrentes para o algoritmo Best Match 25 (BM25) nas linguagens Java e Haskell. Vale citar que isto é tarefa avaliativa para disciplina de Programação Concorrente (2025.1).*

1. Introdução

No modelo de programação concorrente, as tarefas computacionais são realizadas em períodos de tempo que podem se sobrepor. Isto vai contra a noção comum, baseada na programação serial, na qual as tarefas são feitas sequencialmente, cada uma iniciando somente após o término da anterior.

A adoção dessa forma de programar, considerando sistemas com múltiplos processadores, trás múltiplas vantagens como: aumento de desempenho, aumento em responsividade, aumento do *throughput* e uso eficiente dos recursos do sistema. Portanto, é comum que linguagens de programação, principalmente aquelas mais populares, tenham suporte, mesmo que às vezes simples, para a programação concorrente. Para isto, uma linguagem deve possuir alguma maneira de criar *threads* — sequências separadas de execução de operações de dentro de um processo — e consequentemente alguma maneira de prevenir Condições de Corrida, um tipo de bug que ocorre quando uma mesma região de memória é utilizada por mais de uma thread.

Dessa forma, o propósito deste relatório é comparar não só o suporte à concorrência nas linguagens Java e Haskell, como também diferentes maneiras de se obter isso dentro da mesma linguagem. A comparação se baseia na implementação de versões do algoritmo Best Match 25 (BM25) — também conhecido como Okapi 25 — em cada linguagem, visando explorar ao máximo os diferentes recursos de programação concorrente oferecidos. A performance de cada versão vai ser avaliada pela execução do algoritmo em um conjunto de dados de 1,1 GB com documentos, levando em conta aspectos como o tempo de execução, memória utilizada, *throughput* e etc.

Este relatório é estruturado da seguinte maneira: Na seção 1.1 apresento o algoritmo BM25. Na seção 1.2 apresento Haskell com relação à conceitos de programação concorrente. Na seção 2 apresento a configuração dos experimentos. Na seção 3, apresento a versão serial do algoritmo em ambas as linguagens, bem como os resultados dos experimentos, comparando a mesma versão nas duas linguagens. Já nas seções 4, 5, 6, 7 e 8 discuto as versões concorrentes do algoritmo em ambas as linguagens (exceto em Volatile, na qual só há a versão em Java), também com os resultados dos experimentos, apenas comparando a mesma versão entre as linguagens. Por fim, na seção 9, faço uma discussão final comparando as diferentes versões das diferentes linguagens e apresento minha considerações.

Observações:

- A versão de Java considerada neste relatório é a 24.
- Não irei detalhar conceitos de Java (incluindo os concorrentes), pois assumo ser de conhecimento comum do leitor.
- A seção 1.2 não vai entrar em detalhes sobre conceitos de Haskell fora aqueles relevantes para programação concorrente.

1.1. O algoritmo BM25

O BM25 é um algoritmo de ranqueamento, utilizado por motores de busca, para estimar a relevância de documentos dado uma certa pesquisa, denominada *query*. Nele, utiliza-se conceitos do Modelo de Relevância Probabilística cuja explicação está fora do escopo deste relatório.

O algoritmo consiste no cálculo de um *score* (valor que representa a relevância) para cada documento, retornando o documento com o maior valor. Este cálculo leva em conta uma *query*, separada em tokens — sequências de caracteres atômicas — e a frequência destes tokens no conjunto de documentos considerados. Segue a fórmula utilizada:

$$score(D, Q) = \sum_{i=1}^n IDF(q_i) \cdot \frac{f(q_i, D) \cdot (k + 1)}{f(q_i, D) + k \cdot (1 - b + b \cdot \frac{|D|}{avgdl})}$$

Em que D é um documento, Q uma query, n a quantidade de tokens em Q , q_i o i -ésimo token de Q , $IDF(q_i)$ a frequência inversa de documento para q_i , $f(q_i, D)$ a frequência q_i em D e $avgdl$ o tamanho médio (em tokens) dos documentos. Além disso, k e b são parâmetros definidos como 1.5 e 0.75, respectivamente, por convenção. Sobre a função IDF , esta é definida como:

$$IDF(q_i) = \ln \left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1 \right)$$

Em que N é a quantidade total de documentos e $n(q_i)$ a quantidade de documentos que contém q_i [BM25 Wikipédia 2025, Robertson and Zaragoza 2009].

1.2. Haskell

Haskell é uma linguagem programação de propósito geral que segue o paradigma funcional de forma pura. Dentre suas principais características estão: inferência de tipos, tipagem forte e avaliação *lazy*. Sobre a última característica, ela se refere ao fato da linguagem não computar algo que não é imediatamente necessário. Tal característica permite Haskell representar, por exemplo, listas de tamanho infinito, pois não é necessário o cálculo de todos os seus elementos para o seu manuseio no código.

Antes de falar sobre as características relevantes à programação concorrente (PC), gostaria de informar que o compilador utilizado para esta linguagem é o Glasgow Haskell Compiler (GHC) — bem como o interpretador GHCi. Tal compilador pode apresentar detalhes de implementação para PC diferentes de outros, entretanto, por ser o compilador mais popular — e com o melhor suporte para PC — [GHC 2025], não irei diferenciar entre as noções apresentadas por Haskell e GHC.

Primeiramente, Haskell, possui suporte para a criação de threads, utilizando principalmente o compartilhamento de memória simétrico como modelo de programação, mas também permite passagem de mensagens. Em segundo lugar, vale citar que Haskell separa a sua abordagem para utilização de threads em duas principais categorias: Concorrente e Paralela. Em que a primeira permite a criação de threads pelo usuário, as quais podem estar executando em períodos de tempo sobrepostos, e a segunda permite a execução de tarefa computacionais em paralelo, em múltiplos processadores, ou seja, ao mesmo tempo.

Haskell cita que a abordagem focada em melhoria de desempenho é a paralela, enquanto a concorrente deve ser utilizada para aplicações que necessitam deste tipo de abordagem. Entretanto, escolhi utilizar somente a abordagem concorrente, pois ela oferece uma liberdade na manipulação de threads parecida com aquela vista em outras linguagens de programação, como Java. Já na Paralela, o uso de threads é implícito [Haskell Parallel 2025]. Dessa forma, o restante deste texto fala em relação a abordagem Concorrente, que pode divergir ou não da Paralela nas características discutidas.

1.2.1. Controle de região crítica

O controle da região crítica pode ser feito pelo uso de *Locks* ou operações atômicas. Há dois usos de operações atômicas em Haskell: *Software Transactional Memory* (STM) e com `IORef`'s.

Com relação à *Locks*, Haskell possui um tipo denominado `MVar` (*Mutable Variable*), no qual as operações de leitura e escrita são atômicas. A operação de leitura é tal que remove o valor armazenado, e bloqueia a execução da thread caso não há valor para ser lido. Já a operação de escrita não consegue sobrescrever valores da variável, logo ela bloqueia até que o valor possa ser escrito.

```
mvar <- newEmptyMVar :: MVar String
PutMVar mvar "exemplo"
str <- TakeMVar mvar
-- str contém "exemplo"
```

Com relação a STM, utiliza-se uma variável compartilhada do tipo *TVar* (*Transactional Variable*), o qual permite operações atômicas otimizadas de leitura e escrita. Essencialmente, pode haver várias tentativas para executar a ação até que esta ocorra sem conflito.

```
compartilhado <- newTVarIO ""
antes <- atomically $ readTVar compartilhado
atomically $ writeTVar antes "exemplo"
depois <- atomically $ readTVar compartilhado
-- depois contém "exemplo"
```

Em comparação com Java, a abordagem utilizando *Locks*, apesar de conceitualmente parecida, se difere bastante no uso em código, já que em Java os *Locks* são geralmente atribuídos a regiões de código, como métodos, enquanto em Haskell representam pontos de parada da execução, normalmente em uma única linha. A respeito da STM, há bibliotecas disponíveis em Java que a implementam, como Multiverse [Multiverse 2012], por exemplo, em que a utilização se parece com aquela vista em Haskell.

Com relação a `IORef`'s, são referências mutáveis em Haskell. Segue código ilustrativo:

```
r <- newIORef 0
antes <- readIORef r
-- o valor de antes é 0
writeIORef r 1
depois <- readIORef r
-- o valor de depois é 1
```

Apesar destes não serem feitos para aplicações concorrentes, como os outros dois citados até agora, podem ser utilizados neste contexto com funções atômicas fornecidas. Estas são: `atomicModifyIORef` e `atomicWriteIORef`.

```
r <- newIORef 0
atomicWriteIORef r 2
antes <- readIORef r
-- o valor de antes é 2
modifyIORef' r (+ 1)
depois <- readIORef r
-- o valor de depois é 3
```

1.2.2. Suporte a *Virtual* e *Platform Threads*

Em Java, uma *Virtual Thread* é uma thread cuja existência no código não corresponde a uma thread no sistema operacional. Haskell permite a criação de algo similar por meio do método `forkIO`, o qual coloca a chamada de uma função em uma thread separada.

```
threadId <- forkIO . algumaFunção
```

Este tipo de thread utiliza menos memória que uma thread convencional e tem escalonamento pre-emptivo, gerenciado pelo GHC [Haskell Concurrent 2025]. Parte deste gerenciamento inclui a manutenção de uma fila de espera para a associação destas *virtual threads* à threads de sistema. Isso ocorre de maneira similar a Java, em que uma ação IO efetuada por uma virtual thread a leva de volta para a fila de espera.

Outra similaridade a Java, está no fato da possibilidade da criação de threads de sistema (*Platform threads* em Java). Para isto, utiliza-se a função `forkOS`, da mesma maneira que `forkIO`:

```
threadId <- forkOS . algumaFunção
```

Esse tipo de thread é gerenciado pelo GHC da mesma maneira que as *Virtual threads*. A diferença está na maior quantidade de overhead e na possibilidade de comunicação com bibliotecas externas que utilizam estado local de thread, como OpenGL [Haskell Concurrent 2025].

1.2.3. Compilação e Interpretação

Programas em Haskell podem ser tanto compilados, utilizando o GHC, como interpretados com o GHCi. A compilação pode ser feita para código C, LLVM e assembly.

É importante citar que o uso de mais de uma thread de sistema em um programa Haskell só é possível em um programa compilado, utilizando as *flags* `-threaded` e `-N` para permitir o uso de múltiplos processadores e especificar este número, respectivamente. Portanto, somente detalharei o processo de compilação.

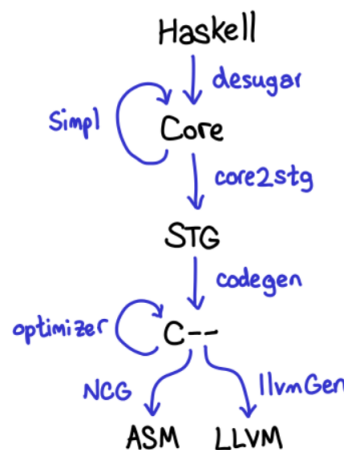


Figura 1: Esboço do processo de compilação (colocar fonte)

Como visto na Figura 1, a compilação pode ser resumida em 5 estágios, em que não há reordenamento de instruções até o estágio STG. A partir do estágio C--, o qual consiste em converter o código do estágio anterior em código para a linguagem C--, pode haver reordenamento de instruções [Yang 2014]. Entretanto, não é certo como este reordenamento ocorre, visto que C-- não possui um modelo de memória.

No fim, observa-se que o GHC reordena operações de *load* frequentemente e pode ou não reordenar operações de *write*. A única garantia proporcionada é que a reordenação de instruções não viola questões de segurança de tipos [Yang 2014].

Claramente, este modelo difere consideravelmente daquele utilizado por Java. Vale notar a ausência de elementos como a compilação *Just in Time* (JIT) e *Ahead of Time* (AOT). Com relação a PC, a maior diferença está na ausência, em Haskell, do uso de barreiras de *load* [Yang 2014].

1.2.4. Modelo de Memória e Problemas de Visibilidade

Haskell é uma linguagem funcional pura, logo suas variáveis são normalmente imutáveis — com exceção daquelas utilizadas para PC e `IORef`'s. Isto torna a definição de um modelo de memória mais simples. Essencialmente, a única maneira de compartilhar uma variável entre threads são com aquelas feitas justamente para este tipo de situação, como *MVars* e *Tvars*, as quais já foram discutidas em 1.2.1, ou com `IORef`'s. Portanto, o

único problema de visibilidade possível em Haskell surge do uso descuidado (sem utilizar as funções atômicas fornecidas) de `IORef`'s.

Haskell utiliza um GC e tem uma tipagem forte, entretanto, estritamente falando, não é uma linguagem *memory safe*, visto que, por meio de `IORef`'s, é possível acessar endereços de memória fora de faixas permitidas [Löh 2022]. Mas, para casos em que esta funcionalidade não seja utilizada, a linguagem pode ser considerada *memory safe*.

1.2.5. Coleta de Lixo

Por padrão, o coletor de lixo utilizado é o geracional — com duas gerações — o qual age em blocos de memória no espaço definido para o programa [Architecture 2012]. Cada bloco tem 4kb e a pode ser ligado a outro bloco. Utilizando essa abordagem de lista encadeada, não há preocupação em compactar a memória disponível.

Um importante aspecto de Haskell, como citado na seção anterior, é a imutabilidade de suas variáveis. Isto causa um uso elevado da memória, na forma de objetos temporários, mas também garante que objetos imutáveis nunca apontem para objetos da nova geração. Logo, o trabalho do GC se torna consideravelmente mais fácil.

Em suma, novos dados são alocados em uma *nursery* de 512kb que ao ficar sem espaço leva a um *minor GC*. Todos os valores que sobreviveram são copiados para a área de memória principal, enquanto os outros são desconsiderados. Isto leva a uma noção contra intuitiva, já que quanto maior quantidade de lixo, mais rápido será a ação do GC, pois não há interação com os valores desconsiderados [Haskell Wiki 2025].

2. Configuração dos Experimentos

2.1. Configuração do Computador

- Tipo de sistema: 64x
- Processador: Intel(R) Core(TM) i7-8750H CPU @2.20GHz 2.21GHz
- Núcleos: 6
- Memória: 16GB
- SO: Windows 10, Versão: 22H2

2.2. Conjunto de dados

Neste projeto, o conjunto de dados, formado por 494 arquivos no formato pdf (como livros, artigos e slides), tem no total 1,11 GB e foi derivado de [Nelson 2025]. Os arquivos estão disponíveis no repositório do projeto em `data/pdfs` [Eduardo 2025].

A especificação do trabalho exigia o uso de um conjunto de dados com um tamanho maior que 1 GB para os experimentos. Entretanto, isto não foi possível em Haskell, devido à memória utilizada durante a execução exceder aquela disponibilizada pela máquina. Portanto, para os experimentos feitos com esta linguagem, foi utilizado um subconjunto de apenas 106 Mb com 26 arquivos, pois, apesar de um conjunto maior (por volta de 300 Mb) não causar problemas relacionados a memória, o tempo de execução não é prático para a condução de vários experimentos. Este conjunto de dados está presente em `data/subset` [Eduardo 2025].

2.3. Java

Nas execuções em Java, foi utilizado um JDK com a versão 24 num projeto Maven com versão 11.

As versões implementadas e documentadas neste relatório são: Uma versão serial (Serial), uma com somente *Platform threads* (Platform), uma com somente *Virtual Threads* (Virtual), uma híbrida em relação às duas anteriores (Hybrid), uma com variáveis atômicas (Atomic) e uma com variáveis voláteis (Volatile). Vale notar que as versões Platform, Virtual, Hybrid e Volatile utilizam o recurso *synchronized* para resolução de condições de corrida e que Atomic e Volatile foram baseadas na Hybrid quanto a tipo das threads. Mais detalhes para cada versão estão disponíveis nas suas respectivas seções. Ademais, a especificação do trabalho exigia uma outra versão, a qual utilizaria um *Reentrant Lock*, entretanto não foi possível desenvolvê-la por questões de tempo.

Com relação aos parâmetros presentes nas versões, todas compartilham um denominado **query**, correspondente a *query* do algoritmo BM25 descrito em 1.1 e o diretório do conjunto de dados utilizado (**path**). Adicionalmente, nas versões concorrentes, pode-se especificar o número de threads nas partes do código aonde são utilizadas: em `Main.java`, com relação ao número de threads utilizadas para ler os arquivos do conjunto de dados (**main_n_threads**); Em `BM25.java`, o número de threads para determinar o arquivo com o maior *score* (**bm25_n_threads**); Em `DocumentData.java`, o número de threads para calcular os dados de um documento (**docdata_n_threads**). Além disso, nas versões concorrentes, o conteúdo textual de um arquivo é separado em blocos — cujo tamanho pode ser configurado (**docdata_block_size**) — e distribuído para diferentes threads (a quantidade de threads é **docdata_n_threads**).

Seguem os valores de cada parâmetro utilizado nos experimentos:

- **query**: *cool example*
- **path**: `../.. /data/pdfs`
- **main_n_threads**: 40
- **bm25_n_threads**: 40
- **docdata_n_threads**: 40
- **docdata_block_size**: 5000

As seguintes flags foram utilizadas:

```
-Xms512m -Xmx1g -XX:+UseStringDeduplication
```

Observação: a leitura de pdf's pela biblioteca `pdfbox` gera uma grande quantidade de *warnings* ao encontrar caracteres especiais — geralmente artefatos de alguma formatação especial do documento. Portanto, incluí a linha `System.setErr(new PrintStream(OutputStream.nullOutputStream()))` para suprimir esses e, colateralmente, quaisquer outros *warnings* em todas as versões Java.

2.3.1. Configuração dos testes JCTest

- Versão da ferramenta: 0.16
- Arquivos utilizados:
 - *A Framework for Building Extensible C++ Class Libraries - 1993.pdf*

– *CUDA Thread-Indexing Cheatsheet.pdf*

- Quantidade de Agentes em cada teste: 2

Vale notar que dentre as versões concorrentes, somente foram testadas Atomic, Hybrid e Volatile, visto que Platform e Virtual não apresentam comportamento diferente de Hybrid para resolução de condições de corrida.

2.3.2. Configuração dos testes JMH

- Versão da ferramenta: 1.35
- Arquivo utilizado: *CUDA Thread-Indexing Cheatsheet.pdf*
- *Warmup*: 3 iterações, cada uma com 10 segundos
- *Measurement*: 8 iterações, cada uma com 10 segundos
- *Timeout*: 10 minutos por iteração
- *Threads*: 1
- *Benchmark mode*: *Throughput* com *ops/ms*

Os seguintes métodos divergem da configuração apresentada acima da seguinte maneira:

- `JMHTests.DocumentData.BenchTest.test_constructor`
 - *Measurement*: 4 iterações
 - *Benchmark mode*: *ops/s*
- `JMHTests.DocumentData.BenchTest.test_cleanString`
 - *Benchmark mode*: *ops/s*
- `JMHTests.DocumentData.BenchTest.test_counter_spawn_thread`
 - *Benchmark mode*: *ops/s*
- `JMHTests.DocumentData.BenchTest.test_get_token_frequency`
 - *Benchmark mode*: *ops/s*

2.3.3. Configuração dos teste JMeter

- Versão da ferramenta: 5.6.3
- Número de threads inicial: 8
- Delay inicial: -70s
- Tempo de *Startup*: 600s
- Manter a carga por: 10s
- Tempo de *Shutdown*: 10s
- Ação ao ser tomada após um erro no sampler: Continuar

Os experimentos foram feitos sobre os arquivos presentes em `data/pdfs` [Eduardo 2025].

2.4. Haskell

Nas execuções em Haskell, foi utilizado o compilador GHC na versão 8.10.7.

As versões implementadas e documentadas neste relatório são: Uma versão serial (Serial), uma que utiliza somente forkIO (Virtual), uma com somente forkOS (Platform), uma híbrida em relação às duas anteriores (Hybrid) e uma que utiliza *Software Transactional Memory* (STM). Vale notar que todas as versões menos a STM utilizam `MVar`'s para resolução de condições de corrida. Além disso, uma versão com `IORef` não foi feita por questões de tempo.

Com relação aos parâmetros presentes nas versões, todas compartilham **query** e **path**, de maneira similar às versões em Java. Adicionalmente, as versões concorrentes possuem os parâmetros **n.threads.doc** e **n.threads.score**, os quais definem, respectivamente, o número de threads para o processamento dos arquivos e para o cálculo do *score*.

As ferramentas escolhidas para a avaliação da implementação em Haskell são: GHC profiler [GHC Profiling 2025], ghc-events-analyze [ghc-events analyze 2014] e Criterion [Criterion 2024]. A primeira faz uma avaliação do programa durante a sua execução — de maneira similar ao JFR — a segunda utiliza este profile gerado e mostra distribuição de processamento entre os eventos e threads durante a execução, já a terceira pode ser utilizada tanto para Microbenchmark quanto para Macrobenchmark. Ademais, vale citar que a avaliação quanto a condições de corrida não foi feita para esta linguagem, entretanto, na pesquisa feita, identificou-se que a ferramenta dejafu [dejafu 2025] parece adequada para este tipo de análise.

Seguem os valores de cada parâmetro utilizado nos experimentos:

- **query:**
 - GHC profiler: *partial function*
 - ghc-events-analyze: *partial function*
 - Criterion: *in out idk*
- **path:** `../../data/subset`
- **n.threads.doc:** 13
- **n.threads.score:** 13

O programa foi compilado com o comando:

```
ghc -O2 -threaded -package stm -package text  
-package deepseq -package directory --make Main.hs
```

E executado com: `./Main +RTS -N6`.

A flag `-N6` indica que o programa vai utilizar 6 cores presentes na máquina ao lidar com threads.

2.4.1. GHC profiler

- Versão do GHC utilizada para este profile em específico: 9.8.4 ¹

¹Utilizei uma versão diferente pois tive problemas de versionamento dos pacotes.

2.4.2. ghc-events-analyze

- Versão da ferramenta: 0.2.8

Nesta ferramenta, é possível inserir *trace events* em diferentes partes do código, os quais são mostrados na visualização gerada (porém, devido a dificuldade de encaixar esta no documento, será mostrado apenas dados textuais). Isto ajuda a determinar a distribuição de tempo de processamento no código. Logo, decidi colocar *trace events* dividindo o código em três partes: coleta de arquivos (GatheringDocs), processamento dos documentos (ProcessingDocs) e cálculo do *score* (CalculatingScore).

2.4.3. Criterion

- Versão da ferramenta: 1.6.4.0
- Arquivo utilizado: *CUDA Thread-Indexing Cheatsheet.pdf*

Vale notar que o Microbenchmark foi somente feito para as versões Hybrid, STM e Serial, visto que Virtual e Platform possuem Microbenchmarks idênticas às de Hybrid.

3. Implementação Serial

Vale notar que a explicação do código, nesta e em outras versões, somente vai entrar em detalhes sobre aspectos concorrentes, visto que este é o escopo do relatório.

3.1. Java

Considerando as classes presentes na Figura 2, o fluxo do código pode ser descrito, de maneira simplificada, da seguinte forma: Os dados de entrada são definidos em `Main`, a leitura e extração de dados relevantes dos documentos é feita em `DocumentData` e o cálculo do *score* para cada documento é feito em `BM25`.

No método `main` (Figura 3), observa-se o fluxo do código citado no parágrafo anterior. O processamento dos arquivos é feito na linha 35, ao passar o caminho de um documento para o construtor da classe `DocumentData` (discutida posteriormente). Em seguida, na mesma linha, um objeto da classe `DocumentData`, a qual tem os dados relevantes do documento (nome, número de tokens e frequência de cada token da *query*), é adicionada a uma lista de um objeto da classe `BM25`. Com base nessa lista, é calculado o *score* de cada documento e, por fim, o documento com maior *score* é retornado pelo método `get_most_relevant_doc` (detalhado na Figura 5). No parágrafo seguinte, irei descrever o funcionamento da classe `BM25`.

Na Figura 4, no método `add` (aquele visto na linha 35 do método `main`), nota-se que o método `update_IDF` é chamado cada vez. Isto é porque, no algoritmo `BM25`, o valor retornado pela função `IDF`, utilizado no cálculo de cada *score*, depende dos dados de todos os documentos. Portanto, é necessário calcular este valor antes do *score*. Por isto este cálculo só é feito após a adição de todos os documentos à lista `docs` — acompanhada pelo cálculo do `IDF`. No parágrafo seguinte, será descrito o processamento dos documentos em `DocumentData`.

Na classe `DocumentData`, o processamento do documento passado como parâmetro é feito logo no construtor, armazenando apenas dados relevantes no objeto

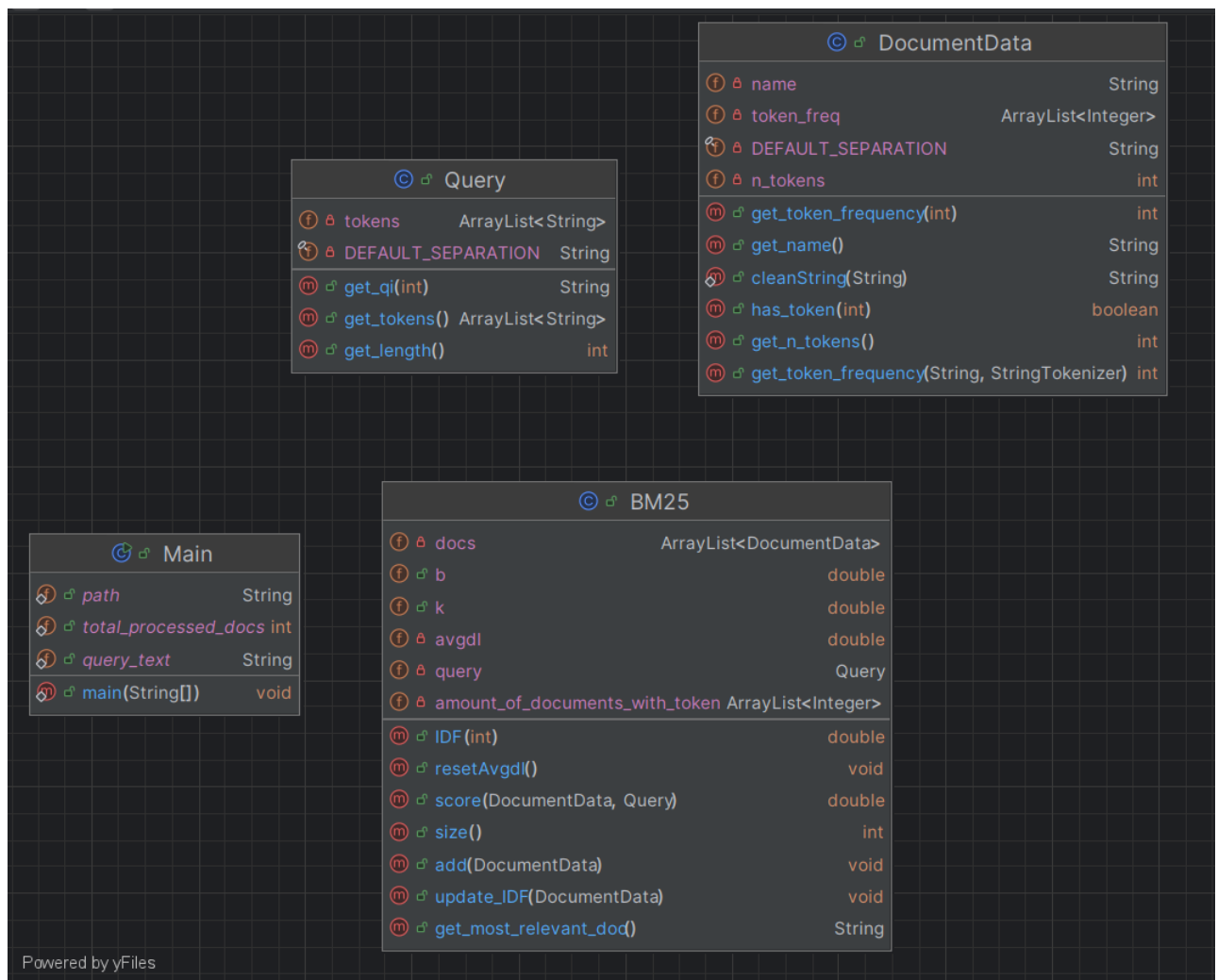


Figura 2: Diagrama de classes da versão Serial em Java

```

22  ▶ static public void main(String args[]) {
23
24      File[] files = (new File(path)).listFiles();
25
26      if (files != null) {
27
28          Query query = new Query(query_text);
29          BM25 bm25 = new BM25(query);
30
31          for (File file : files) {
32              String filename = file.getName();
33
34              try {
35                  bm25.add(new DocumentData( filepath: path + filename, query));
36                  System.out.println("Successfully read the file " + filename);
37              }
38              catch (IOException e) {
39                  System.out.println("Could not read the file " + filename);
40              }
41          }
42
43          System.out.println("Processed " + bm25.size() + " out of " + files.length + " files");
44          System.out.println("Most relevant doc: " + bm25.get_most_relevant_doc());
45          total_processed_docs = bm25.size();
46      }
47      else {
48          System.out.println("No documents found");
49      }
50  }

```

Figura 3: Método main na versão Serial em Java

```

35
36      public void add(DocumentData doc) {
37          this.docs.add(doc);
38          this.avgdL += doc.get_n_tokens();
39          update_IDF(doc);
40      }
41
42      public void update_IDF(DocumentData doc) { 2 usages
43          for (int i = 0; i < query.get_length(); i++) {
44              if (doc.has_token(i)) {
45                  amount_of_documents_with_token.set(i,
46                      amount_of_documents_with_token.get(i) + 1);
47              }
48          }
49      }
50

```

Figura 4: Métodos add e update_IDF na versão Serial em Java, presente na classe BM25.

```

59     public String get_most_relevant_doc() { 2 usages
60         this.avgdl *= 1.0 / docs.size();
61
62         DocumentData most_relevant_doc = null;
63         double highest_score = Double.NEGATIVE_INFINITY;
64
65         for (DocumentData doc : docs) {
66
67             double scr = score(doc, query);
68             if (scr >= highest_score) {
69                 most_relevant_doc = doc;
70                 highest_score = scr;
71             }
72         }
73
74         return most_relevant_doc == null ? "" : most_relevant_doc.get_name();
75     }
76 }

```

Figura 5: Método `get_most_relevant_doc` na versão Serial em Java, presente na classe `BM25`.

```

18 @ public DocumentData(String filepath, Query query) throws IOException { 4 usages
19
20     this.name = filepath;
21
22     PDDocument document = PDDocument.load(new File(filepath));
23     var my_text = (new PDFTextStripper()).getText(document);
24     document.close();
25     StringTokenizer tokenizer = new StringTokenizer(my_text, DEFAULT_SEPARATION);
26
27     this.n_tokens = tokenizer.countTokens();
28     this.token_freq = new ArrayList<>();
29     for (String token : query.get_tokens()) {
30         this.token_freq.add(get_token_frequency(token,
31             new StringTokenizer(my_text, DEFAULT_SEPARATION)));
32     }
33 }

```

Figura 6: Construtor da classe `DocumentData` na versão Serial em Java.

da classe. Tal estratégia tem como objetivo reduzir o mais rápido o possível a quantidade de memória utilizada pelo programa. A leitura do arquivo é feita com auxílio da biblioteca `pdfbox` (em sua versão 2.0.32) e a tokenização do texto extraído, pela classe `StringTokenizer` presente em `java.util`.

3.1.1. Resultados da Implementação

Observação:

- Houve tentativas de avaliação com o `ParallelGC`, entretanto estas frequentemente utilizavam 100% do CPU da máquina, tornando sua execução extremamente lenta. Logo, devido a uma questão de praticidade e, pelo resultado inferior claro desta versão, decidi excluí-la do relatório.
- O GC utilizado na Microbenchmark foi o `G1GC`. Não se julgou necessário realizá-la com GC's diferentes.

3.1.2. Avaliação com Microbenchmark

Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.BM25_BenchTest.test_IDF	thrpt	8	15,152 ± 0,103		ops/ms
JMHTests.BM25_BenchTest.test_add	thrpt	8	7,364 ± 0,023		ops/ms
JMHTests.BM25_BenchTest.test_constructor	thrpt	8	4,826 ± 0,020		ops/ms
JMHTests.BM25_BenchTest.test_get_most_relevant_doc	thrpt	8	2,763 ± 0,008		ops/ms
JMHTests.BM25_BenchTest.test_score	thrpt	8	3,382 ± 0,016		ops/ms
JMHTests.BM25_BenchTest.test_update_IDF	thrpt	8	10,607 ± 3,543		ops/ms
JMHTests.DocumentData_BenchTest.test_cleanString	thrpt	8	242,380 ± 4,164		ops/s
JMHTests.DocumentData_BenchTest.test_constructor	thrpt	4	0,178 ± 0,002		ops/s
JMHTests.DocumentData_BenchTest.test_get_token_frequency	thrpt	8	54,125 ± 1,322		ops/s
JMHTests.Query_BenchTest.test_constructor	thrpt	8	1,253 ± 0,020		ops/ms

Figura 7: Resultados do JMH para os métodos da versão Serial em Java.

3.1.3. Avaliação com Macrobenchmark

```
-- G1GC
Benchmark                               Mode  Cnt      Score      Error  Units
JMHTests.Main_BenchTest.test_main      avgt    8  136376,352 ± 3410,230  ms/op

-- ZGC
Benchmark                               Mode  Cnt      Score      Error  Units
JMHTests.Main_BenchTest.test_main      avgt    8  138627,469 ± 5910,498  ms/op
```

Figura 8: Tempo médio de execução . Versão Serial em Java

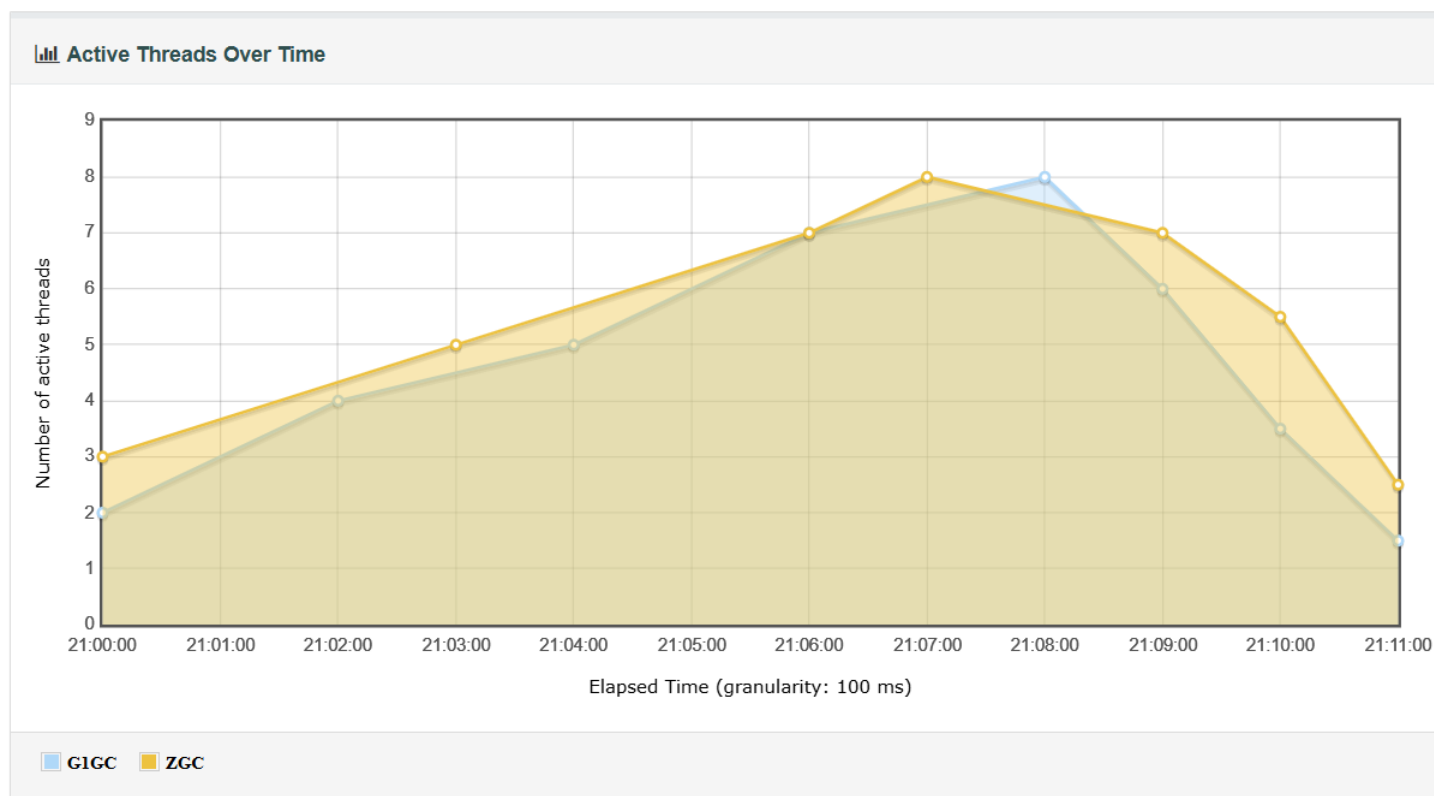


Figura 9: Número de threads ativas × tempo . Versão Serial em Java

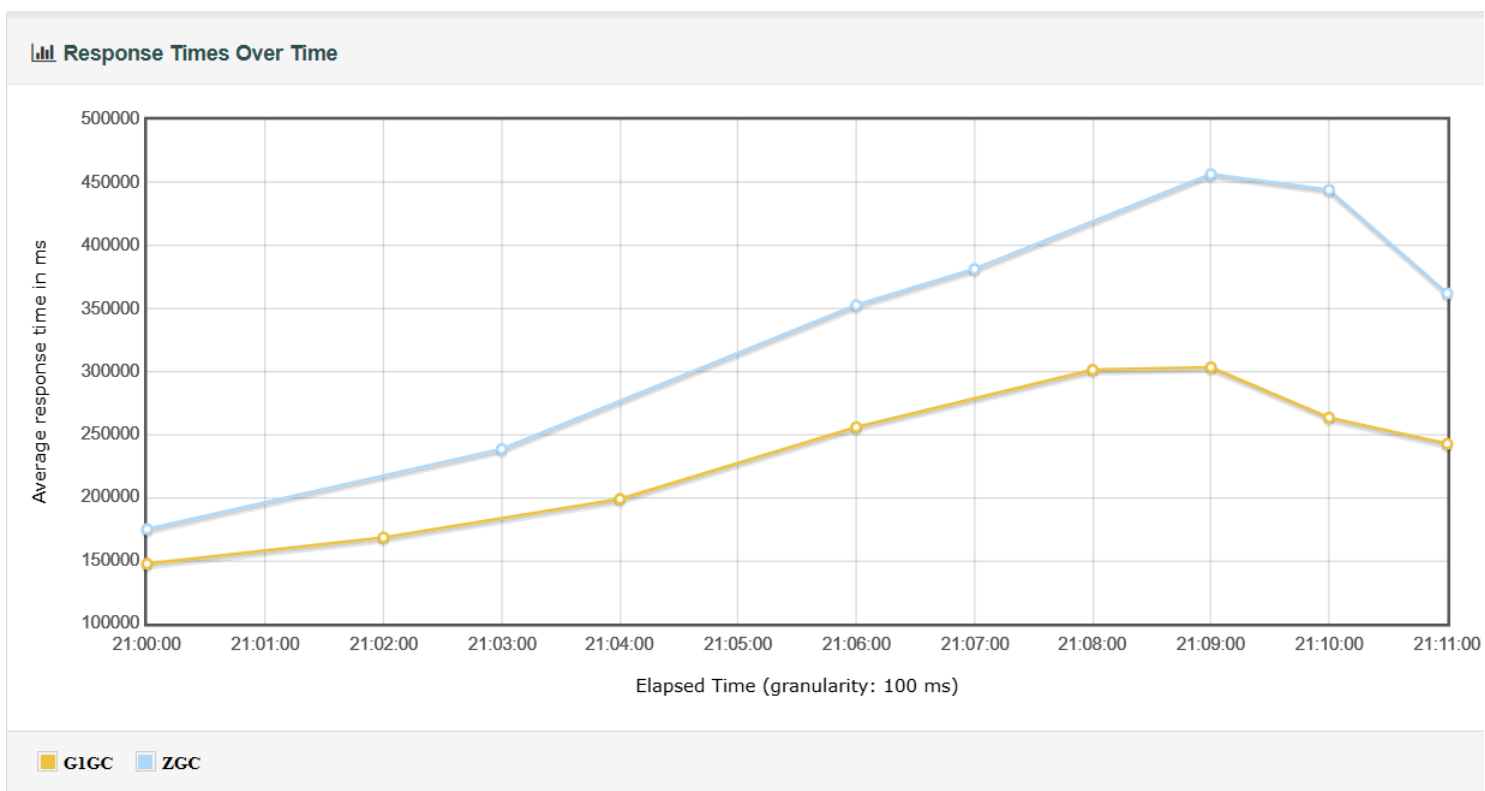


Figura 10: Tempo de resposta × tempo. Versão Serial em Java



Figura 11: *Throughput* × tempo. Versão Serial em Java

Statistics												GCG1	
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ↕	FAIL ↕	Error % ↕	Average ↕	Min ↕	Max ↕	Median ↕	90th pct ↕	95th pct ↕	99th pct ↕	Transactions/s ↕	Received ↕	Sent ↕
Total	14	0	0.00%	246438.00	147496	306005	256004.00	304220.50	306005.00	306005.00	0.02	0.00	0.00
Serial Test Sample	14	0	0.00%	246438.00	147496	306005	256004.00	304220.50	306005.00	306005.00	0.02	0.00	0.00

Statistics												ZGC	
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ↕	FAIL ↕	Error % ↕	Average ↕	Min ↕	Max ↕	Median ↕	90th pct ↕	95th pct ↕	99th pct ↕	Transactions/s ↕	Received ↕	Sent ↕
Total	11	0	0.00%	358426.45	174993	456877	375080.00	454982.40	456877.00	456877.00	0.01	0.00	0.00
Serial Test Sample	11	0	0.00%	358426.45	174993	456877	375080.00	454982.40	456877.00	456877.00	0.01	0.00	0.00

Figura 12: Estatísticas. Versão Serial em Java

Observações:

- As versões com G1GC e ZGC foram avaliadas em grupos de testes separados e seus gráficos sobrepostos posteriormente com a ajuda de um script de autoria própria.
- O início e o fim do tempo de execução foi normalizado para a sobreposição dos gráficos.
- O número 21 no eixo x se refere ao dia em que a medição foi feita.

3.1.4. Avaliação de Profile

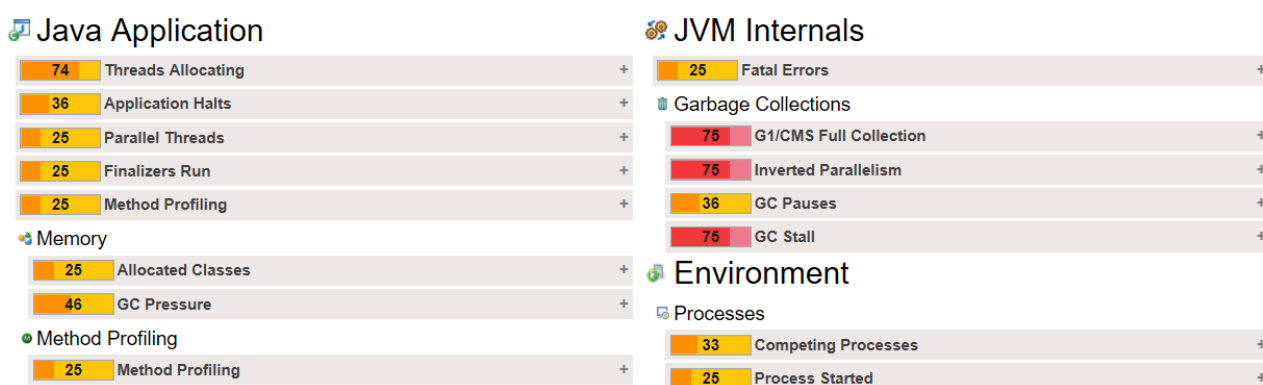


Figura 13: Resumo do JMC. G1GC. Versão Serial em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	622	13,4 %
java.util.TreeMap.getCeilingEntry(Object)	463	10 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	316	6,83 %
java.util.HashMap.getNode(Object)	234	5,06 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	219	4,74 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	207	4,48 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	163	3,52 %
java.util.TreeMap.put(Object, Object, boolean)	140	3,03 %
java.util.TreeMap.fixAfterInsertion(TreeMap\$Entry)	109	2,36 %
java.math.BigDecimal.<init>(char[], int, int, MathContext)	91	1,97 %
org.apache.pdfbox.pdfparser.PDFStreamParser.parseNextToken()	87	1,88 %
java.util.concurrent.ConcurrentHashMap.get(Object)	83	1,79 %

Figura 14: Hot methods. G1GC. Versão Serial em Java

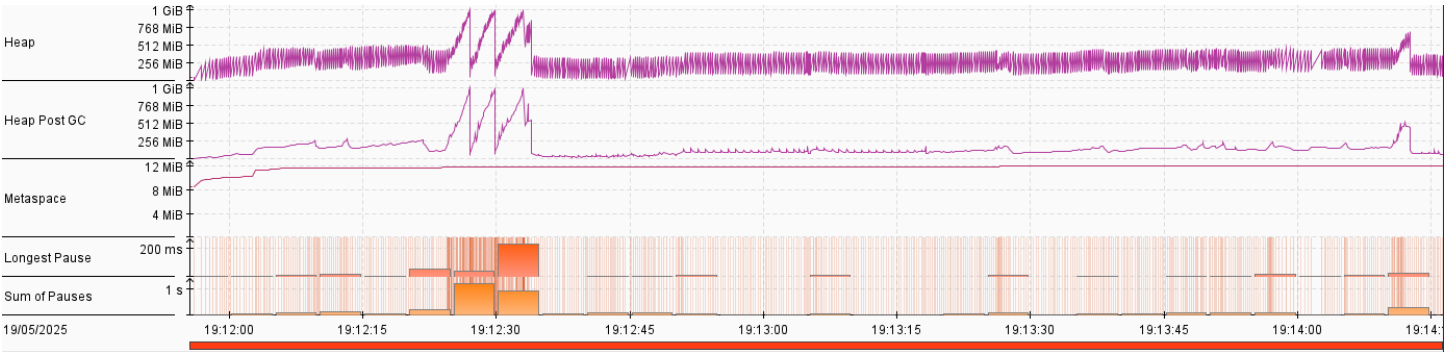


Figura 15: *Garbage Collector*. G1GC. Versão Serial em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	617	GC Count	110
Average GC Time	7,290 ms	Average GC Time	77,069 ms
Maximum GC Time	61,754 ms	Maximum GC Time	691,668 ms
Total GC Time	4,498 s	Total GC Time	8,478 s
All Collections Total Time		All Collections Pause Time	
GC Count	727	Average Pause	6,136 ms
Average GC Time	17,848 ms	Longest Pause	235,157 ms
Maximum GC Time	691,668 ms	Sum of Pauses	5,081 s
Total GC Time	12,975 s		

Figura 16: *Garbage Collector Summary*. G1GC. Versão Serial em Java

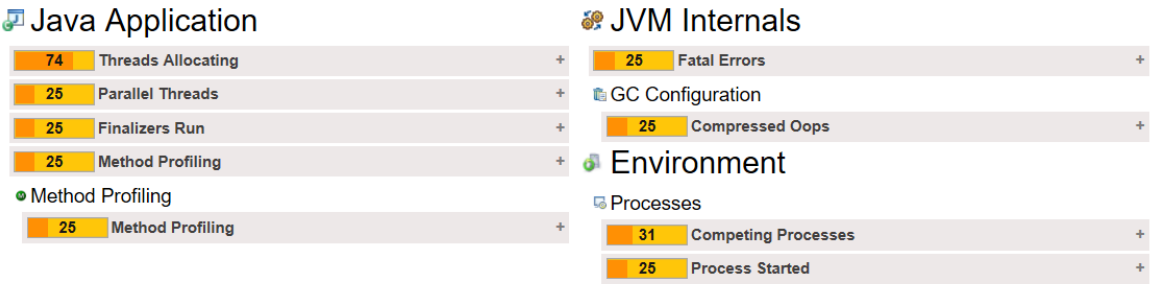


Figura 17: Resumo do JMC. ZGC. Versão Serial em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	752	14,4 %
java.util.TreeMap.getCeilingEntry(Object)	528	10,1 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	335	6,43 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	276	5,29 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	204	3,91 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	198	3,8 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	150	2,88 %
org.apache.pdfbox.pdfparser.PDFStreamParser.parseNextToken()	111	2,13 %
java.util.HashMap.getNode(Object)	106	2,03 %
java.util.concurrent.ConcurrentHashMap.get(Object)	105	2,01 %
java.lang.Float.compare(float, float)	93	1,78 %

Figura 18: *Hot methods*. ZGC. Versão Serial em Java

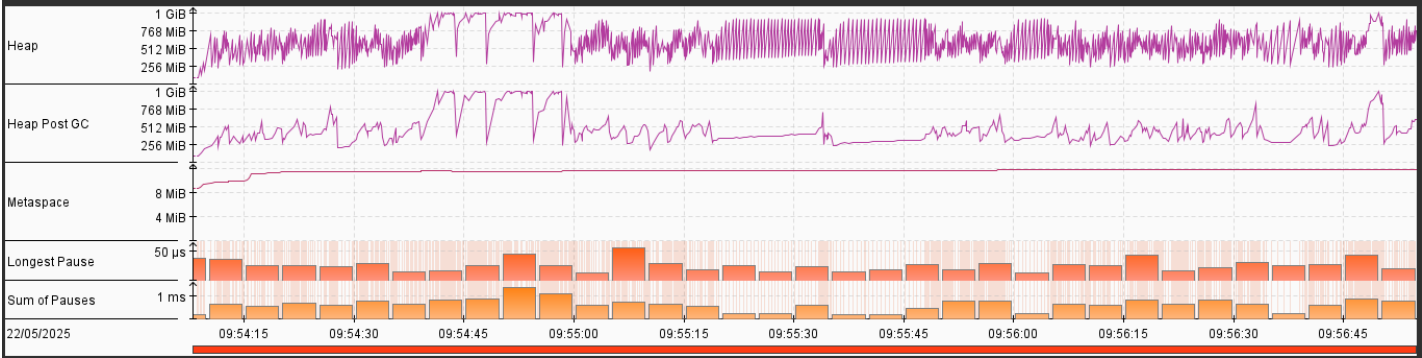


Figura 19: *Garbage Collector*. ZGC. Versão Serial em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	0	GC Count	0
Average GC Time	N/A	Average GC Time	N/A
Maximum GC Time	N/A	Maximum GC Time	N/A
Total GC Time	N/A	Total GC Time	N/A
All Collections Total Time		All Collections Pause Time	
GC Count	645	Average Pause	10,695 μs
Average GC Time	102,429 ms	Longest Pause	56,261 μs
Maximum GC Time	1,740 s	Sum of Pauses	22,813 ms
Total GC Time	1 min 6 s		

Figura 20: *Garbage Collector Summary*. ZGC. Versão Serial em Java

3.2. Haskell

A implementação serial em Haskell possui uma lógica similar àquela presente em Java. Não há classes em Haskell, mas o código está dividido nos seguintes arquivos: `Main.hs`, `BM25.hs`, `DocReader.hs` e `Utils.hs`. Espera-se que os nomes, em conjunto com o que foi apresentado na versão em Java, sejam auto-explicativos.

```
15 main = do
16     ---- Gathering docs
17     files <- getDirectoryContents filepath
18     let pdf_names = filter (is_file_type "pdf") (map (filepath ++) files)
19
20     ---- Processing docs
21     doc_data_list <- mapM (get_doc_data query . tokenizeDoc) pdf_names :: IO [DocumentData]
22
23     let documents = filter (not_empty . name) doc_data_list :: [DocumentData]
24     let n_processed_docs = length documents
25
26     putStrLn $ "Processed " ++ (show n_processed_docs) ++ " out of " ++ (show $ length pdf_names)
27
28     ---- Calculating Score
29
30     let nDocs = fromIntegral n_processed_docs :: Double
31     let avgd1 = get_avgd1 nDocs (map n_tokens doc_data_list) :: Double
32     let idfs = idf nDocs doc_data_list query
33
34     putStrLn $ fst $ get_most_relevant_doc (nDocs, avgd1, idfs) doc_data_list query
35
```

Figura 21: Função main da versão Serial em Haskell

Como comentado no código da Figura 21, pode-se dividir o código em três partes: Coleta de documentos, Processamento dos documentos e Cálculo do *score*. Nos parágrafos seguintes, irei descrever com maiores as duas últimas partes.

O processamento dos documentos é feito aplicando a função composta `get_doc_data query . tokenizeDoc` a uma lista de caminhos para os arquivos pdf, retornando uma lista de elementos do tipo `DocumentData` (este pode ser visto na Figura 22). A função `tokenizeDoc` extrai o texto presente no documento com auxílio da biblioteca `Pdf.Document` e tokeniza seu conteúdo. Em seguida, a função `get_doc_data query` extrai os dados relevantes — os mesmos da versão em Java — do texto tokenizado e os encapsula num `DocumentData`, retornando-o.

Antes do cálculo do *score*, os documentos que não foram processados corretamente — os quais são identificados por não possuírem um nome — são removidos (linha 23 da Figura 21).

Por fim, todos os dados necessários antes do cálculo são extraídos, como: o número total de documentos considerados no cálculo, o tamanho médio de um documento e o IDF para cada token da *query*. Após isto, por meio da função `get_most_relevant_doc` (Figura 23), o cálculo é efetuado e o documento com maior *score* determinado.

```

13 data DocumentData = DocumentData {
14     name :: String,
15     n_tokens :: Int,
16     token_freq :: [(String, Int)]
17 }
18 deriving (Show)

```

Figura 22: Definição do tipo DocumentData

```

10 get_most_relevant_doc :: (Double, Double, [Double]) ->
11     [DocumentData] -> [Token] -> (String, Double)
12 get_most_relevant_doc _ [] _ = ("", 0)
13 get_most_relevant_doc params (doc_data : xs) query = do
14     let score = doc_score params doc_data query
15     let doc_name = name doc_data
16     let best_doc = get_most_relevant_doc params xs query
17     if score > (snd best_doc)
18     then (doc_name, score)
19     else best_doc

```

Figura 23: Função get_most_relevant_doc na versão Serial em Haskell

3.2.1. Resultados da Implementação

Observação:

- Os experimentos foram feitos com apenas o GC padrão de Haskell (GC geracional).

3.2.2. Avaliação com Microbenchmark

A ferramenta Criterion gera um relatório com o gráfico presente na Figura 24, junto com outros detalhes para cada método testado. Não irei mostrar os detalhes por questão de simplicidade, porém estes estão disponíveis em [Eduardo 2025].

Uma importante observação nesta avaliação (e na Microbenchmark das outras versões em Haskell) é a ausência da função `tokenizeDoc` no gráfico mostrado. Esta função foi retirada devido ao seu tempo de execução ser ordem de magnitudes maior que as outras, prejudicando a visualização do gráfico. O tempo de execução médio de `tokenizeDoc` foi de 45.5 ms.

overview

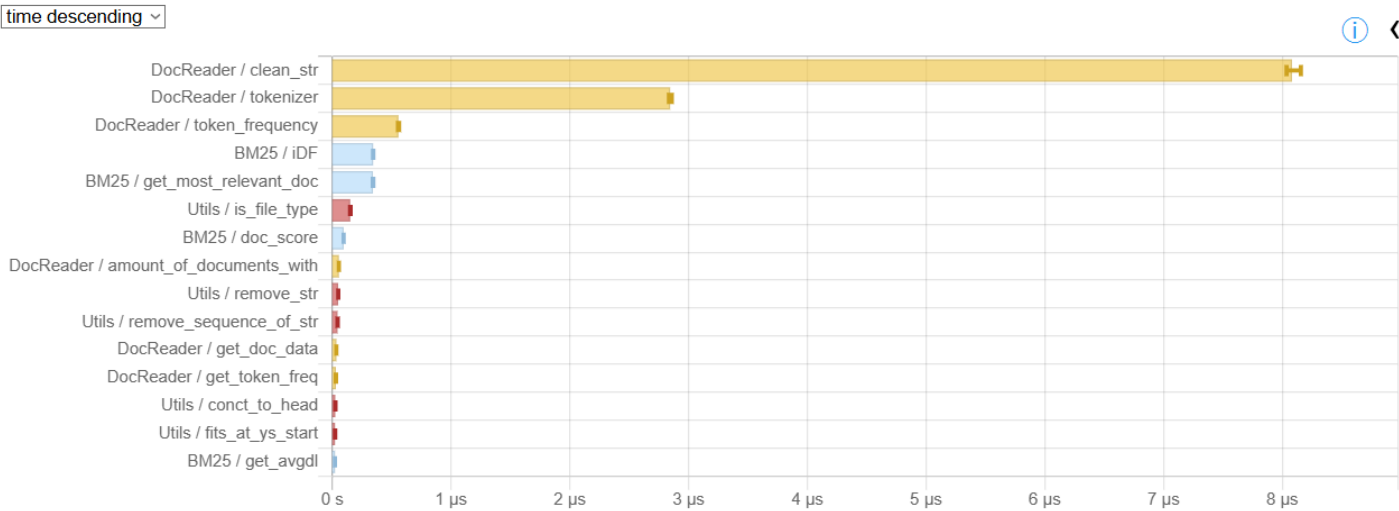
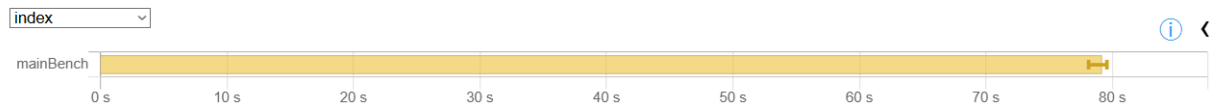


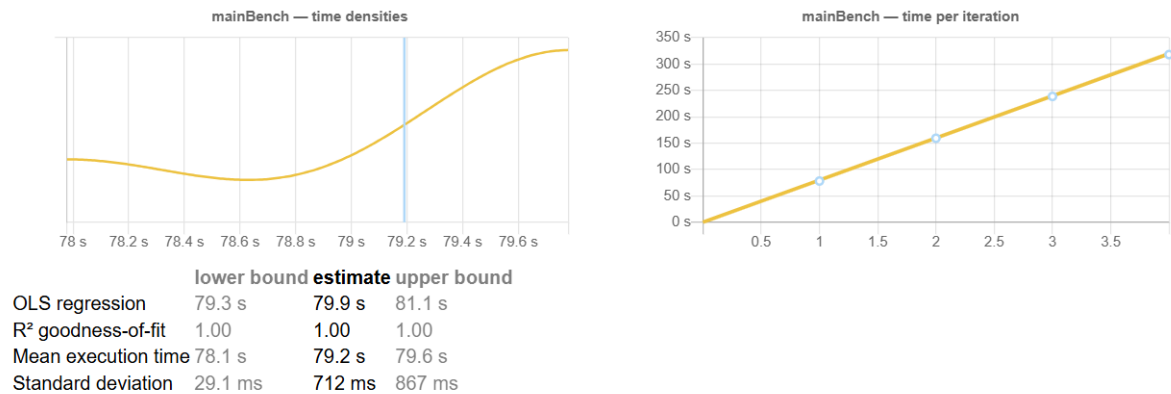
Figura 24: Resultados do Criterion para as funções da versão Serial em Haskell.

3.2.3. Avaliação com Macrobenchmark

overview



mainBench



Outlying measurements have a moderate (18.8%) effect on estimated standard deviation.

Figura 25: Resultados do Criterion para main da versão Serial em Haskell.

Na pesquisa feita por ferramentas de avaliação para Haskell as quais obtêm uma Macrobenchmark parecidas com o JMeter, nada foi encontrado. Portanto, utilizei o Criterion avaliando a função main (Figura 25).

3.2.4. Avaliação de Profile

GC	45054389700ns	45.054s
USER EVENTS (user events are corrected for GC)		
ProcessingDocs	31765102700ns	31.765s
CalculatingScore	9137855800ns	9.138s
GatheringDocs	464700ns	0.000s
TOTAL	40903423200ns	40.903s
THREAD EVENTS		
1	106400ns	0.000s
2	81300ns	0.000s
3	35141549300ns	35.142s
weak finalizer thread:4	26600ns	0.000s
weak finalizer thread:5	2100ns	0.000s
weak finalizer thread:6	2300ns	0.000s
weak finalizer thread:7	15200ns	0.000s
weak finalizer thread:8	5800ns	0.000s
weak finalizer thread:9	6400ns	0.000s
weak finalizer thread:10	3000ns	0.000s
weak finalizer thread:11	9500ns	0.000s
12	6100ns	0.000s
TOTAL	35141814000ns	35.142s

Figura 26: Resultados do ghc-events-analyze para versão Serial em Haskell.

3.3. Discussão

Esta seção primeiro irá abordar as diferenças entre Java e Haskell, em seguida a diferença entre as execuções em Java com G1GC e ZGC.

Em primeiro lugar, com relação ao tempo de execução, observa-se que a média para a versão em Haskell é de 79,2 segundos (Figura 25), já a versão em Java (G1GC), 2,27 minutos (Figura 8). Entretanto, a versão em Haskell utilizou um conjunto de dados de apenas 106 MB, enquanto Java 1,1 GB. Logo, não se pode afirmar ao certo, mas extrapolando linearmente a performance de Haskell, esta é consideravelmente pior que Java, já que, teoricamente, demoraria mais de 10 minutos em um conjunto de dados com 1,1 GB.

Aprofundando a análise, note a Figura 26. Nela está presente o tempo total gasto — na versão em Haskell — pelo GC, o tempo entre os eventos de usuário definidos (em *wall-clock time*) e o tempo gasto pelas threads. Para obter o tempo de execução total, basta somar o tempo de GC mais o tempo das threads (nenhuma criada explicitamente pelo usuário, pois isto se trata da versão Serial). O tempo dos eventos de usuário não entra neste cálculo. Assim, observa-se que o tempo gasto pelo GC é de aproximadamente 56% do tempo total de execução, o que é ineficiente. Logo, um melhor uso do GC, no código ou por parâmetros, ou até a adoção de um outro GC, são recomendações promissoras para a melhoria da eficiência. Em comparação, na versão em Java (G1GC), apenas 3,61% do tempo total de execução é gasto em GC (calculado feito com base na Figura 16 e nos dados do *flight*).

Em seguida, em relação à Microbenchmark (Figura 24), nota-se que o tempo de execução de grande parte dos métodos em Haskell é ordens de magnitude menor do que aqueles em Java. A única exceção é o método `tokenizeDoc` — grosseiramente equivalente ao construtor da classe `DocumentData` em Java — o qual tem um tempo médio de execução de 45,5ms, enquanto o construtor citado demora 178 ms. Entretanto, o que torna `tokenizeDoc` mais lento que os outros em Haskell é o fato dele imprimir algo na tela. Logo, com relação a essa disparidade entre as duas linguagens, suponho que isto se deva a maior simplicidade das funções em Haskell (Figura 27), possuindo um `overhead` bem menor que aquelas em Java. Há também o fator de um programa em Java não ser completamente compilado para linguagem de máquina como em Haskell.

```
private int scanToken(int startPos) {
    int position = startPos;
    while (position < maxPosition) {
        if (!hasSurrogates) {
            char c = str.charAt(position);
            if ((c <= maxDelimCodePoint) && (delimiters.indexOf(c) >= 0))
                break;
            position++;
        } else {
            int c = str.codePointAt(position);
            if ((c <= maxDelimCodePoint) && isDelimiter(c))
                break;
            position += Character.charCount(c);
        }
    }
    if (retDelims && (startPos == position)) {
        if (!hasSurrogates) {
            char c = str.charAt(position);
            if ((c <= maxDelimCodePoint) && (delimiters.indexOf(c) >= 0))
                position++;
        } else {
            int c = str.codePointAt(position);
            if ((c <= maxDelimCodePoint) && isDelimiter(c))
                position += Character.charCount(c);
        }
    }
    return position;
}
```

```
46 tokenizer :: String -> [String]
47 tokenizer "" = []
48 tokenizer (' ':[]) = []
49 tokenizer (' ':xs) = if ((head xs) == ' ') then tokenizer xs else []:(tokenizer xs)
50 tokenizer (x:xs) = conct_to_head x (tokenizer xs)

12 conct_to_head :: Char -> [String] -> [String]
13 conct_to_head c [] = [[c]]
14 conct_to_head c (x:xs) = ([c] ++ x) : xs
```

Figura 27: Em cima está apenas parte do código utilizado para tokenizar uma string em Java (retirado de uma biblioteca). Em baixo, o código completo para esta mesma função em Haskell.

Em suma, a versão de Java superou drasticamente a versão em Haskell na Avaliação de Profile e Macrobenchmark. Entretanto, com relação à Microbenchmark, Haskell se saiu melhor.

Agora, com relação à comparação entre as execuções em Java com dois GC diferentes, G1GC e ZGC, observa-se que o ZGC teve uma melhor performance em geral. Notavelmente, ficou para trás em relação ao tempo de execução: 2,27 minutos G1GC e 2,31 minutos para ZGC (Figura 8). Suponho que a razão disso é o trabalho concorrente feito pelo ZGC, o qual acaba utilizando CPU de maneira demasiada, tornando a execução do programa mais lenta.

- A execução com ZGC apresentou uma avaliação de profile menos problemática (Figuras 13 e 17).
- O uso de GC com o ZGC foi de apenas 0,01% (Figura 20, em comparação à 3,61% do G1GC (Figura 16).
- O gráfico do uso da Heap do G1GC aparenta ter quedas menores, porém mais frequentes (Figuras 15 e 19).

Ademais, ambas versões apresentaram *Hot methods* similares (Figuras 14 e 18), com `java.io.FilterInputStream.read()` sendo o método mais utilizado. Este método é essencialmente o um método de leitura de dados, provavelmente utilizado pela biblioteca `pdfbox`, a qual foi utilizada para a leitura dos documentos. Em adição, nota-se que os métodos mais populares tendem a ser desta mesma biblioteca. Portanto, para a melhoria da eficiência, recomenda-se investigar a performance de outras bibliotecas que exercem a mesma função.

Por fim, quanto ao *Throughput* e *Response Time*, percebe-se que a execução com ZGC teve um desempenho pior, algo esperado devido a sua execução mais lenta. Dessa forma, como visto na Figura 12, o *Response Time* médio do G1GC é 68,7% daquele do ZGC e o *Throughput* o dobro.

4. Implementação Concorrente – Hybrid

4.1. Java

A implementação Hybrid, bem como as outras concorrentes, se assemelha bastante à implementação Serial. A principal diferença está no uso de threads em três locais do código: leitura de arquivos, processamento de um documento e cálculo do *score*. Em cada uma dessas regiões há um método `spawn_thread` definido, o qual contém o código executado de maneira concorrente.

Primeiro, irei falar sobre a implementação concorrente na leitura de arquivos (Figura 28). Nela, o trabalho é distribuído igualmente entre as threads, com cada uma sendo responsável pela leitura de um conjunto de arquivos. A leitura é feita da mesma maneira que a versão Serial, como pode-se observar na linha 71 da Figura 28. Por fim, o código é executado em uma *Virtual Thread* (linha 80), já que esta é melhor adaptada para operações que envolvem IO intensivo, e as condições de corrida são evitadas pelo uso de um `synchronized` no método `add` (presente na linha 71).

Com relação ao uso de threads no processamento de um documento, observe a Figura 30. Nela, nota-se que o texto do documento — extraído no construtor como na versão Serial — é tokenizado em threads separadas, com a sincronização feita por um

```

64     private static void spawn_thread(
65         File[] files, String path, Query query, BM25 bm25, CountdownLatch controller) {
66
67         Runnable runnable = () -> {
68             for (File file : files) {
69                 String filename = file.getName();
70                 try {
71                     bm25.add(new DocumentData( filepath: path + filename, query));
72                     System.out.println("Successfully read the file " + filename);
73                 } catch (IOException e) {
74                     System.out.println("Could not read the file " + filename);
75                 }
76             }
77             controller.countDown();
78         };
79
80         Thread.ofVirtual().start(runnable);
81     }
82
83 }

```

Figura 28: Método `spawn_thread` na versão Hybrid em Java

```

34     Query query = new Query(query_text);
35     BM25 bm25 = new BM25(query);
36
37     final int actual_n_threads = Math.min(n_threads, files.length);
38     CountdownLatch controller = new CountdownLatch(actual_n_threads);
39
40     for (int i = 0; i < actual_n_threads; i++) {
41         File[] file_batch = create_batch(i, actual_n_threads, files);
42         spawn_thread(file_batch, path, query, bm25, controller);
43     }
44
45     try {
46         controller.await();
47     }
48     catch (InterruptedException e) {
49         System.out.println("One or more threads have been interrupted in Hybridv.Main");
50     }
51
52     System.out.println("Processed " + bm25.size() + " out of " + files.length + " files");
53     System.out.println("Most relevant doc: " + bm25.get_most_relevant_doc());

```

Figura 29: Uso do método `spawn_thread` na main na versão Hybrid em Java

CountDownLatch. Ademais, o método `spawn_thread` desta parte pode ser visto na Figura 31. Este pertence à classe `Counter`, utilizada exclusivamente para este método e declarada dentro da classe `DocumentData`. A lógica desta classe, em conjunto com o método `get_token_frequency`, constitui em utilizar threads separadas para contar a frequência de um token num texto. Utiliza-se *Platform threads* por isto não envolver IO. Por fim, as condições de corrida são evitadas pelo uso de um `synchronized` nos métodos `increment` e `get`.

```
67 @      public int get_token_frequency(String token, ArrayList<StringTokenizer> sts) { 2 usages
68
69     /*
70      Reads each chunk of the text in a separate thread
71     */
72
73     final int actual_n_threads = Math.min(n_threads, sts.size());
74     final CountDownLatch controller = new CountDownLatch(actual_n_threads);
75     Counter counter = new Counter(controller);
76
77     for (int i = 0; i < actual_n_threads; i++) {
78         ArrayList<StringTokenizer> tokenizer_batch = create_batch(i, actual_n_threads, sts);
79         counter.spawn_thread(tokenizer_batch, token);
80     }
81
82     try {
83         controller.await();
84     }
85     catch (InterruptedException e) {
86         System.out.println("One or more threads have been interrupted in DocumentData");
87     }
88
89     return counter.get();
90 }
```

Figura 30: Método `get_token_frequency` na na versão Hybrid em Java

Com relação ao uso de threads no cálculo do *score*, observe a Figura 32. Nela, nota-se que o uso de threads é feito pela classe `Auction` (Figura 33). A lógica desta classe, em conjunto com o método `get_most_relevant_doc`, é calcular o *score* de cada documento ao mesmo tempo que se realiza um "leilão", de forma a determinar o documento com maior *score*. A lógica principal pode ser vista no método `spawn_thread` (Figura 33). Por fim, também utiliza-se *Platform threads* e as condições de corrida são evitadas pelo uso de um `synchronized`.

```

108 @ public static class Counter { 4 usages 2 inheritors
109     private int total = 0; 2 usages
110     protected final CountDownLatch controller; 3 usages
111
112 >     public Counter(CountDownLatch controller) { this.controller = controller; }
115
116 >     public synchronized void increment() { total++; }
119
120 >     public synchronized int get() { return total; }
123
124     public void spawn_thread(ArrayList<StringTokenizer> sts, String token) { 2 usages
125         Runnable runnable = () -> {
126             for (StringTokenizer st : sts) {
127                 while (st.hasMoreTokens())
128                     if (cleanString(st.nextToken()).equalsIgnoreCase(token))
129                         increment();
130             }
131             controller.countDown();
132         };
133         Thread.ofPlatform().start(runnable);
134     }
135 }

```

Figura 31: Classe Counter na versão Hybrid em Java

```

61 public String get_most_relevant_doc() { 2 usages
62
63     /*
64         Calculates the score for each doc on a separate thread
65     */
66
67     this.avgdl *= 1.0 / docs.size();
68
69     final int actual_n_threads = Math.min(n_threads, docs.size());
70     final CountDownLatch controller = new CountDownLatch(actual_n_threads);
71     Auction auction = new Auction(controller);
72
73     for (int i = 0; i < actual_n_threads; i++) {
74         ArrayList<DocumentData> doc_batch = create_batch(i, actual_n_threads, docs);
75         auction.spawn_thread(doc_batch, query);
76     }
77
78     try {
79         controller.await();
80     }
81     catch (InterruptedException e) {
82         System.out.println("One or more threads have been interrupted in BM25");
83     }
84
85     DocumentData most_relevant_doc = auction.get_highest_bidder();
86     return most_relevant_doc == null ? "" : most_relevant_doc.get_name();
87 }

```

Figura 32: Método get_most_relevant_doc na versão Hybrid em Java

```

98  @ public class Auction { 4 usages 2 inheritors
99      public DocumentData highest_bidder = null; 2 usages
100     protected double highest_bidder_score = Double.NEGATIVE_INFINITY; 3 usages
101     protected final CountDownLatch controller; 3 usages
102
103     > public Auction(CountDownLatch controller) { this.controller = controller; }
106
107     public synchronized void challenge_highest_bidder(Double bid, DocumentData doc) { 4 usages
108         if (bid >= highest_bidder_score) {
109             highest_bidder_score = bid;
110             highest_bidder = doc;
111         }
112     }
113
114     > public DocumentData get_highest_bidder() { return highest_bidder; }
117
118     public void spawn_thread(ArrayList<DocumentData> docs, Query query) { 2 usages
119         Runnable runnable = () -> {
120             for (DocumentData doc : docs) {
121                 double scr = score(doc, query);
122                 challenge_highest_bidder(scr, doc);
123             }
124             controller.countDown();
125         };
126         Thread.ofPlatform().start(runnable);
127     }
128 }

```

Figura 33: Classe Auction na versão Hybrid em Java

4.1.1. Resultados da Implementação

Ver as observações em 3.1.1.

4.1.2. Avaliação de Condição de Corrida

All tests

JCStressTests.BM25_JCStressTest.BM25_add_StressTest	10 ³	PASSED
JCStressTests.BM25_JCStressTest.BM25_challenge_highest_bidder_StressTest	10 ³	PASSED
JCStressTests.DocumentData_JCStressTest.DocumentData_get_token_frequency_StressTest	10 ⁸	PASSED

Figura 34: Resultados do JCStress para a versão Hybrid em Java.

4.1.3. Avaliação com Microbenchmark

Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.BM25_BenchTest.test_IDF	thrpt	8	15,109 ±	0,109	ops/ms
JMHTests.BM25_BenchTest.test_add	thrpt	8	3,777 ±	0,067	ops/ms
JMHTests.BM25_BenchTest.test_auction_challenge_highest_bidder_doc	thrpt	8	3,395 ±	0,159	ops/ms
JMHTests.BM25_BenchTest.test_auction_spawn_thread	thrpt	8	0,122 ±	0,001	ops/ms
JMHTests.BM25_BenchTest.test_constructor	thrpt	8	4,847 ±	0,053	ops/ms
JMHTests.BM25_BenchTest.test_get_most_relevant_doc	thrpt	8	0,003 ±	0,001	ops/ms
JMHTests.BM25_BenchTest.test_score	thrpt	8	3,041 ±	0,277	ops/ms
JMHTests.BM25_BenchTest.test_update_IDF	thrpt	8	9,575 ±	1,394	ops/ms
JMHTests.DocumentData_BenchTest.test_cleanString	thrpt	8	247,883 ±	0,703	ops/s
JMHTests.DocumentData_BenchTest.test_constructor	thrpt	4	0,162 ±	0,002	ops/s
JMHTests.DocumentData_BenchTest.test_counter_spawn_thread	thrpt	8	85,044 ±	4,068	ops/s
JMHTests.DocumentData_BenchTest.test_get_token_frequency	thrpt	8	25,272 ±	0,277	ops/s
JMHTests.DocumentData_BenchTest.test_get_tokenizers	thrpt	8	57,426 ±	0,303	ops/ms
JMHTests.Query_BenchTest.test_constructor	thrpt	8	1,127 ±	0,010	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_Array	thrpt	8	2,670 ±	0,114	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_ArrayList	thrpt	8	5,928 ±	0,117	ops/ms

Figura 35: Resultados do JMH para os métodos da versão Hybrid em Java.

4.1.4. Avaliação com Macrobenchmark

-- G1GC						
Benchmark	Mode	Cnt	Score	Error	Units	
JMHTests.Main_BenchTest.test_main	avgt	8	45262,479 ±	1402,659	ms/op	
-- ZGC						
Benchmark	Mode	Cnt	Score	Error	Units	
JMHTests.Main_BenchTest.test_main	avgt	8	47551,678 ±	2639,907	ms/op	

Figura 36: Tempo médio de execução . Versão Hybrid em Java

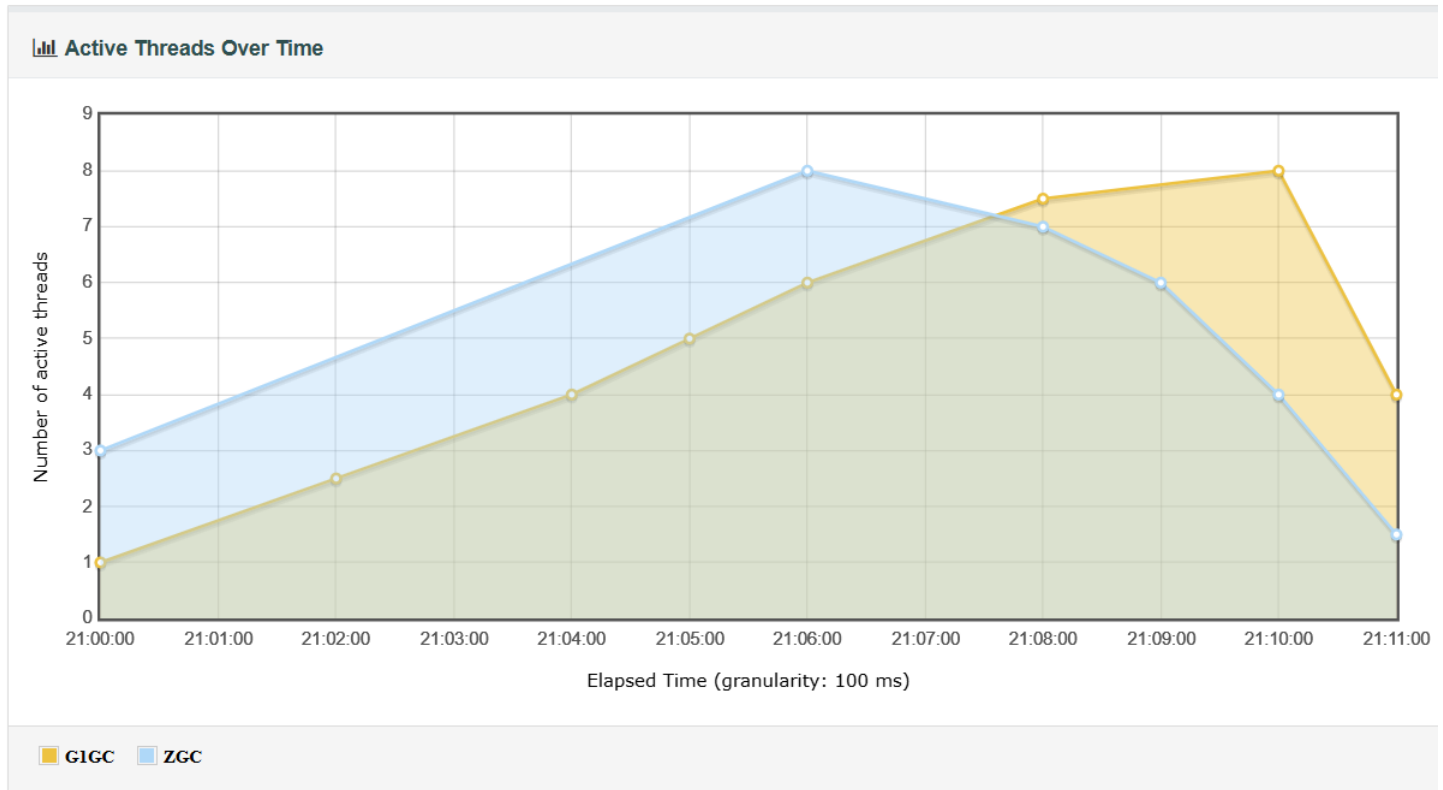


Figura 37: Número de threads ativas × tempo . Versão Hybrid em Java

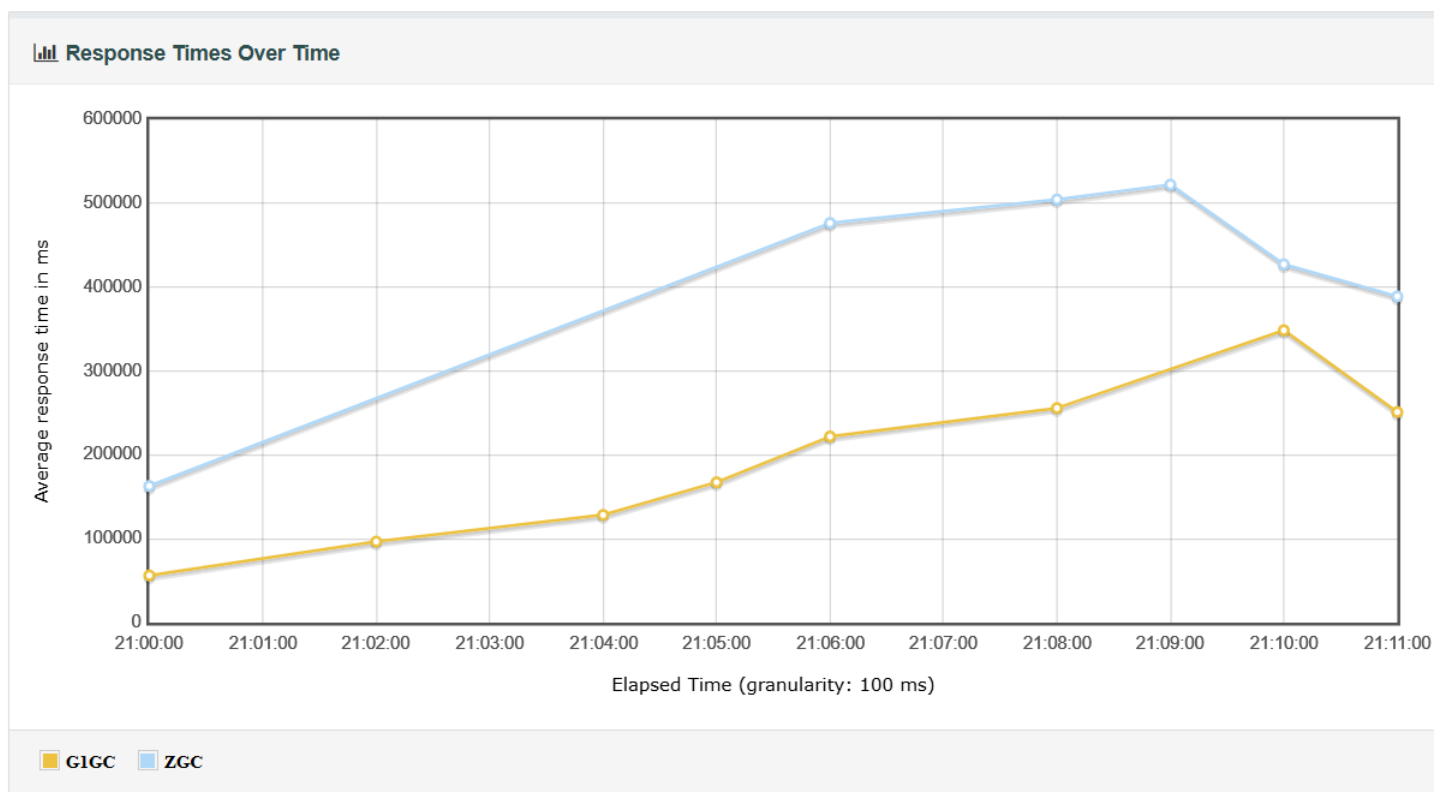


Figura 38: Tempo de resposta × tempo. Versão Hybrid em Java

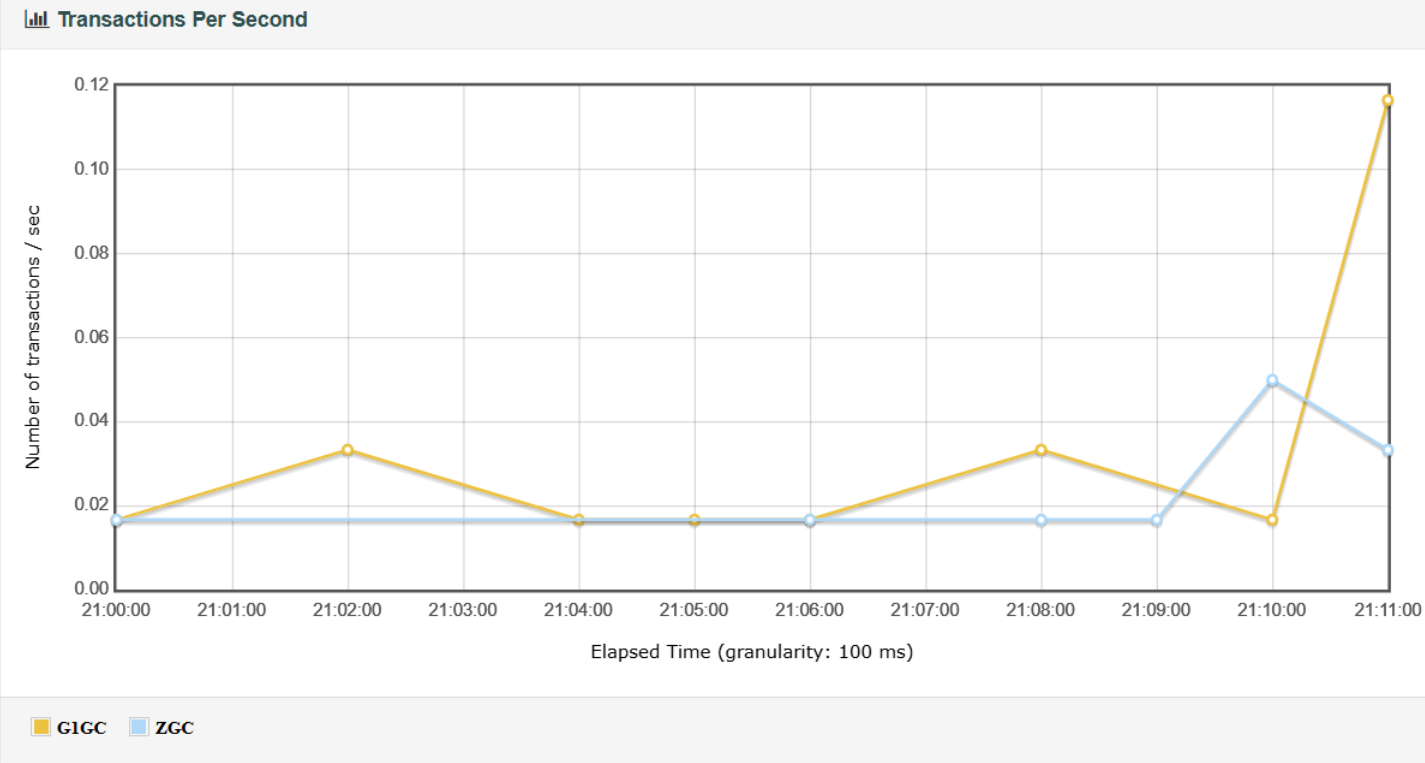


Figura 39: *Throughput* × tempo. Versão Hybrid em Java

Statistics											G1GC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ↕	FAIL ↕	Error % ↕	Average ↕	Min ↕	Max ↕	Median ↕	90th pct ↕	95th pct ↕	99th pct ↕	Transactions/s ↕	Received ↕	Sent ↕
Total	16	0	0.00%	211636.31	56312	377581	208947.50	357417.50	377581.00	377581.00	0.02	0.00	0.00
Hybrid Test Sample	16	0	0.00%	211636.31	56312	377581	208947.50	357417.50	377581.00	377581.00	0.02	0.00	0.00

Statistics											ZGC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ↕	FAIL ↕	Error % ↕	Average ↕	Min ↕	Max ↕	Median ↕	90th pct ↕	95th pct ↕	99th pct ↕	Transactions/s ↕	Received ↕	Sent ↕
Total	9	0	0.00%	414146.33	162993	548591	462083.00	548591.00	548591.00	548591.00	0.01	0.00	0.00
Hybrid Test Sample	9	0	0.00%	414146.33	162993	548591	462083.00	548591.00	548591.00	548591.00	0.01	0.00	0.00

Figura 40: Estatísticas. Versão Hybrid em Java

Observações:

- A versão que utilizou G1GC não apresentou conflito de contenção, portanto uma imagem para isto não foi incluída.
- O número 21 no eixo x se refere ao dia em que a medição foi feita.

4.1.5. Avaliação de Profile

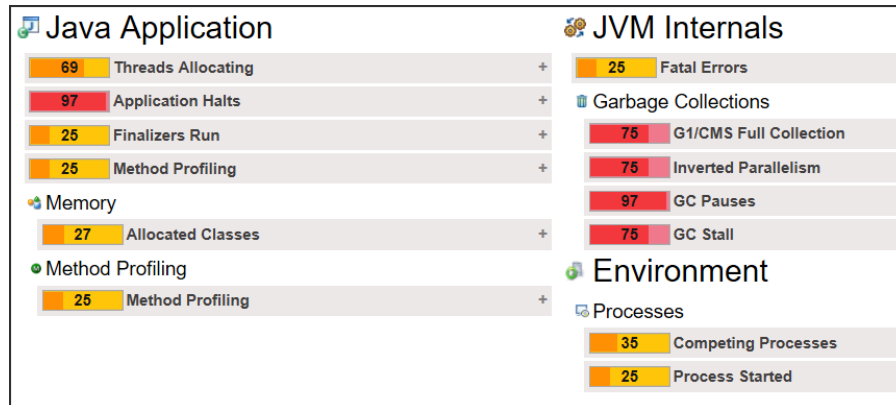


Figura 41: Resumo do JMC. G1GC. Versão Hybrid em Java

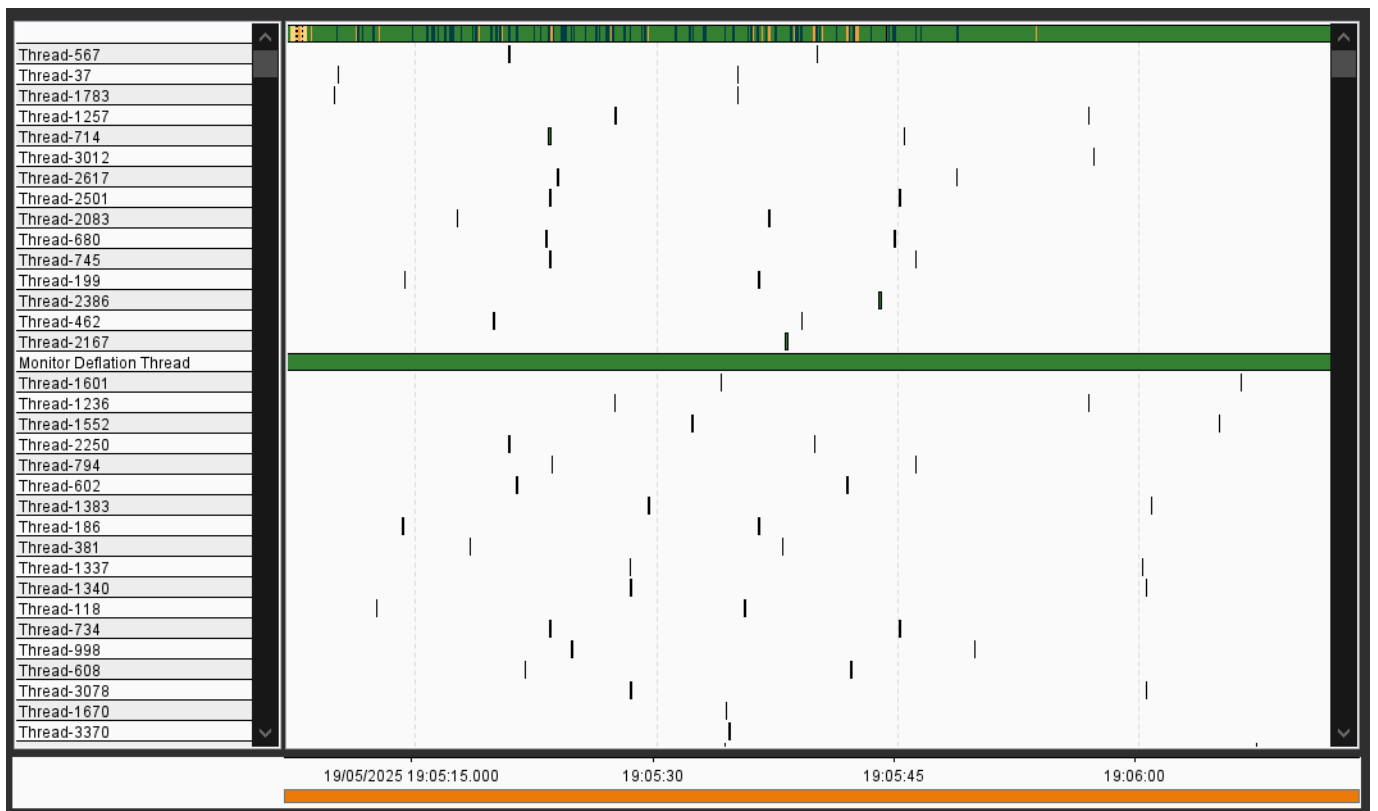


Figura 42: Threads. G1GC. Versão Hybrid em Java

Method	Count	Percentage
java.util.TreeMap.getCeilingEntry(Object)	516	8,92 %
java.io.FilterInputStream.read()	444	7,68 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	362	6,26 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	284	4,91 %
java.util.HashMap.getNode(Object)	273	4,72 %
java.io.PushbackInputStream.read()	217	3,75 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	216	3,74 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	190	3,29 %
org.apache.pdfbox.contentstream.operator.Operator.<init>(String)	120	2,08 %
java.util.TreeMap.put(Object, Object, boolean)	111	1,92 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	105	1,82 %

Figura 43: Hot methods. G1GC. Versão Hybrid em Java

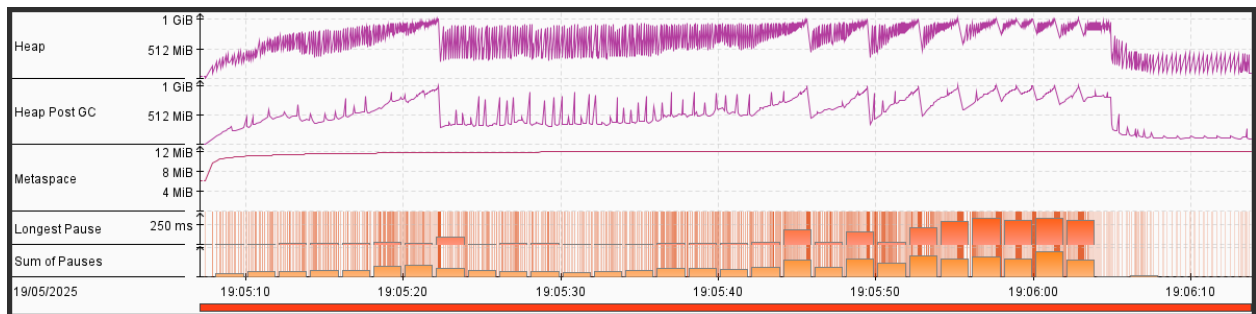


Figura 44: *Garbage Collector. G1GC. Versão Hybrid em Java*

Young Collection Total Time		Old Collection Total Time	
GC Count	784	GC Count	108
Average GC Time	11,056 ms	Average GC Time	359,185 ms
Maximum GC Time	43,978 ms	Maximum GC Time	1,433 s
Total GC Time	8,668 s	Total GC Time	38,792 s
All Collections Total Time		All Collections Pause Time	
GC Count	892	Average Pause	12,000 ms
Average GC Time	53,206 ms	Longest Pause	336,872 ms
Maximum GC Time	1,433 s	Sum of Pauses	11,676 s
Total GC Time	47,460 s		

Figura 45: *Garbage Collector Summary. G1GC. Versão Hybrid em Java*

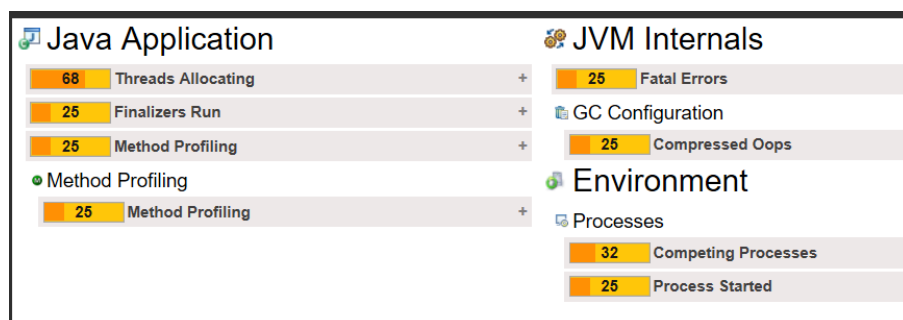


Figura 46: *Resumo do JMC. ZGC. Versão Hybrid em Java*

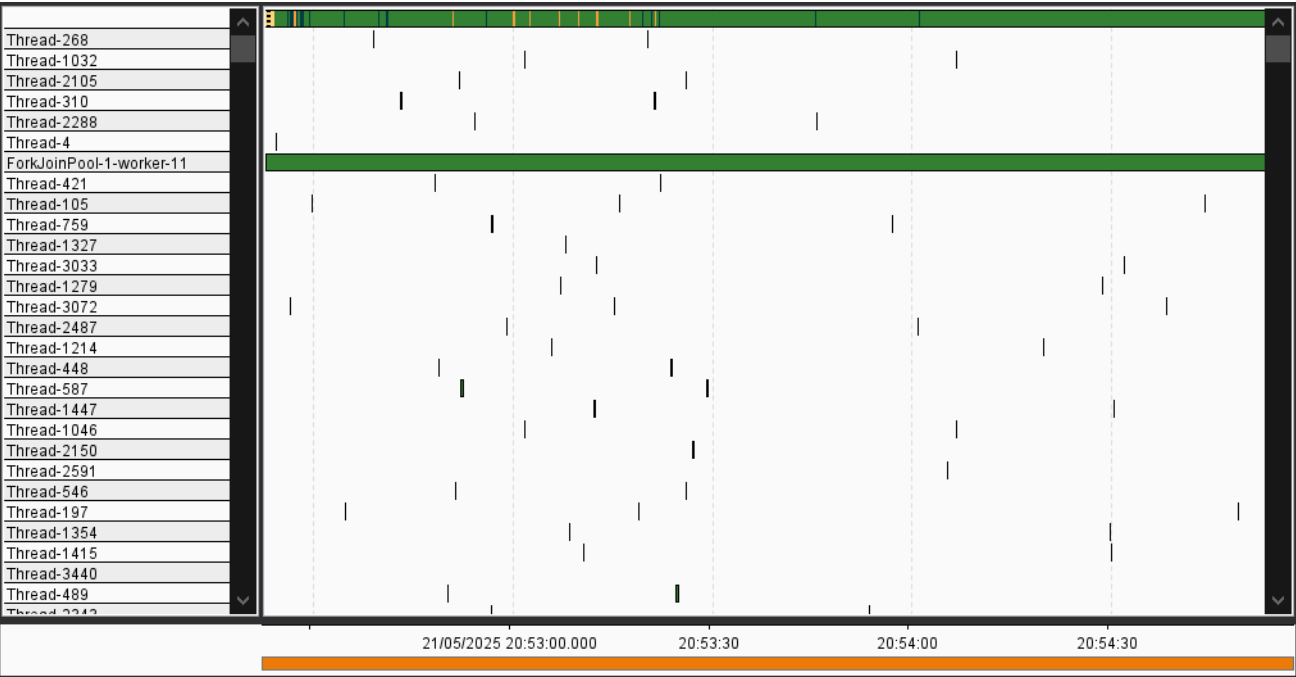


Figura 47: Threads. ZGC. Versão Hybrid em Java

Monitor Class	Total Blocked Time	Distinct Threads	Count
 java.lang.Object	60,586 ms	1	3

Monitor Address	Total Blocked Time	Distinct Threads	Count
 0x1BAA364D510	60,586 ms	1	3

Thread	Total Blocked Time	Count
	60,586 ms	3

Figura 48: Contenção. ZGC. Versão Hybrid em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	1.232	14,8 %
java.util.TreeMap.getCeilingEntry(Object)	845	10,2 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	457	5,5 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	388	4,67 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	377	4,54 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	374	4,5 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	308	3,71 %
java.util.TreeMap.put(Object, Object)	222	2,67 %
org.apache.pdfbox.text.PDFTextStripper.processTextPosition(Text...	191	2,3 %
java.util.TreeMap.put(Object, Object, boolean)	180	2,17 %
org.apache.pdfbox.cos.COSNumber.isFloat(String)	119	1,43 %

Figura 49: *Hot methods*. ZGC. Versão Hybrid em Java

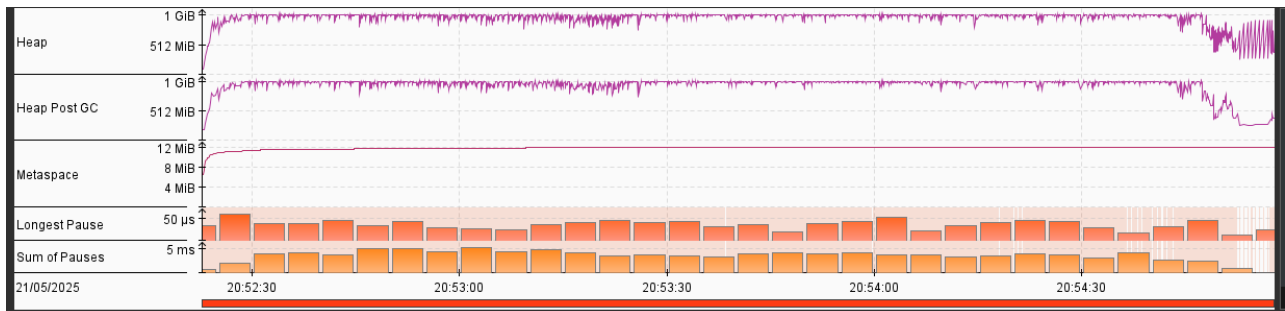


Figura 50: *Garbage Collector*. ZGC. Versão Hybrid em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	0	GC Count	0
Average GC Time	N/A	Average GC Time	N/A
Maximum GC Time	N/A	Maximum GC Time	N/A
Total GC Time	N/A	Total GC Time	N/A
All Collections Total Time		All Collections Pause Time	
GC Count	2.908	Average Pause	12,560 μs
Average GC Time	86,333 ms	Longest Pause	59,869 μs
Maximum GC Time	1,674 s	Sum of Pauses	116,983 ms
Total GC Time	4 min 11 s		

Figura 51: *Garbage Collector Summary*. ZGC. Versão Hybrid em Java

4.2. Haskell

A implementação Hybrid, bem como as outras concorrentes, se assemelha bastante à implementação Serial. A principal diferença está no uso de threads em dois locais do código: processamento de um documento e cálculo do *score*. Em cada uma dessas regiões há uma função `fork` (IO ou OS) composta com uma função que encapsula o trabalho feito.

Primeiramente, irei falar sobre o aspecto concorrente do processamento de um documento (Figura 52). Esta versão do algoritmo utiliza `MVars` (locks) para a resolução de condições de corrida, mas também para a comunicação com a thread principal. Observe as linhas 36, 41 e 48 da Figura 52, nelas está o fluxo dos dados coletados a partir dos documentos, os quais estão armazenados em uma variável chamada `mvar_doc_data`. Inicialmente (linha 36) esta variável, a qual é uma lista de `MVar`'s, é declarada com cada um de seus elementos vazios, ou seja, com o "lock aberto". A associação de cada seção da lista à cada thread é feita na linha 41 e a extração dos dados de volta para thread principal é feita na linha 48. Mas antes dessa extração, quando cada `MVar` é preenchida com dados (algo que acontece na função `tokenizeDoc`), isto causa o lock a ficar fechado e este somente é reaberto na linha 48, onde esta abertura serve de confirmação para a thread principal que dada thread de trabalho foi concluída. Ademais, foram utilizadas *Virtual Threads*² para o processamento de arquivos, visto que estas lidam melhor com uma maior frequência de operações IO.

```
36      mvar_doc_data <- create_empty_mvars n_pdf_names :: IO [MVar DocumentData]
37      thread_print <- newEmptyMVar
38
39      -- dividing work between n_threads
40      let process_function = mapM $ wrapper (get_doc_data query) (tokenizeDoc thread_print)
41      let n_threads_input = split_into_n_lists n_threads_doc (zip mvar_doc_data pdf_names)
42      threadIds <- mapM (fork_aux forkIO process_function) n_threads_input
43
44      putStrLn $ "Processing docs with: " ++ (show $ length threadIds) ++ " threads."
45
46      -- waits until all threads finish
47      replicateM_ n_pdf_names (printMVar thread_print)
48      doc_data_list <- mapM takeMVar mvar_doc_data :: IO [DocumentData]
49
50      -- Filtering out empty docs
51      let documents = filter (not_empty . name) doc_data_list :: [DocumentData]
```

Figura 52: Processamento dos documentos na versão Hybrid em Haskell.

Em segundo lugar, sobre o aspecto concorrente do cálculo do score, observe a Figura 53. Nela, o código para uso de threads se assemelha com aquele visto no processamento de documentos: declara-se uma lista de `MVar`'s para em seguida distribuí-las entre as threads. Vale notar que esta parte não retorna o documento de maior score, isto é feito de maneira serial posteriormente. Ademais, foram utilizadas *Platform Threads*³ nesta parte, como um paralelo à implementação em Java — pois lá estas lidam melhor

²Haskell não as chama assim, entretanto é um conceito parecido.

³Novamente, Haskell não usa esse nome, entretanto é um conceito parecido

com operações que não são IO intensivas — entretanto, em teoria, isto não se reflete em Haskell [Haskell Concurrent 2025].

```
65      -- dividing work between n_threads
66      doc_score_mvars <- create_empty_mvars n_processed_docs :: IO [MVar (String, Double)]
67      let n_threads_input = split_into_n_lists n_threads_score (zip doc_score_mvars documents)
68      threadIds <- mapM (fork_aux forkOS
69          (mapM $ multithread_doc_score (nDocs, avgdl, idfs) query)) n_threads_input
70
```

Figura 53: Processamento dos documentos na versão Hybrid em Haskell.

4.2.1. Resultados da Implementação

Observação:

- Os experimentos foram feitos com apenas o GC padrão de Haskell (GC geracional).

4.2.2. Avaliação com Microbenchmark

overview

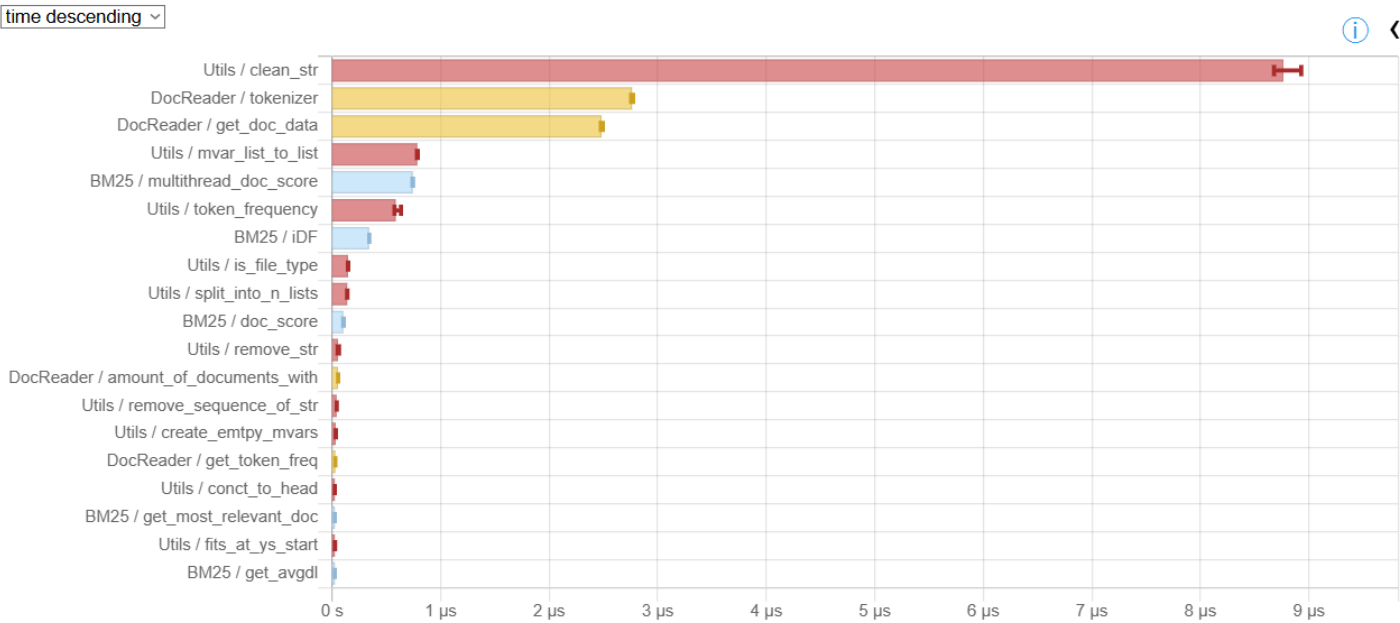


Figura 54: Resultados do Criterion para as funções da versão Hybrid em Haskell.

O método `tokenizeDoc` teve um tempo de execução médio de 41.4 ms. Este não aparece no gráfico pelo que foi explicado em 3.2.2.

4.2.3. Avaliação com Macrobenchmark

overview



mainBench

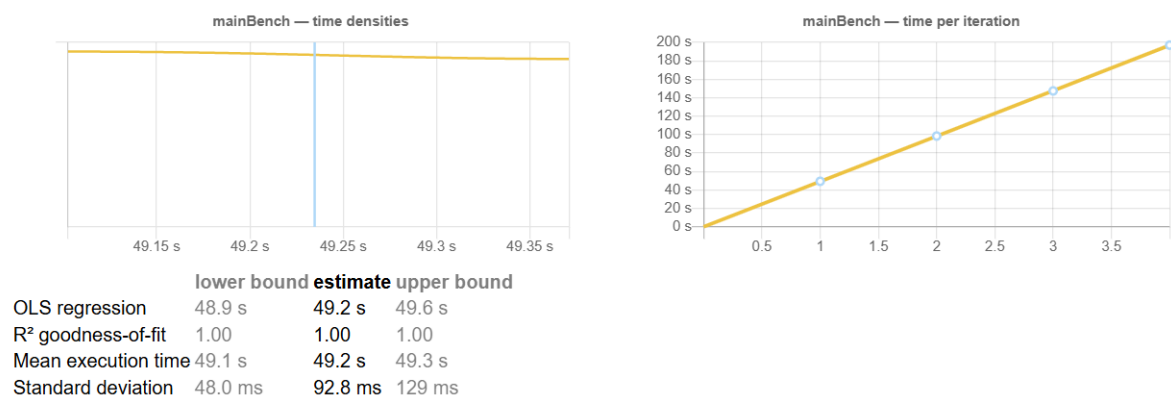


Figura 55: Resultados do Criterion para main da versão Hybrid em Haskell.

4.2.4. Avaliação de Profile

Por algum motivo, o tempo do evento de usuário CalculatingScore começa a ser considerado no início do programa até o seu fim, o que não é correto. Não consegui resolver este bug, entretanto, felizmente, ele não afeta as discussões deste relatório.

GC	30788630600ns	30.789s
USER EVENTS (user events are corrected for GC)		
CalculatingScore	18267904600ns	18.268s
ProcessingDocs	18263014100ns	18.263s
GatheringDocs	574700ns	0.001s
TOTAL	36531493400ns	36.531s
THREAD EVENTS		
1	283000ns	0.000s
IOManagerThread:2	82600ns	0.000s
3	2885800ns	0.003s
4	162681000ns	0.163s
5	2460218100ns	2.460s
6	607388700ns	0.607s
7	2644153400ns	2.644s
8	806946100ns	0.807s
9	446946400ns	0.447s
10	563432900ns	0.563s
11	11345313300ns	11.345s
12	1197304400ns	1.197s
13	16697977900ns	16.698s
14	2310490200ns	2.310s
15	846505900ns	0.847s
weak finalizer thread:16	22100ns	0.000s
weak finalizer thread:17	2100ns	0.000s
### tempos omitidos para visualização ###		
45	7400ns	0.000s
46	10900ns	0.000s
47	6500ns	0.000s
TOTAL	40092820000ns	40.093s

Figura 56: Resultados do ghc-events-analyze para versão Hybrid em Haskell.

4.3. Discussão

Esta seção primeiro irá abordar as diferenças entre Java e Haskell, em seguida a diferença entre as execuções em Java com G1GC e ZGC.

Em primeiro lugar, nota-se que o tempo médio de execução da versão em Haskell foi de 49,2 segundos, enquanto Java (G1GC) 45,2 segundos (Figura 36). Percebe-se facilmente que a versão de Java é mais rápida, tomando em conta o fato que opera em cima de um conjunto de dados 11 vezes maior e tem tempo de execução similar a versão em Haskell. Mesmo assim, adotando a mesma extrapolação linear na comparação entre as versões seriais (em 3.3), a versão de Haskell demoraria cerca de 9 minutos com o conjunto de dados de 1,1 GB.

De maneira similar à análise feita em 3.3, tem-se que 42% do tempo de execução da versão em Haskell foi gasto em GC, menos que a versão Serial, porém ainda ineficiente (Figura 56). Em comparação, a versão em Java (G1GC) gastou 17,5% do seu tempo com GC (calculado feito com base na Figura 45 e nos dados do *flight*), o que também não é ideal, porém melhor que Haskell. Portanto, recomenda-se um melhor uso do GC em ambas linguagens para melhoria em desempenho, seguindo o que foi sugerido em 3.3.

Com relação à Microbenchmark, esta não apresentou resultados relevantes que já não foram discutidos em 3.3.

Em suma, a versão de Java superou drasticamente a versão em Haskell na Avaliação de Profile e Macrobenchmark. Entretanto, com relação à Microbenchmark, Haskell se saiu melhor.

Em seguida, para as diferenças entre as execuções com G1GC e ZGC, novamente, o ZGC obteve uma performance superior, entretanto com um tempo de execução maior: 45,2 segundos para G1GC e 47,5 segundos para ZGC (Figura 36).

- A execução com ZGC apresentou uma avaliação de profile menos problemática (Figuras 41 e 46).
- O uso de GC com o ZGC foi de apenas 0,07% (Figura 51, em comparação à 17,5% do G1GC (Figura 45)).
- O gráfico do uso da Heap do G1GC aparenta ter quedas maiores, com uma frequência parecida com o ZGC (Figuras 44 e 50).

Ambas versões apresentam comportamento de threads similar (Figuras 42 e 47), entretanto somente a versão com ZGC apresentou contenções (Figura 48). Porém, a quantidade não é o suficiente para chegar a conclusões.

Ademais, ambas versões apresentaram *Hot methods* similares, entretanto, dessa vez o método `java.util.TreeMap.getCeilingEntry(Object)` foi o mais utilizado na execução com G1GC. Seu uso é feito pela biblioteca `pdfbox` para navegar as páginas de um documento, as quais são organizadas em uma árvore (tal organização é um padrão em documentos pdf [Gyan 2014]). Esta diferença não parece estar relacionada com os diferentes GC's. Novamente, nota-se que os métodos mais populares tendem a ser desta mesma biblioteca. Portanto, para a melhoria da eficiência, recomenda-se investigar a performance de outras bibliotecas que exercem a mesma função.

Por fim, quanto ao *Throughput* e *Response Time*, percebe-se que a execução com ZGC teve um desempenho pior, algo esperado devido a sua execução mais lenta. Dessa

forma, como visto na Figura 40, o *Response Time* médio do G1GC é 51,1% daquele do ZGC e o *Throughput* o dobro.

5. Implementação Concorrente - Virtual

5.1. Java

A única diferença entre a versão Hybrid e Virtual é a completa utilização de *Virtual Threads* nas partes concorrentes. Portanto, considera-se que a implementação já foi discutida na seção anterior.

5.1.1. Resultados da Implementação

Ver as observações em 3.1.1.

5.1.2. Avaliação de Condição de Corrida

Esta não apresenta diferenças à feita para a versão Hybrid, logo não foi inclusa.

5.1.3. Avaliação com Microbenchmark

Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.BM25_BenchTest.test_IDF	thrpt	8	15,157 ± 0,061		ops/ms
JMHTests.BM25_BenchTest.test_add	thrpt	8	3,821 ± 0,010		ops/ms
JMHTests.BM25_BenchTest.test_auction_challenge_highest_bidder_doc	thrpt	8	3,364 ± 0,241		ops/ms
JMHTests.BM25_BenchTest.test_auction_spawn_thread	thrpt	8	0,110 ± 0,001		ops/ms
JMHTests.BM25_BenchTest.test_constructor	thrpt	8	4,865 ± 0,007		ops/ms
JMHTests.BM25_BenchTest.test_get_most_relevant_doc	thrpt	8	0,079 ± 0,005		ops/ms
JMHTests.BM25_BenchTest.test_score	thrpt	8	3,360 ± 0,010		ops/ms
JMHTests.BM25_BenchTest.test_update_IDF	thrpt	8	11,837 ± 0,094		ops/ms
JMHTests.DocumentData_BenchTest.test_cleanString	thrpt	8	242,138 ± 0,754		ops/s
JMHTests.DocumentData_BenchTest.test_constructor	thrpt	4	0,178 ± 0,009		ops/s
JMHTests.DocumentData_BenchTest.test_counter_spawn_thread	thrpt	8	18,375 ± 0,106		ops/s
JMHTests.DocumentData_BenchTest.test_get_token_frequency	thrpt	8	47,188 ± 0,465		ops/s
JMHTests.DocumentData_BenchTest.test_get_tokenizers	thrpt	8	65,332 ± 0,186		ops/ms
JMHTests.Query_BenchTest.test_constructor	thrpt	8	1,289 ± 0,017		ops/ms
JMHTests.Utils_BenchTest.test_create_batch_Array	thrpt	8	2,746 ± 0,155		ops/ms
JMHTests.Utils_BenchTest.test_create_batch_ArrayList	thrpt	8	6,233 ± 0,039		ops/ms

Figura 57: Resultados do JMH para os métodos da versão Virtual em Java.

5.1.4. Avaliação com Macrobenchmark

-- G1GC						
Benchmark	Mode	Cnt	Score	Error	Units	
JMHTests.Main_BenchTest.test_main	avgt	8	47957,110	± 4716,065	ms/op	
-- ZGC						
Benchmark	Mode	Cnt	Score	Error	Units	
JMHTests.Main_BenchTest.test_main	avgt	8	48259,438	± 4174,188	ms/op	

Figura 58: Tempo médio de execução . Versão Virtual em Java

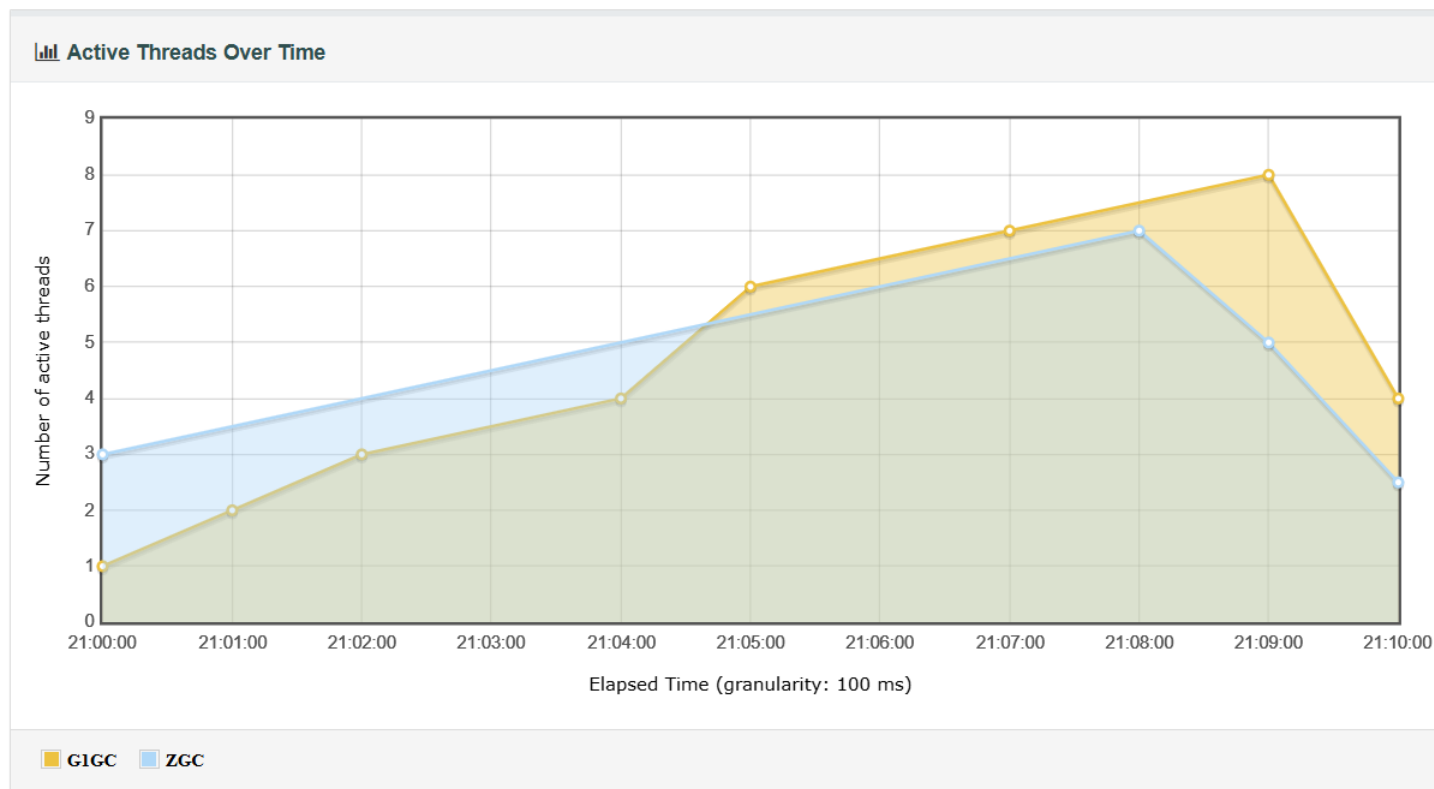


Figura 59: Número de threads ativas × tempo . Versão Virtual em Java

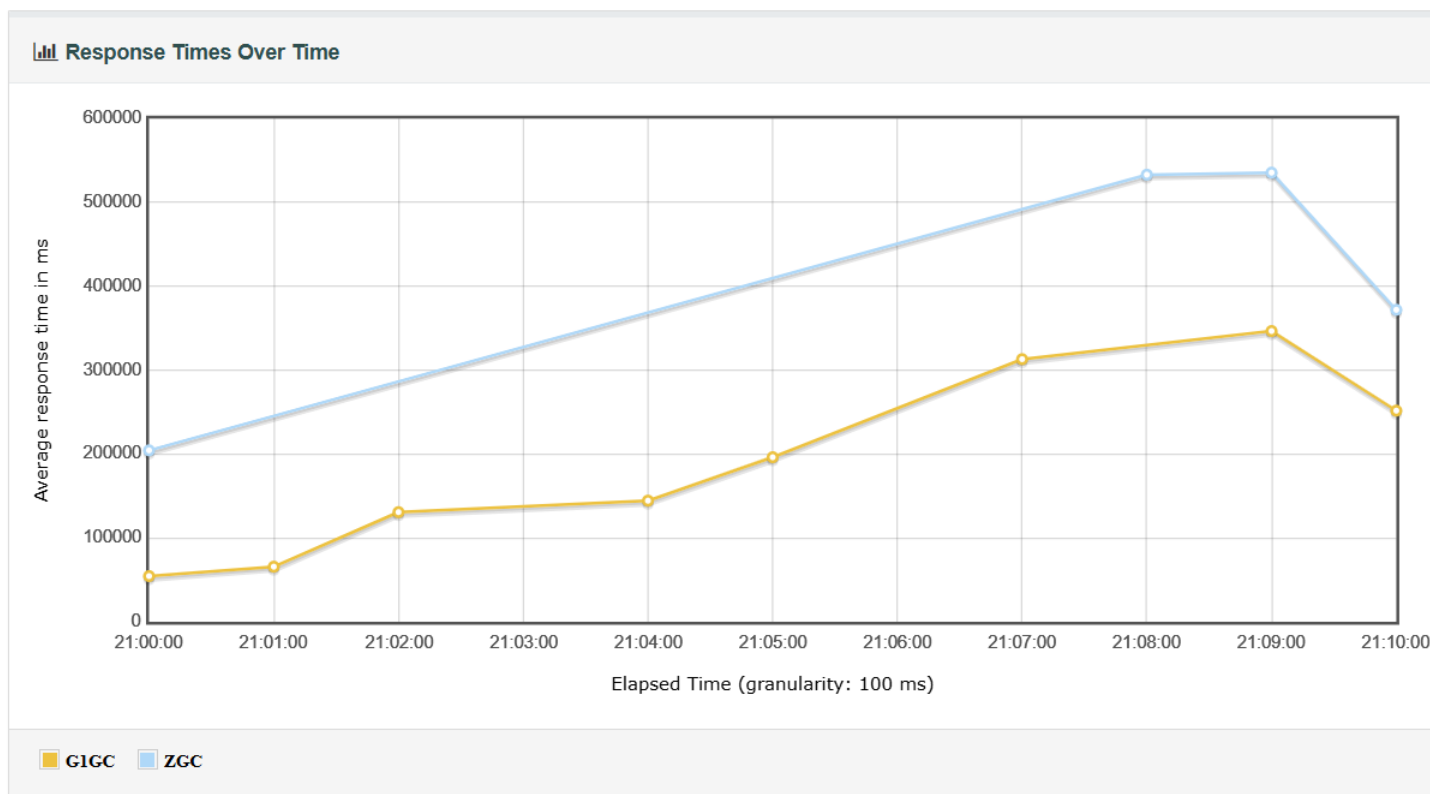


Figura 60: Tempo de resposta × tempo. Versão Virtual em Java

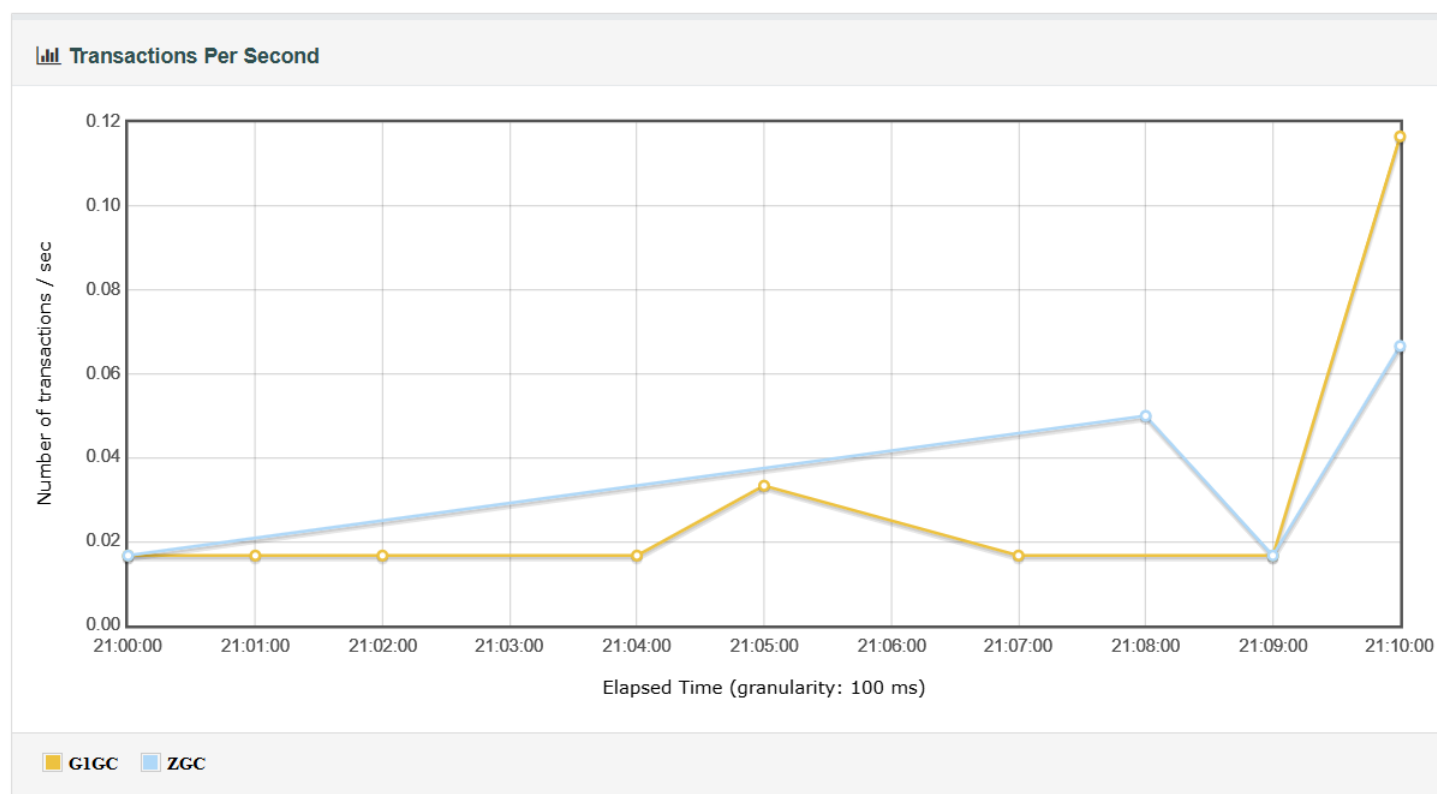


Figura 61: *Throughput* × tempo. Versão Virtual em Java

Statistics											G1GC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	15	0	0.00%	213998.60	54500	363011	210315.00	353231.00	363011.00	363011.00	0.02	0.00	0.00
Virtual Test Sample	15	0	0.00%	213998.60	54500	363011	210315.00	353231.00	363011.00	363011.00	0.02	0.00	0.00

Statistics											GZC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	9	0	0.00%	425302.89	204412	612257	476189.00	612257.00	612257.00	612257.00	0.01	0.00	0.00
Virtual Test Sample	9	0	0.00%	425302.89	204412	612257	476189.00	612257.00	612257.00	612257.00	0.01	0.00	0.00

Figura 62: Estatísticas. Versão Virtual em Java

Observações:

- A versão que utilizou GZC não apresentou conflito de contenção, portanto uma imagem para isto não foi incluída.
- O número 21 no eixo x se refere ao dia em que a medição foi feita.

5.1.5. Avaliação de Profile

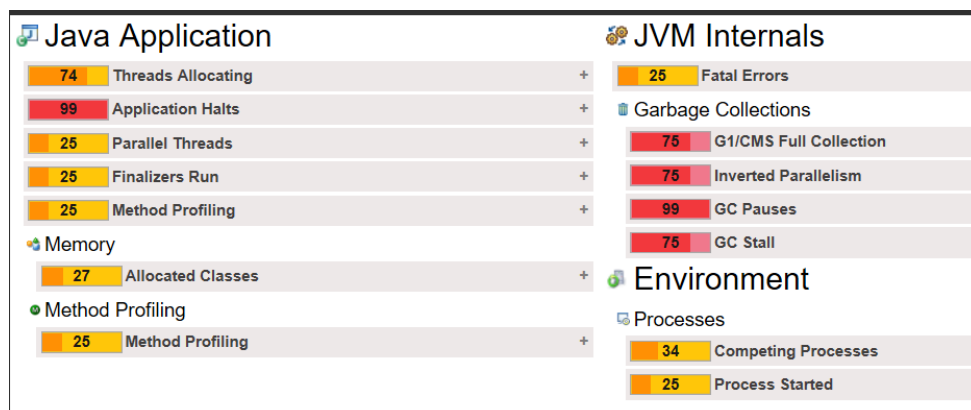


Figura 63: Resumo do JMC. G1GC. Versão Virtual em Java

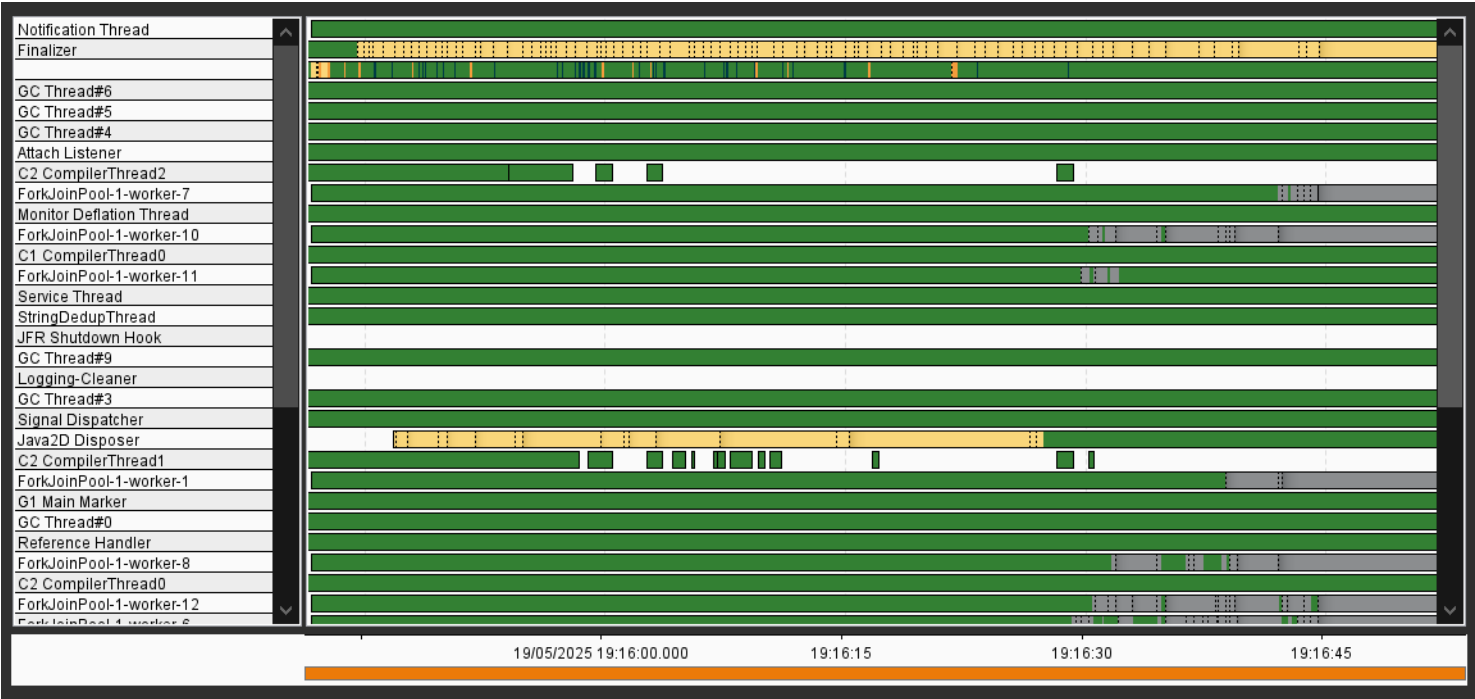


Figura 64: Threads. G1GC. Versão Virtual em Java

Monitor Class	Total Blocked Time	Distinct Threads	Count
 int[]	50,628 ms	1	2

Monitor Address	Total Blocked Time	Distinct Threads	Count
 0x2B785A0E660	25,989 ms	1	1
 0x2B785C31C30	24,639 ms	1	1

Thread	Total Blocked Time	Count
	50,628 ms	2

Figura 65: Contenção. G1GC. Versão Virtual em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	596	11,2 %
java.util.TreeMap.getCeilingEntry(Object)	528	9,88 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	314	5,88 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	196	3,67 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	186	3,48 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	181	3,39 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	158	2,96 %
java.io.PushbackInputStream.read()	137	2,56 %
org.apache.pdfbox.text.PDFTextStripper.processTextPosition(Text...	136	2,55 %
java.math.BigDecimal.<init>(char[], int, int, MathContext)	106	1,98 %
java.util.HashMap\$TreeNode.find(int, Object, Class)	105	1,97 %

Figura 66: *Hot methods*. G1GC. Versão Virtual em Java

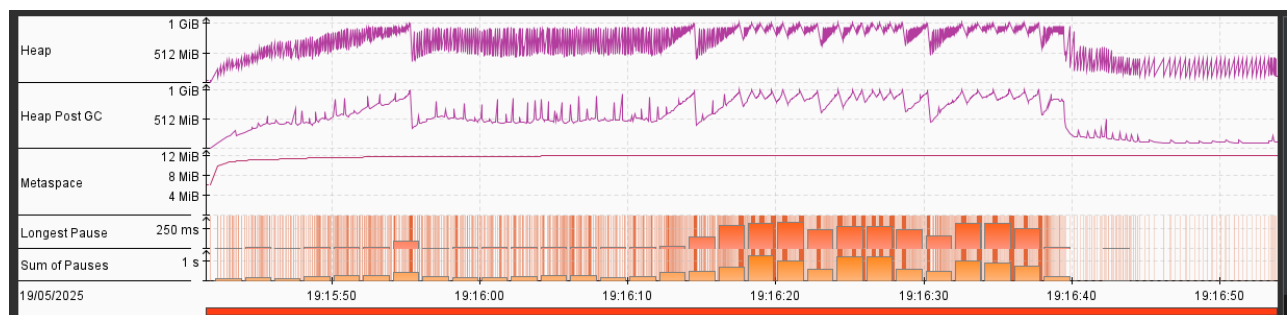


Figura 67: *Garbage Collector*. G1GC. Versão Virtual em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	972	GC Count	125
Average GC Time	9,358 ms	Average GC Time	385,066 ms
Maximum GC Time	50,500 ms	Maximum GC Time	1,155 s
Total GC Time	9,096 s	Total GC Time	48,133 s
All Collections Total Time		All Collections Pause Time	
GC Count	1.097	Average Pause	13,704 ms
Average GC Time	52,168 ms	Longest Pause	339,137 ms
Maximum GC Time	1,155 s	Sum of Pauses	15,814 s
Total GC Time	57,229 s		

Figura 68: *Garbage Collector Summary*. G1GC. Versão Virtual em Java

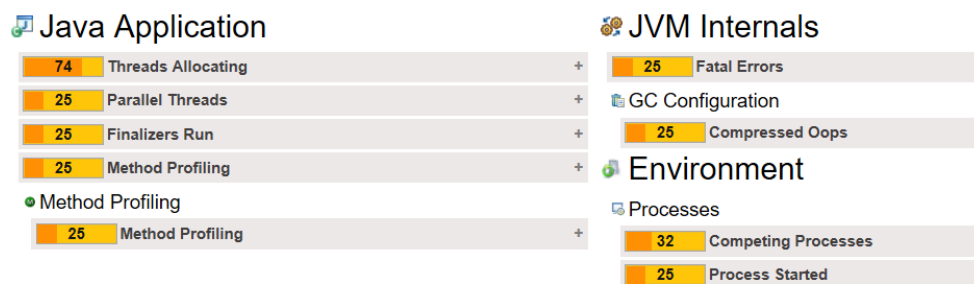


Figura 69: Resumo do JMC. ZGC. Versão Virtual em Java

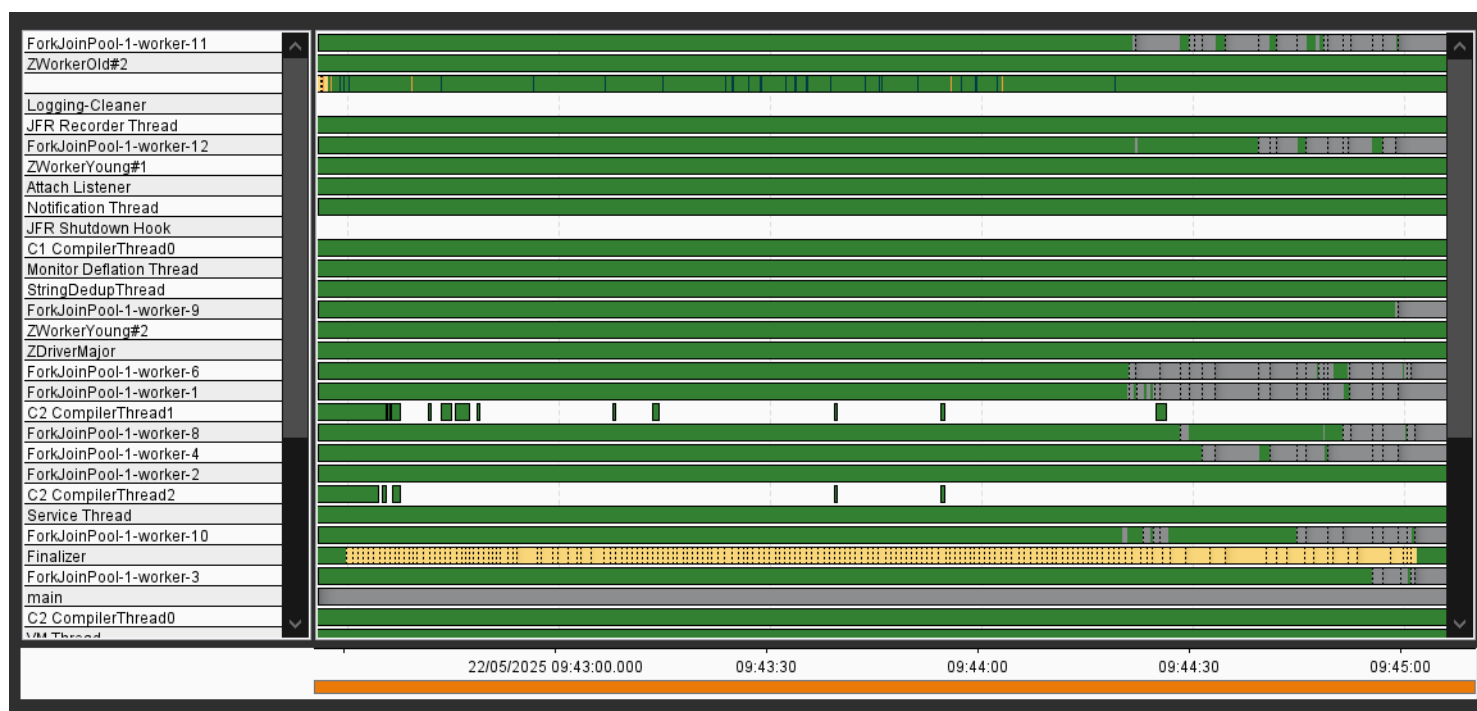


Figura 70: Threads. GZC. Versão Virtual em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	1.035	11,7 %
java.util.TreeMap.getCeilingEntry(Object)	924	10,5 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	519	5,89 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	451	5,12 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	428	4,85 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	367	4,16 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	323	3,66 %
org.apache.pdfbox.text.PDFTextStripper.processTextPosition(Text...	194	2,2 %
java.util.TreeMap.put(Object, Object, boolean)	158	1,79 %
java.lang.AbstractStringBuilder.ensureCapacityInternal(int)	149	1,69 %
org.apache.pdfbox.cos.COSNumber.isFloat(String)	143	1,62 %

Figura 71: Hot methods. ZGC. Versão Virtual em Java

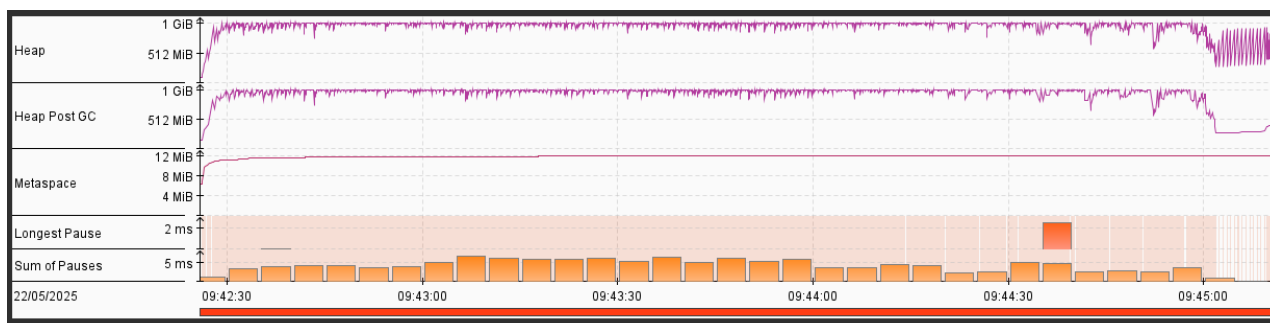


Figura 72: *Garbage Collector. ZGC. Versão Virtual em Java*

Young Collection Total Time		Old Collection Total Time	
GC Count	0	GC Count	0
Average GC Time	N/A	Average GC Time	N/A
Maximum GC Time	N/A	Maximum GC Time	N/A
Total GC Time	N/A	Total GC Time	N/A
All Collections Total Time		All Collections Pause Time	
GC Count	3,228	Average Pause	14,213 μ s
Average GC Time	83,269 ms	Longest Pause	2,556 ms
Maximum GC Time	1,521 s	Sum of Pauses	146,342 ms
Total GC Time	4 min 29 s		

Figura 73: *Garbage Collector Summary. ZGC. Versão Virtual em Java*

5.2. Haskell

Como em Java, a única diferença entre a implementação da versão Hybrid e Virtual é a utilização da função `forkIO` em vez de `forkOS` no código. Portanto, considera-se que a implementação já foi discutida em seções anteriores.

5.2.1. Resultados da Implementação

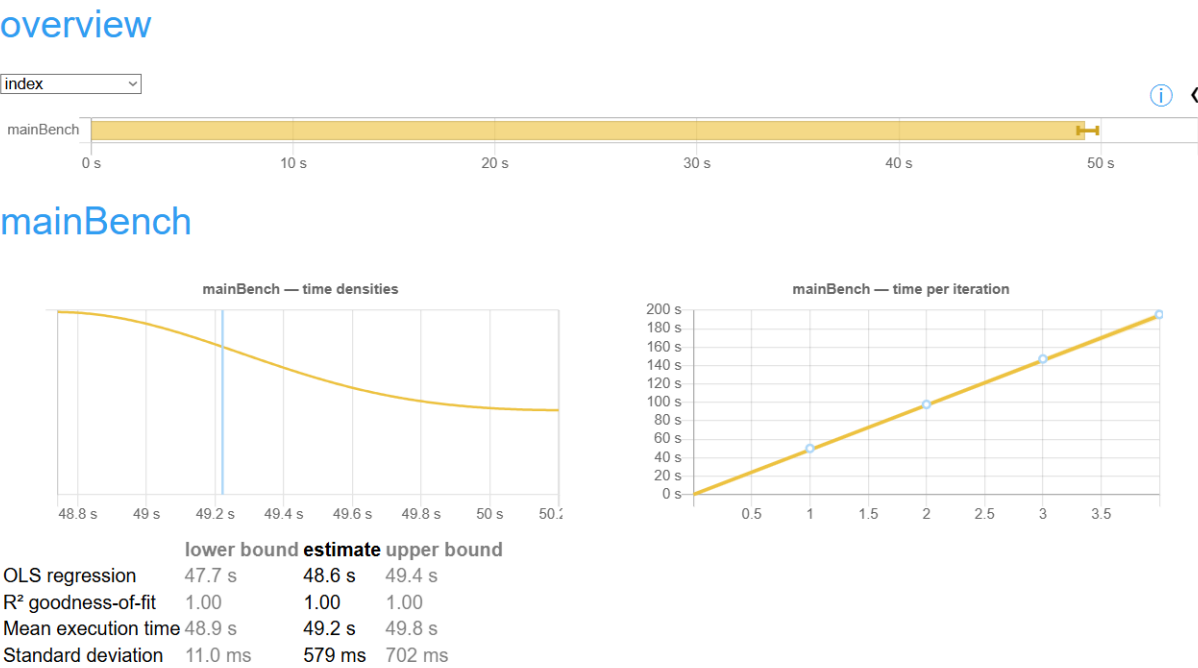
Observação:

- Os experimentos foram feitos com apenas o GC padrão de Haskell (GC geracional).

5.2.2. Avaliação com Microbenchmark

Esta não apresenta diferenças à feita para a versão Hybrid, logo não foi inclusa.

5.2.3. Avaliação com Macrobenchmark



Outlying measurements have a moderate (18.7%) effect on estimated standard deviation.

Figura 74: Resultados do Criterion para main da versão Virtual em Haskell.

5.2.4. Avaliação de Profile

GC	30555893600ns	30.556s
USER EVENTS (user events are corrected for GC)		
CalculatingScore	18193182300ns	18.193s
ProcessingDocs	18185943300ns	18.186s
GatheringDocs	687200ns	0.001s
TOTAL	36379812800ns	36.380s
THREAD EVENTS		
1	335700ns	0.000s
IOManagerThread:2	87900ns	0.000s
3	3130100ns	0.003s
4	172232000ns	0.172s
5	2534103300ns	2.534s
6	624824300ns	0.625s
7	2689387600ns	2.689s
8	735473300ns	0.735s
9	504566700ns	0.505s
10	623085400ns	0.623s
11	11281927000ns	11.282s
12	1260801900ns	1.261s
13	16651380500ns	16.651s
14	2349960800ns	2.350s
### tempos omitidos para visualização ###		
41	1000ns	0.000s
42	800ns	0.000s
43	1200ns	0.000s
44	2800ns	0.000s
45	800ns	0.000s
46	7900ns	0.000s
TOTAL	40239554500ns	40.240s

Figura 75: Resultados do ghc-events-analyze para versão Virtual em Haskell.

5.3. Discussão

Esta seção primeiro irá abordar as diferenças entre Java e Haskell, em seguida a diferença entre as execuções em Java com G1GC e ZGC.

Em primeiro lugar, com relação ao tempo de execução, observa-se que a média para a versão Haskell é de 49,2 segundos (Figura 74), já a versão em Java (G1GC), 47,9 segundos (Figura 58). Novamente, a versão em Java é mais rápida, e ao fazer uma extrapolação linear do tempo de execução de Haskell para o conjunto de dados de 1,1 GB, obtém-se aproximadamente 9 minutos.

Com relação a porcentagem do tempo de execução da versão em Haskell gasta pelo GC, na versão Virtual isto é 42%, essencialmente igual à versão Hybrid, ou seja, ineficiente. Em comparação, a versão em Java (G1GC) gastou 21,8% do seu tempo com GC (cálculo feito com base na Figura 68 e nos dados do *flight*), o que também não é desejado. Portanto, como nas versões anteriores, recomenda-se um melhor uso do GC para a melhoria do desempenho em ambas as linguagens, seguindo o que foi sugerido em 3.3.

Com relação à Microbenchmark, como citado em 5.2.2, as versões Hybrid e Virtual em Haskell tem benchmarks idênticas. Ademais, esta não apresentou resultados relevantes que já não foram discutidos em 3.3.

Em suma, a versão de Java superou drasticamente a versão em Haskell na Avaliação de Profile e Macrobenchmark. Entretanto, com relação à Microbenchmark, Haskell se saiu melhor.

Em seguida, para as diferenças entre as execuções com G1GC e ZGC, novamente o ZGC obteve uma performance superior, entretanto com um tempo de execução maior: 47,9 segundos para G1GC e 48,2 segundos para ZGC (Figura 58).

- A execução com ZGC apresentou uma avaliação de profile menos problemática (Figuras 63 e 69).
- O uso de GC com o ZGC foi de apenas 0,08% (Figura 73, em comparação à 21,8% do G1GC (Figura 68)).
- O gráfico do uso da Heap do G1GC aparenta ter quedas maiores, com uma frequência parecida com o ZGC. (Figuras 67 e 72).

Com relação ao comportamento das threads (Figuras 64 e 70), não parece variar com o uso de diferentes GC's, entretanto claramente difere da versão Hybrid. Tal diferença de comportamento é explicada pela análise feita pelo JMC não distinguir entre uma *Virtual Thread* que está trabalhando e uma que não está. Portanto, ao utilizar-se *Virtual Threads* no código, aparenta-se que estas estão ativas por mais tempo, sendo que isso nem sempre é o que de fato ocorre.

Quanto a contenção (Figura 65), apenas ocorreu na execução com o G1GC e de maneira não significativa.

Ambas versões apresentam *Hot methods* similares, com um resultado semelhante ao das versões seriais em 3.3.

Por fim, quanto ao *Throughput* e *Response Time*, percebe-se que a execução com ZGC teve um desempenho pior, algo esperado devido a sua execução mais lenta. Dessa

forma, como visto na Figura 62, o *Response Time* médio do G1GC é 50,3% daquele do ZGC e o *Throughput* o dobro.

6. Implementação Concorrente - Platform

6.1. Java

A única diferença entre a versão Hybrid e Platform é a completa utilização de *Platform Threads* nas partes concorrentes. Portanto, considera-se que a implementação já foi discutida em seções anteriores.

6.1.1. Resultados da Implementação

Ver as observações em 3.1.1.

6.1.2. Avaliação de Condição de Corrida

Esta não apresenta diferenças à feita para a versão Hybrid, logo não foi inclusa.

6.1.3. Avaliação com Microbenchmark

Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.BM25_BenchTest.test_IDF	thrpt	8	14,976 ±	0,290	ops/ms
JMHTests.BM25_BenchTest.test_add	thrpt	8	3,803 ±	0,041	ops/ms
JMHTests.BM25_BenchTest.test_auction_challenge_highest_bidder_doc	thrpt	8	3,370 ±	0,175	ops/ms
JMHTests.BM25_BenchTest.test_auction_spawn_thread	thrpt	8	0,123 ±	0,001	ops/ms
JMHTests.BM25_BenchTest.test_constructor	thrpt	8	4,875 ±	0,022	ops/ms
JMHTests.BM25_BenchTest.test_get_most_relevant_doc	thrpt	8	0,003 ±	0,001	ops/ms
JMHTests.BM25_BenchTest.test_score	thrpt	8	3,366 ±	0,016	ops/ms
JMHTests.BM25_BenchTest.test_update_IDF	thrpt	8	11,809 ±	0,080	ops/ms
JMHTests.DocumentData_BenchTest.test_cleanString	thrpt	8	244,367 ±	3,362	ops/s
JMHTests.DocumentData_BenchTest.test_constructor	thrpt	4	0,153 ±	0,013	ops/s
JMHTests.DocumentData_BenchTest.test_counter_spawn_thread	thrpt	8	85,975 ±	0,865	ops/s
JMHTests.DocumentData_BenchTest.test_get_token_frequency	thrpt	8	24,221 ±	1,280	ops/s
JMHTests.DocumentData_BenchTest.test_get_tokenizers	thrpt	8	64,262 ±	0,689	ops/ms
JMHTests.Query_BenchTest.test_constructor	thrpt	8	1,288 ±	0,017	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_Array	thrpt	8	2,831 ±	0,074	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_ArrayList	thrpt	8	6,327 ±	0,078	ops/ms

Figura 76: Resultados do JMH para os métodos da versão Platform em Java.

6.1.4. Avaliação com Macrobenchmark

-- G1GC						
Benchmark	Mode	Cnt	Score	Error	Units	
JMHTests.Main_BenchTest.test_main	avgt	8	47939,184	± 1319,464	ms/op	
-- ZGC						
Benchmark	Mode	Cnt	Score	Error	Units	
JMHTests.Main_BenchTest.test_main	avgt	8	49057,804	± 6980,171	ms/op	

Figura 77: Tempo médio de execução . Versão Platform em Java

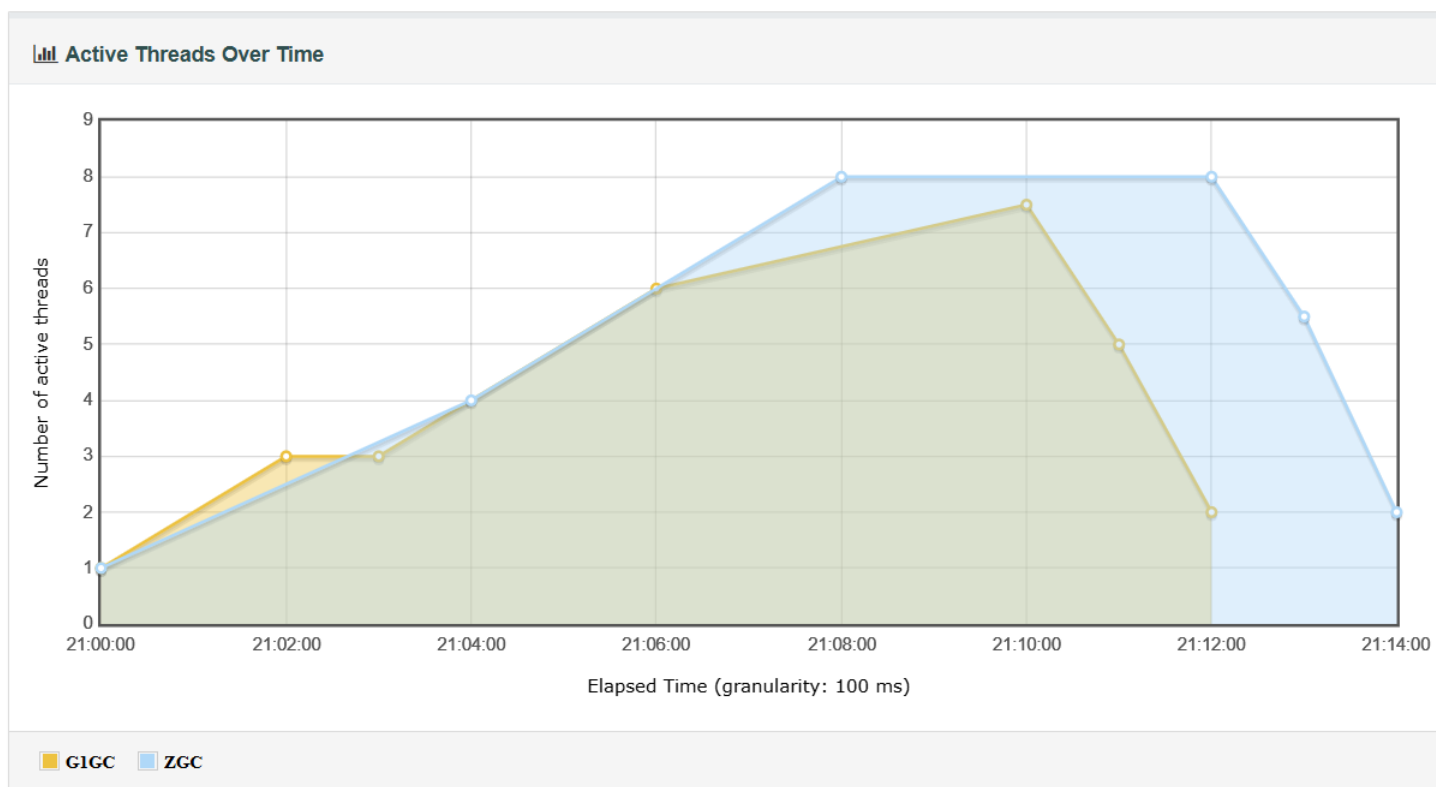


Figura 78: Número de threads ativas × tempo . Versão Platform em Java

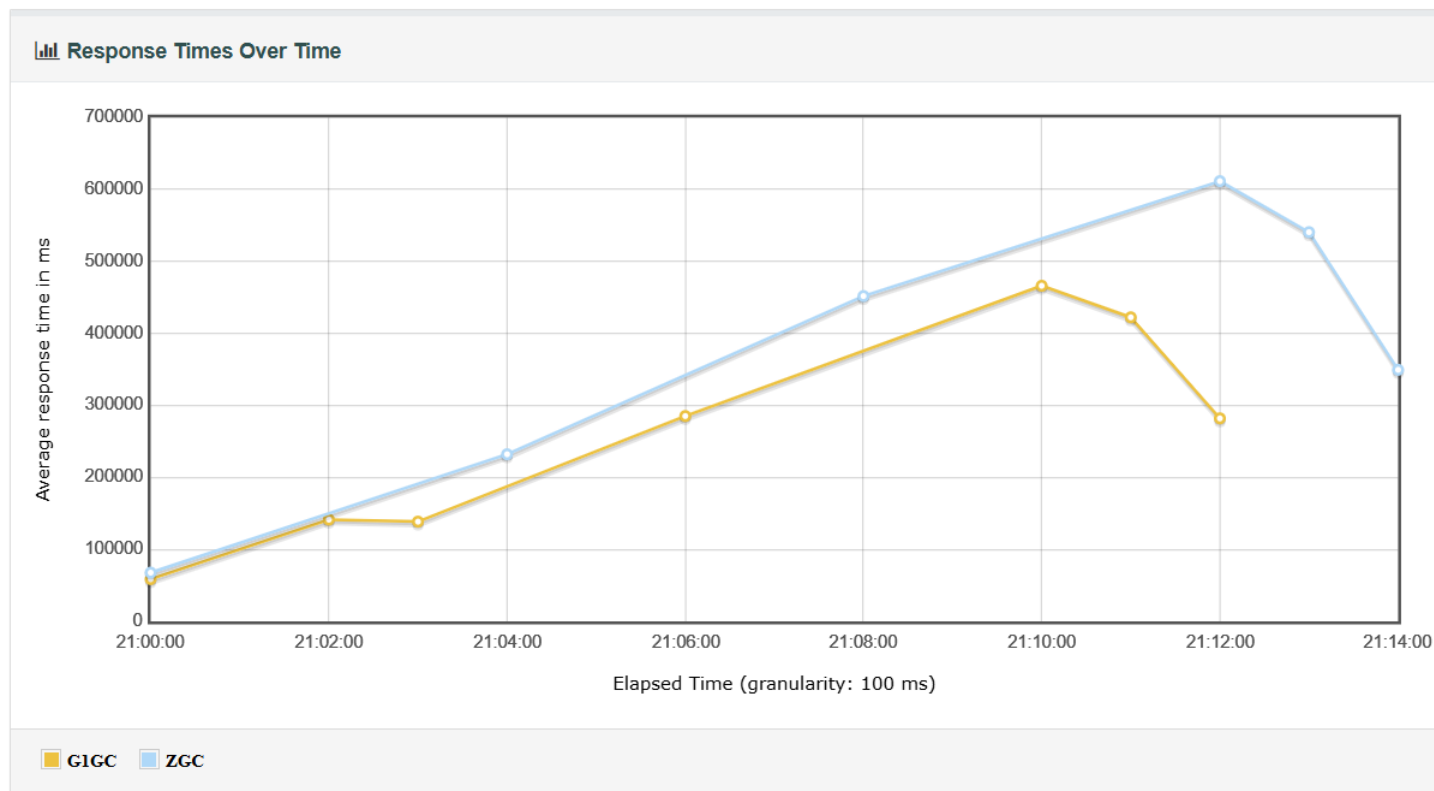


Figura 79: Tempo de resposta × tempo. Versão Platform em Java

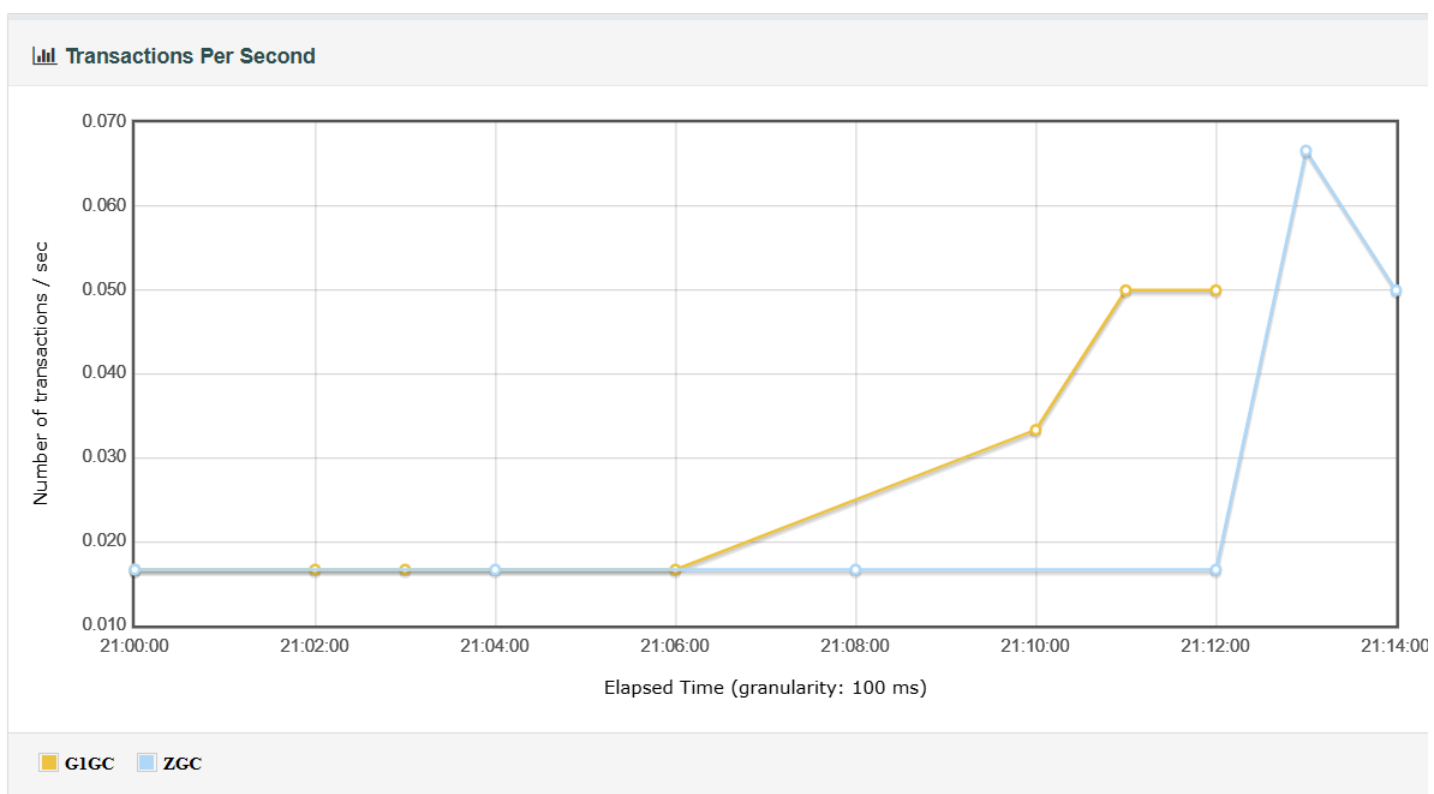


Figura 80: *Throughput* × tempo. Versão Platform em Java

Statistics											G1GC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	12	0	0.00%	305857.00	58705	466538	309251.50	466404.20	466538.00	466538.00	0.02	0.00	0.00
Platform Test Sample	12	0	0.00%	305857.00	58705	466538	309251.50	466404.20	466538.00	466538.00	0.02	0.00	0.00

Statistics											ZGC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	11	0	0.00%	415945.18	67661	611707	452000.00	610283.00	611707.00	611707.00	0.01	0.00	0.00
Platform Test Sample	11	0	0.00%	415945.18	67661	611707	452000.00	610283.00	611707.00	611707.00	0.01	0.00	0.00

Figura 81: Estatísticas. Versão Platform em Java

Observações:

- O número 21 no eixo x se refere ao dia em que a medição foi feita.

6.1.5. Avaliação de Profile

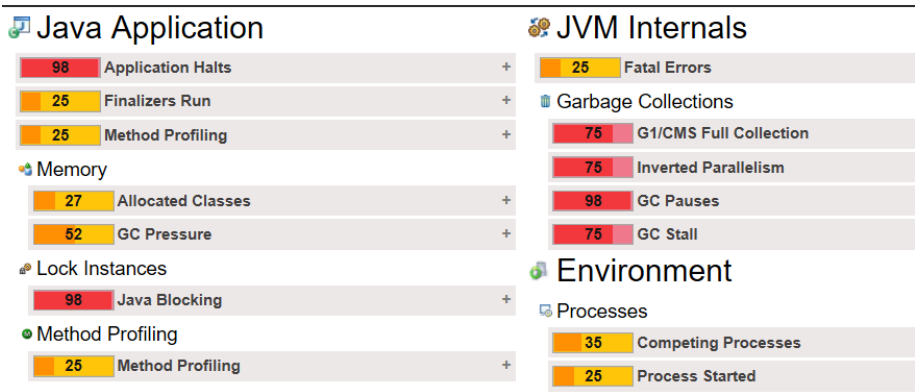


Figura 82: Resumo do JMC. G1GC. Versão Platform em Java

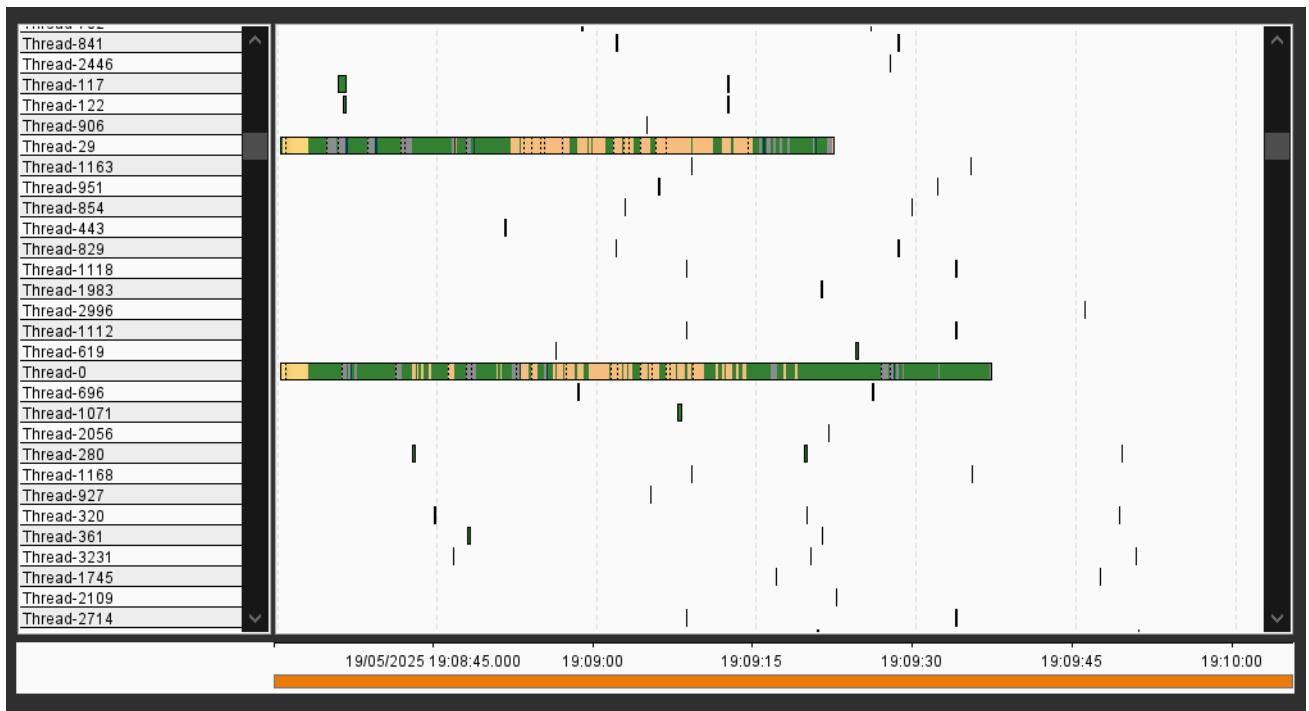


Figura 83: Threads. G1GC. Versão Platform em Java

Monitor Class	Total Blocked Time	Distinct Threads	Count
java.util.logging.ConsoleHandler	3 min 57 s	38	1.302
org.apache.pdfbox.pdmodel.font.FontMapperImpl	7,918 s	10	10
java.io.PrintStream	7,176 s	32	71
java.lang.Object	2,962 s	37	84
int[]	1,602 s	25	45
org.apache.pdfbox.pdmodel.font.FileSystemFontProvider\$FSFontIn...	1,414 s	8	10
jdk.internal.ref.CleanerImpl\$CleanableList	256,170 ms	3	3

Monitor Address	Total Blocked Time	Distinct Threads	Count
0x21347208550	3 min 57 s	38	1.302
0x21347364020	7,918 s	10	10
0x213479C3D20	7,176 s	32	71
0x21348CC5DA0	925,701 ms	6	6
0x21347A02790	870,641 ms	24	24
0x2134769B460	868,342 ms	27	27
0x21346E6B9C0	688,267 ms	19	19

Thread	Total Blocked Time	Count
Thread-3	15,208 s	84
Thread-30	14,818 s	76
Thread-17	14,682 s	79
Thread-29	13,496 s	72
Thread-9	11,328 s	71
Thread-8	10,615 s	54
Thread-0	10,387 s	61

Figura 84: Contenção. G1GC. Versão Platform em Java

Method	Count	Percentage
 <code>java.util.TreeMap.getCeilingEntry(Object)</code>	516	9,17 %
 <code>java.io.FilterInputStream.read()</code>	471	8,37 %
 <code>org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...</code>	337	5,99 %
 <code>java.io.PushbackInputStream.read()</code>	204	3,63 %
 <code>org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...</code>	192	3,41 %
 <code>org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...</code>	177	3,15 %
 <code>jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)</code>	173	3,08 %
 <code>java.util.TreeMap.put(Object, Object, boolean)</code>	163	2,9 %
 <code>java.util.HashMap.getNode(Object)</code>	158	2,81 %
 <code>java.util.HashMap\$TreeNode.getTreeNode(int, Object)</code>	144	2,56 %
 <code>java.util.concurrent.ConcurrentHashMap.get(Object)</code>	128	2,28 %

Figura 85: *Hot methods*. G1GC. Versão Platform em Java

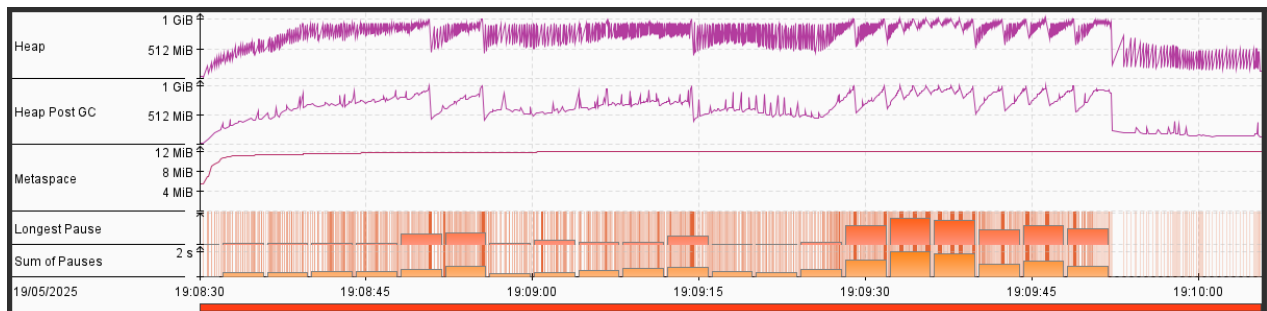


Figura 86: *Garbage Collector*. G1GC. Versão Platform em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	995	GC Count	119
Average GC Time	12,083 ms	Average GC Time	498,402 ms
Maximum GC Time	96,022 ms	Maximum GC Time	1,449 s
Total GC Time	12,023 s	Total GC Time	59,310 s
All Collections Total Time		All Collections Pause Time	
GC Count	1,114	Average Pause	14,712 ms
Average GC Time	64,033 ms	Longest Pause	425,252 ms
Maximum GC Time	1,449 s	Sum of Pauses	17,404 s
Total GC Time	1 min 11 s		

Figura 87: *Garbage Collector Summary*. G1GC. Versão Platform em Java

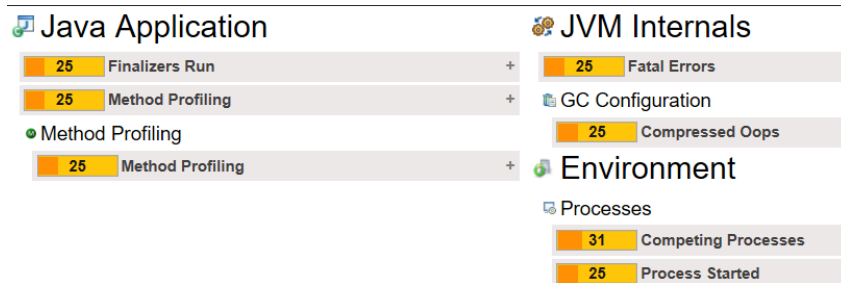


Figura 88: Resumo do JMC. ZGC. Versão Platform em Java

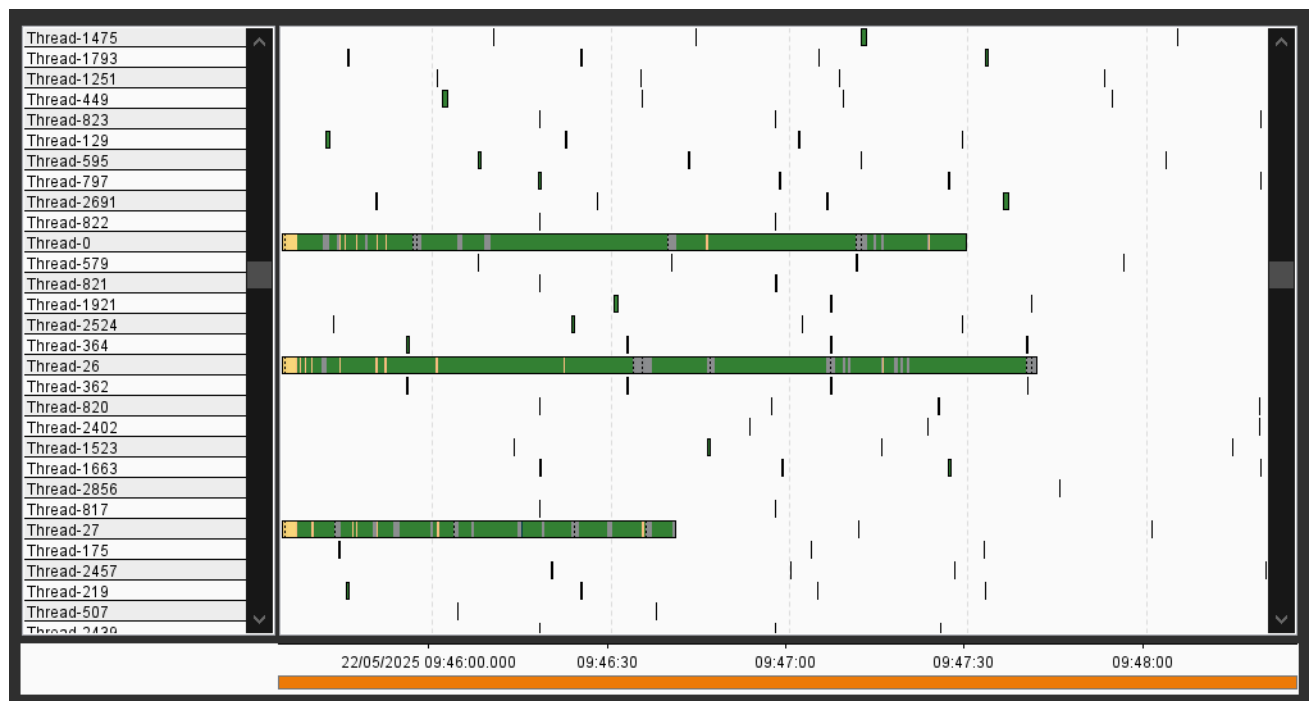


Figura 89: Threads. ZGC. Versão Platform em Java

Monitor Class	total Blocked Time	distinct Threads	Count
org.apache.pdfbox.pdmodel.font.FontMapperImpl	7,389 s	8	8
java.lang.Object	4,846 s	39	130
jdk.internal.ref.CleanerImpl\$CleanableList	4,058 s	40	119
java.util.logging.ConsoleHandler	2,998 s	21	53
int[]	2,099 s	33	44
org.apache.commons.logging.impl.Jdk14Logger	1,456 s	17	24
org.apache.pdfbox.pdmodel.font.FileSystemFontProvider\$FSFontIn...	717,421 ms	5	6

Monitor Address	total Blocked Time	distinct Threads	Count
0x12D56485370	7,389 s	8	8
0x12D56315560	4,058 s	40	119
0x12D5659E100	2,537 s	19	41
0x12D55CBA570	1,456 s	17	24
0x12D563168E0	1,136 s	39	39
0x12D56488D30	773,362 ms	22	22
0x12D56B5B5A0	704,678 ms	16	16

Thread	total Blocked Time	Count
Thread-8	1,951 s	17
Thread-37	1,652 s	9
Thread-6	1,649 s	17
Thread-5	1,502 s	8
Thread-34	1,482 s	10
Thread-38	1,432 s	10
Thread-1	1,130 s	15

Figura 90: Contenção. ZGC. Versão Platform em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	801	9,76 %
java.util.TreeMap.getCeilingEntry(Object)	786	9,58 %
java.util.HashMap.getNode(Object)	377	4,59 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	375	4,57 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matrx...	365	4,45 %
java.io.PushbackInputStream.read()	338	4,12 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	331	4,03 %
java.util.TreeMap.put(Object, Object, boolean)	330	4,02 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	314	3,83 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	182	2,22 %
java.lang.AbstractStringBuilder.ensureCapacityInternal(int)	160	1,95 %

Figura 91: Hot methods. ZGC. Versão Platform em Java

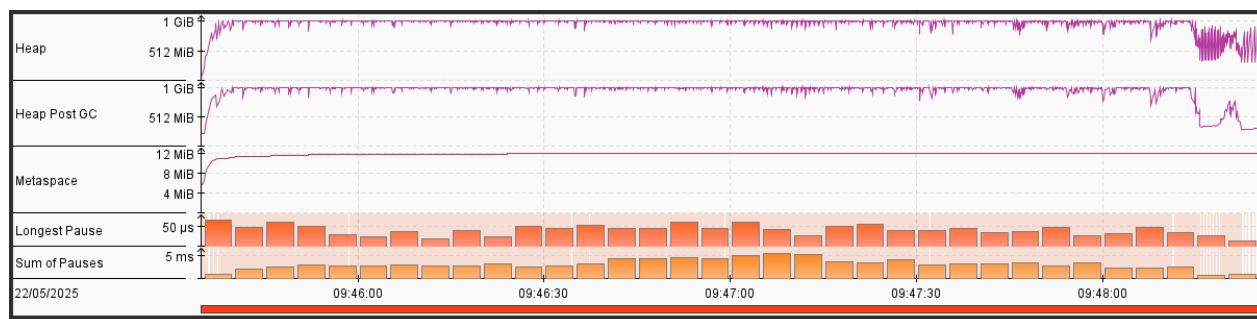


Figura 92: *Garbage Collector. ZGC. Versão Platform em Java*

Young Collection Total Time		Old Collection Total Time	
GC Count	0	GC Count	0
Average GC Time	N/A	Average GC Time	N/A
Maximum GC Time	N/A	Maximum GC Time	N/A
Total GC Time	N/A	Total GC Time	N/A
All Collections Total Time		All Collections Pause Time	
GC Count	2.907	Average Pause	12,274 μs
Average GC Time	94,350 ms	Longest Pause	66,642 μs
Maximum GC Time	1,562 s	Sum of Pauses	115,539 ms
Total GC Time	4 min 34 s		

Figura 93: *Garbage Collector Summary. ZGC. Versão Platform em Java*

6.2. Haskell

Como em Java, a única diferença entre a implementação da versão Hybrid e Platform é a utilização da função `forkOS` em vez de `forkIO` no código. Portanto, considera-se que a implementação já foi discutida em seções anteriores.

6.2.1. Resultados da Implementação

Observação:

- Os experimentos foram feitos com apenas o GC padrão de Haskell (GC geracional).

6.2.2. Avaliação com Microbenchmark

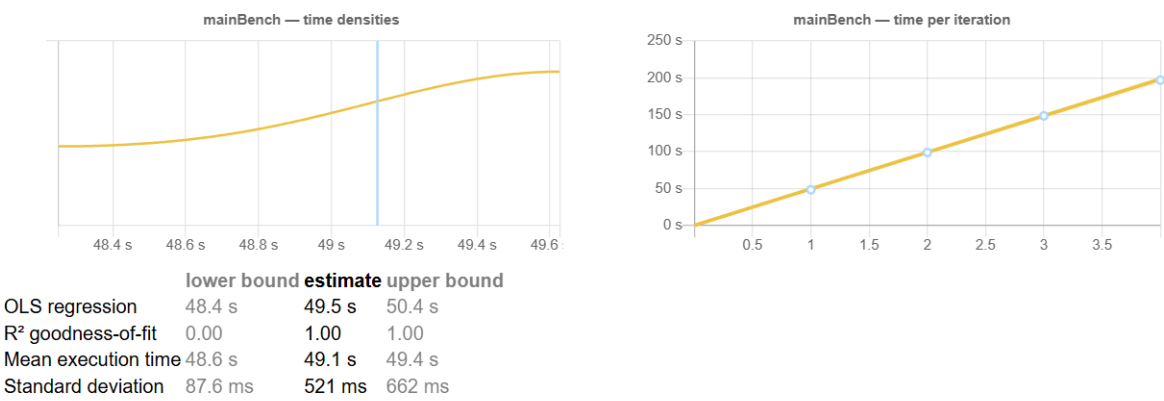
Esta não apresenta diferenças à feita para a versão Hybrid, logo não foi inclusa.

6.2.3. Avaliação com Macrobenchmark

overview



mainBench



Outlying measurements have a moderate (18.8%) effect on estimated standard deviation.

Figura 94: Resultados do Criterion para main da versão Platform em Haskell.

6.2.4. Avaliação de Profile

GC	30509007000ns	30.509s
USER EVENTS (user events are corrected for GC)		
CalculatingScore	18460069300ns	18.460s
ProcessingDocs	18455045700ns	18.455s
GatheringDocs	504300ns	0.001s
TOTAL	36915619300ns	36.916s
THREAD EVENTS		
1	259800ns	0.000s
IOManagerThread:2	64500ns	0.000s
3	2645200ns	0.003s
4	168528900ns	0.169s
5	2685829800ns	2.686s
6	643481900ns	0.643s
7	2800074900ns	2.800s
8	787711700ns	0.788s
9	527702100ns	0.528s
10	586208300ns	0.586s
11	11284722500ns	11.285s
12	1337397800ns	1.337s
13	16952208700ns	16.952s
14	2528151100ns	2.528s
15	802774800ns	0.803s
weak finalizer thread:16	36500ns	0.000s
weak finalizer thread:17	12000ns	0.000s
### tempos omitidos para visualização ###		
42	12600ns	0.000s
43	8300ns	0.000s
44	7800ns	0.000s
45	7500ns	0.000s
46	7400ns	0.000s
47	6400ns	0.000s
TOTAL	41108047100ns	41.108s

Figura 95: Resultados do ghc-events-analyze para versão Platform em Haskell.

6.3. Discussão

Esta seção primeiro irá abordar as diferenças entre Java e Haskell, em seguida a diferença entre as execuções em Java com G1GC e ZGC.

Em primeiro lugar, com relação ao tempo de execução, observa-se que a média para a versão Haskell é de 49,1 segundos (Figura 94), já a versão em Java (G1GC), 47,9 segundos (Figura 77). Novamente, a versão em Java é mais rápida, e ao fazer uma extrapolação linear do tempo de execução de Haskell para o conjunto de dados de 1,1 GB, obtém-se aproximadamente 9 minutos.

Com relação a porcentagem do tempo de execução da versão em Haskell gasta pelo GC, na versão Platform isto é 42%, essencialmente igual à versão Hybrid e Virtual, ou seja, ineficiente. Em comparação, a versão em Java (G1GC) gastou 18,2% do seu tempo com GC (cálculo feito com base na Figura 87 e nos dados do *flight*), o que também não é desejado. Portanto, como nas versões anteriores, recomenda-se um melhor uso do GC para a melhoria do desempenho em ambas as linguagens, seguindo o que foi sugerido em 3.3.

Com relação à Microbenchmark, como citado em 5.2.2, as versões Hybrid e Virtual em Haskell tem benchmarks idênticas. Ademais, esta não apresentou resultados relevantes que já não foram discutidos em 3.3.

Em suma, a versão de Java superou drasticamente a versão em Haskell na Avaliação de Profile e Macrobenchmark. Entretanto, com relação à Microbenchmark, Haskell se saiu melhor.

Em seguida, para as diferenças entre as execuções com G1GC e ZGC, novamente o ZGC obteve uma performance superior, entretanto com um tempo de execução maior: 47,9 segundos para G1GC e 49,0 segundos para ZGC (Figura 77)

- A execução com ZGC apresentou uma avaliação de profile menos problemática (Figuras 82 e 88).
- O uso de GC com o ZGC foi de apenas 0,06% (Figura 93, em comparação à 18,2% do G1GC (Figura 87)).
- O gráfico do uso da Heap do G1GC aparenta ter quedas maiores, com uma frequência parecida com o ZGC. (Figuras 86 e 92).

Com relação ao comportamento das threads (Figuras 83 e 89), ambas execuções parecem apresentar o mesmo comportamento e não há nada de relevante a se notar.

Quanto a contenção (Figuras 84 e 90), ambas apresentam um nível de contenção consideravelmente maior do que as versões apresentadas até agora. Observa-se que a versão com G1GC (Figura 83) teve uma contenção maior do que a com ZGC (Figura 89) no caso do método `java.util.logging.ConsoleHandler`, o qual não tem níveis de contenção semelhante no ZGC (visto que nem aparece na captura de tela). Entretanto, para os outros métodos, a versão com ZGC parece ter tido um nível de contenção maior. Não se sabe o porquê deste comportamento.

Ambas versões apresentam *Hot methods* similares, com um resultado semelhante ao das versões híbridas em 4.3.

Por fim, quanto ao *Throughput* e *Response Time*, percebe-se que a execução com ZGC teve um desempenho pior, algo esperado devido a sua execução mais lenta. Dessa

forma, como visto na Figura 81, o *Response Time* médio do G1GC é 73,3% daquele do ZGC e o *Throughput* o dobro.

7. Implementação Concorrente - Atomic e STM

Ambas variáveis atômicas em Java e STM em Haskell são locks otimistas, portanto decidi agrupá-los na mesma seção.

7.1. Java

A única diferença entre a implementação Atomic e Hybrid é a utilização de variáveis atômicas para resolução de condições de corrida, em vez do `synchronized`, nas partes presentes nas Figuras 96 e 97. Mais especificamente, na primeira imagem, a variável `total` agora é do tipo `AtomicInteger` e os métodos `increment` e `get` não possuem mais o modificador `synchronized`. Já na segunda imagem, as variáveis `highest_bidder` e `highest_bidder_score` agora são atômicas e os métodos `challenge_highest_bidder` e `get` não utilizam mais `synchronized`.

```
108 @l public static class Counter { 4 usages 2 inheritors
109     private final AtomicInteger total; 3 usages
110     protected final CountDownLatch controller; 3 usages
111
112     public Counter(CountDownLatch controller) { 3 usages
113         this.controller = controller;
114         this.total = new AtomicInteger(initialValue: 0);
115     }
116
117 >     public void increment() { total.incrementAndGet(); }
120
121 >     public int get() { return total.get(); }
124
```

Figura 96: Classe `Counter` na versão Atomic em Java

```

99  public class Auction { 4 usages 2 inheritors
100      public AtomicReference<DocumentData> highest_bidder; 3 usages
101      protected final AtomicReference<Double> highest_bidder_score; 4 usages
102      protected final CountDownLatch controller; 3 usages
103
104      public Auction(CountDownLatch controller) { 3 usages
105          this.controller = controller;
106          this.highest_bidder = new AtomicReference<> ( initialValue: null);
107          this.highest_bidder_score = new AtomicReference<> (Double.NEGATIVE_INFINITY);
108      }
109
110      public void challenge_highest_bidder(Double bid, DocumentData doc) { 4 usages
111          if (bid >= highest_bidder_score.get()) {
112              highest_bidder_score.set(bid);
113              highest_bidder.set(doc);
114          }
115      }
116
117      > public DocumentData get_highest_bidder() { return highest_bidder.get(); }
120

```

Figura 97: Classe Auction na versão Atomic em Java

7.1.1. Resultados da Implementação

Ver as observações em 3.1.1.

7.1.2. Avaliação de Condição de Corrida

All tests

JCStressTests.BM25_JCStressTest.BM25_add_StressTest	10 ³	PASSED
JCStressTests.BM25_JCStressTest.BM25_challenge_highest_bidder_StressTest	10 ³	PASSED
JCStressTests.DocumentData_JCStressTest.DocumentData_get_token_frequency_StressTest	10 ⁸	PASSED

Figura 98: Resultados do JCStress para a versão Atomic em Java.

7.1.3. Avaliação com Microbenchmark

Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.BM25_BenchTest.test_IDF	thrpt	8	14,640 ±	0,185	ops/ms
JMHTests.BM25_BenchTest.test_add	thrpt	8	3,706 ±	0,058	ops/ms
JMHTests.BM25_BenchTest.test_auction_challenge_highest_bidder_doc	thrpt	8	5,290 ±	0,035	ops/ms
JMHTests.BM25_BenchTest.test_auction_spawn_thread	thrpt	8	0,111 ±	0,014	ops/ms
JMHTests.BM25_BenchTest.test_constructor	thrpt	8	4,629 ±	0,040	ops/ms
JMHTests.BM25_BenchTest.test_get_most_relevant_doc	thrpt	8	0,003 ±	0,001	ops/ms
JMHTests.BM25_BenchTest.test_score	thrpt	8	3,261 ±	0,017	ops/ms
JMHTests.BM25_BenchTest.test_update_IDF	thrpt	8	11,479 ±	0,200	ops/ms
JMHTests.DocumentData_BenchTest.test_cleanString	thrpt	8	243,236 ±	3,126	ops/s
JMHTests.DocumentData_BenchTest.test_constructor	thrpt	4	0,149 ±	0,002	ops/s
JMHTests.DocumentData_BenchTest.test_counter_spawn_thread	thrpt	8	77,008 ±	13,460	ops/s
JMHTests.DocumentData_BenchTest.test_get_token_frequency	thrpt	8	23,862 ±	0,681	ops/s
JMHTests.DocumentData_BenchTest.test_get_tokenizers	thrpt	8	54,584 ±	0,476	ops/ms
JMHTests.Query_BenchTest.test_constructor	thrpt	8	1,096 ±	0,095	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_Array	thrpt	8	2,694 ±	0,096	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_ArrayList	thrpt	8	5,902 ±	0,220	ops/ms

Figura 99: Resultados do JMH para os métodos da versão Atomic em Java.

7.1.4. Avaliação com Macrobenchmark

-- G1GC					
Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.Main_BenchTest.test_main	avgt	8	45553,879 ±	1844,740	ms/op
-- ZGC					
Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.Main_BenchTest.test_main	avgt	8	46141,143 ±	1309,079	ms/op

Figura 100: Tempo médio de execução . Versão Atomic em Java

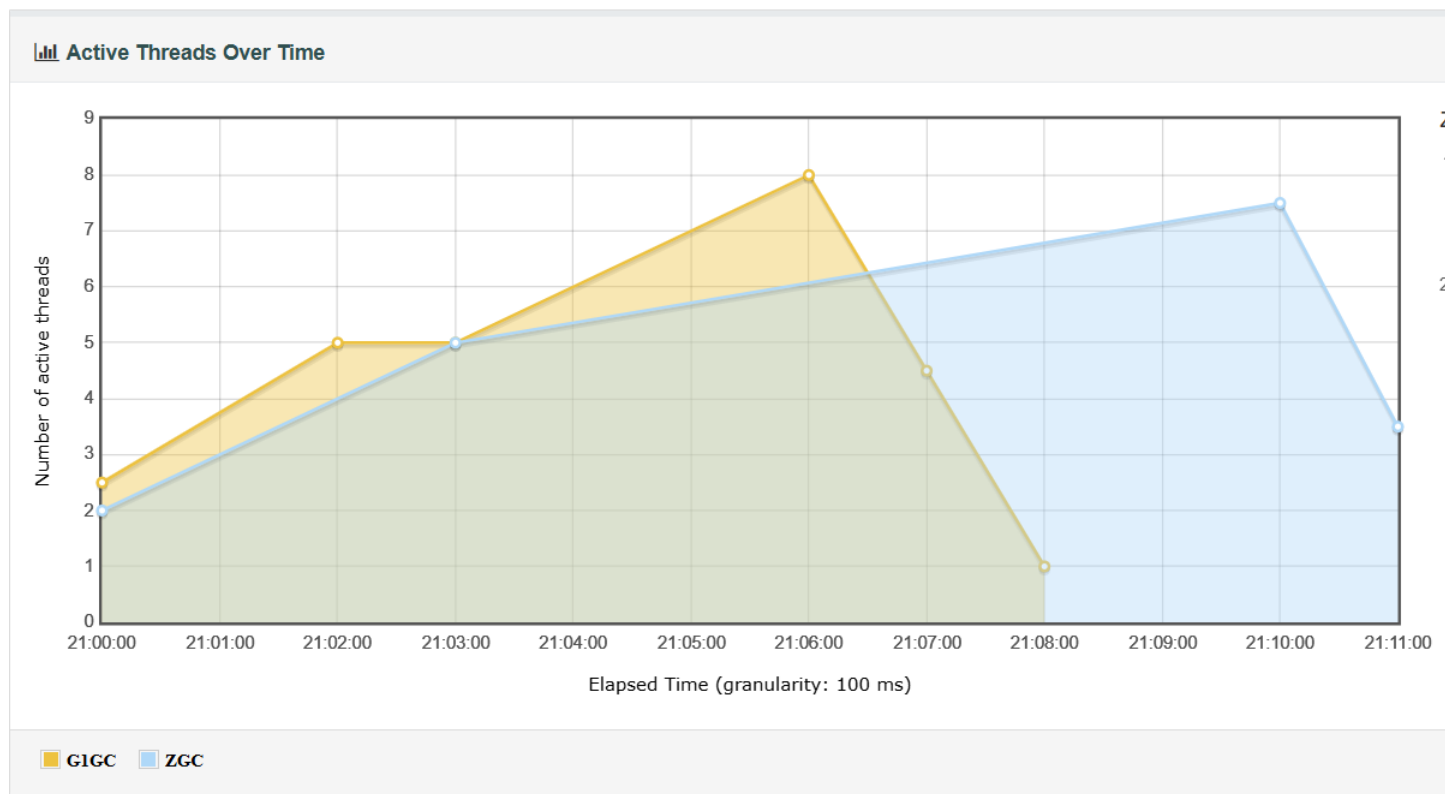


Figura 101: Número de threads ativas × tempo . Versão Atomic em Java

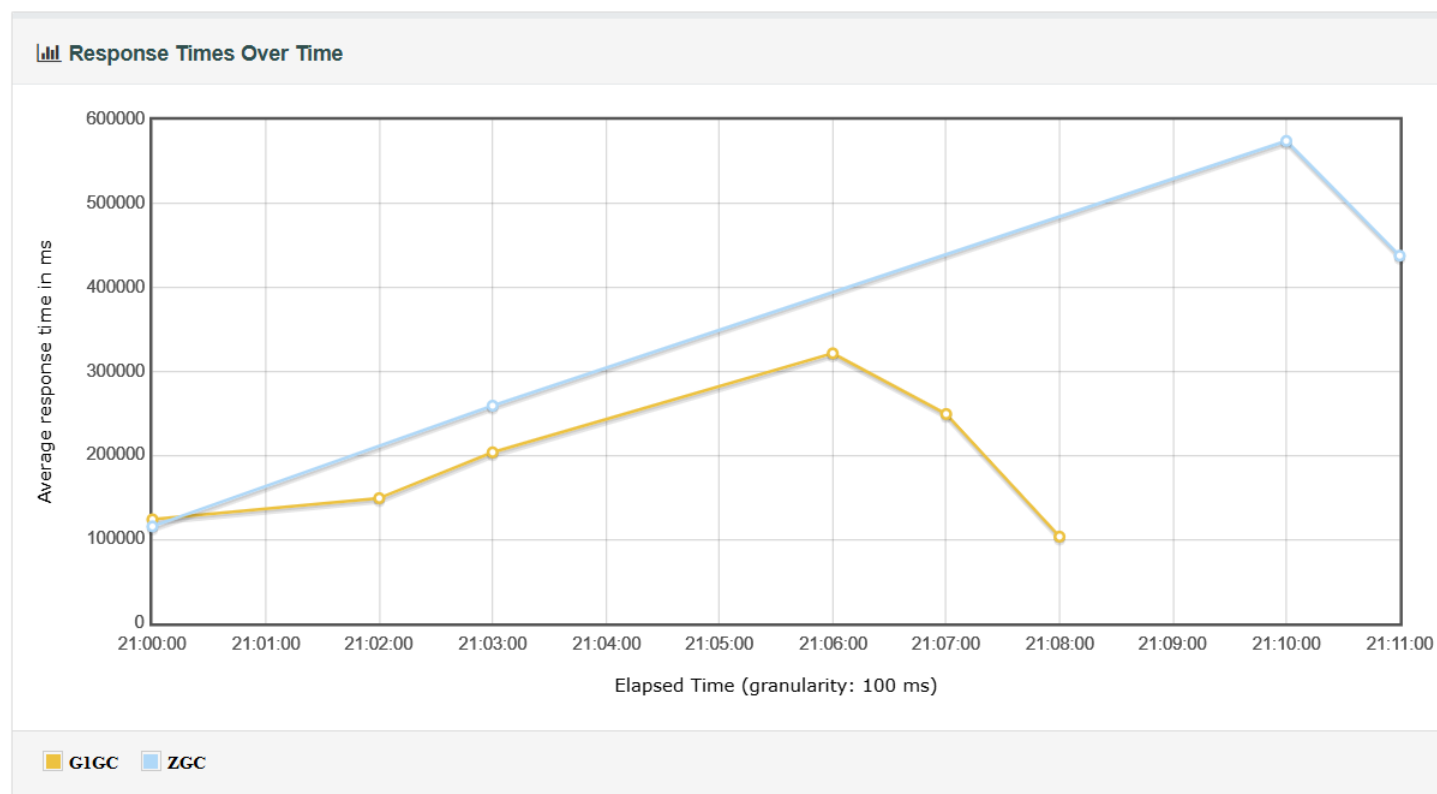


Figura 102: Tempo de resposta × tempo. Versão Atomic em Java

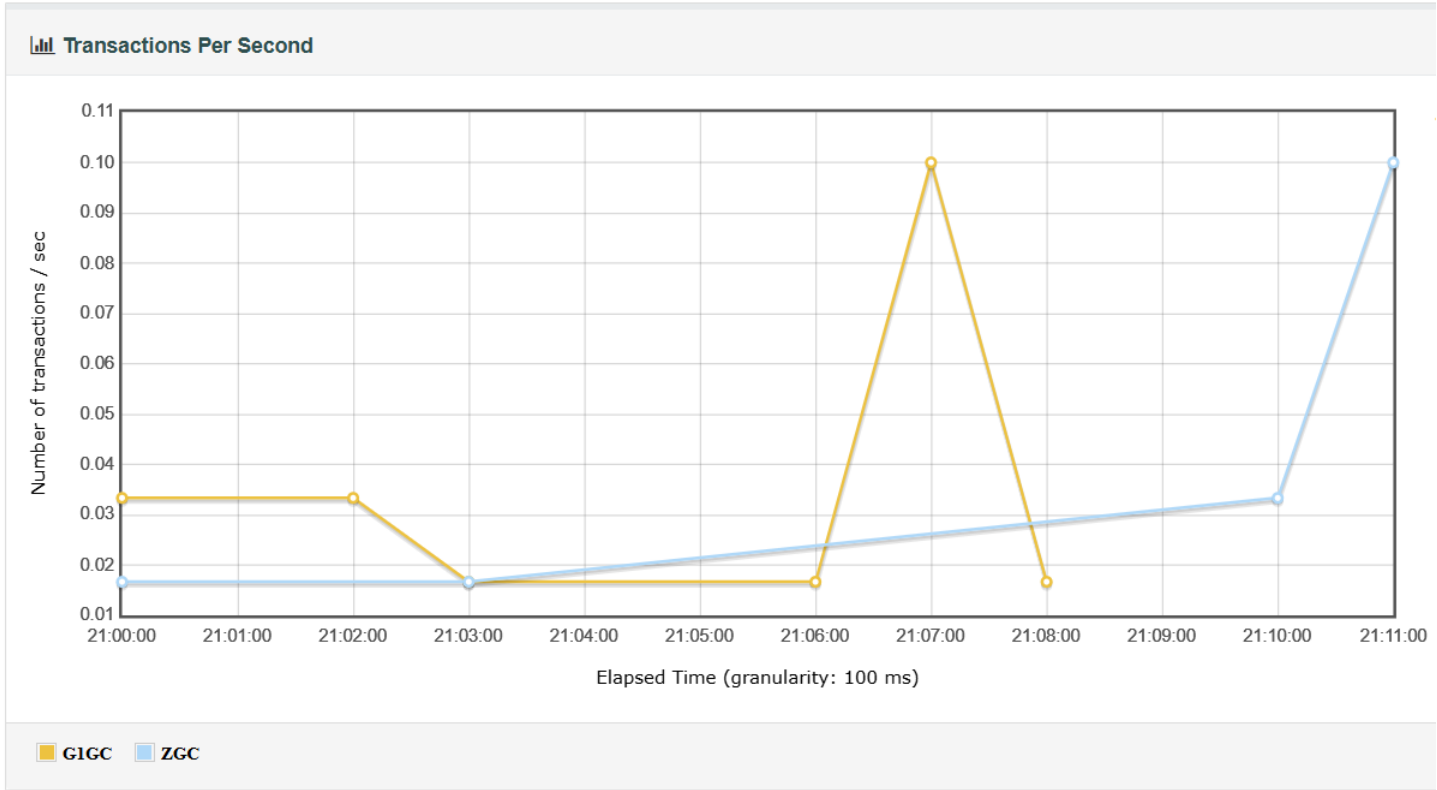


Figura 103: *Throughput* × tempo. Versão Atomic em Java

Statistics											G1GC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ↕	FAIL ↕	Error % ↕	Average ↕	Min ↕	Max ↕	Median ↕	90th pct ↕	95th pct ↕	99th pct ↕	Transactions/s ↕	Received ↕	Sent ↕
Total	13	0	0.00%	205530.23	103316	321807	203804.00	309411.80	321807.00	321807.00	0.02	0.00	0.00
Atomic Test Sample	13	0	0.00%	205530.23	103316	321807	203804.00	309411.80	321807.00	321807.00	0.02	0.00	0.00

Statistics											ZGC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ↕	FAIL ↕	Error % ↕	Average ↕	Min ↕	Max ↕	Median ↕	90th pct ↕	95th pct ↕	99th pct ↕	Transactions/s ↕	Received ↕	Sent ↕
Total	10	0	0.00%	415394.10	115497	592801	457958.50	589236.00	592801.00	592801.00	0.01	0.00	0.00
Atomic Test Sample	10	0	0.00%	415394.10	115497	592801	457958.50	589236.00	592801.00	592801.00	0.01	0.00	0.00

Figura 104: Estatísticas. Versão Atomic em Java

Observações:

- A versão que utilizou G1GC não apresentou conflito de contenção, portanto uma imagem para isto não foi incluída.
- O número 21 no eixo x se refere ao dia em que a medição foi feita.

7.1.5. Avaliação de Profile

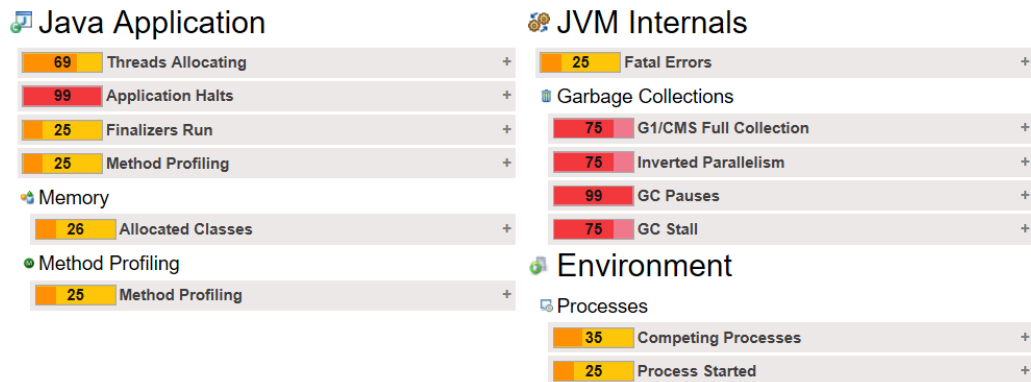


Figura 105: Resumo do JMC. G1GC. Versão Atomic em Java

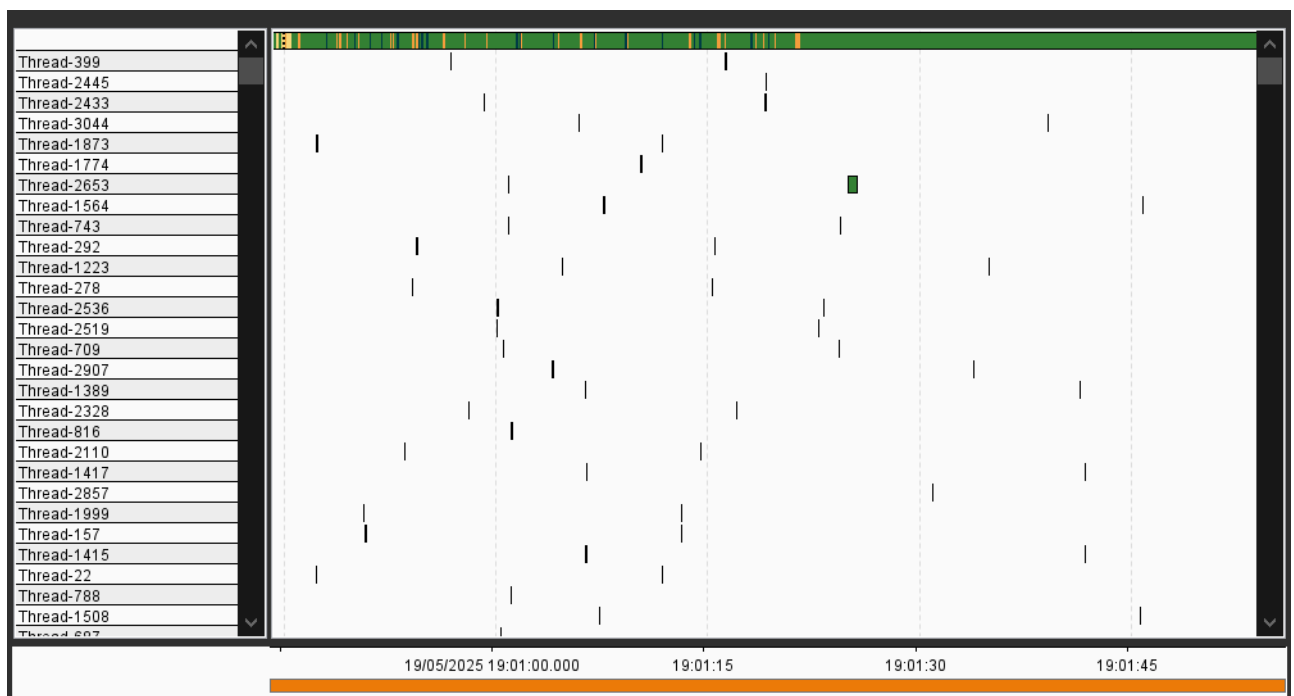


Figura 106: Threads. G1GC. Versão Atomic em Java

Method	Count	Percentage
java.util.TreeMap.getCeilingEntry(Object)	523	9,75 %
java.io.FilterInputStream.read()	436	8,13 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	309	5,76 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	245	4,57 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	225	4,2 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	220	4,1 %
java.io.PushbackInputStream.read()	217	4,05 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	187	3,49 %
org.apache.pdfbox.text.PDFTextStripper.processTextPosition(Text...	136	2,54 %
java.math.BigDecimal.<init>(char[], int, int, MathContext)	111	2,07 %
org.apache.pdfbox.contentstream.operator.Operator.<init>(String)	110	2,05 %

Figura 107: *Hot methods*. G1GC. Versão Atomic em Java

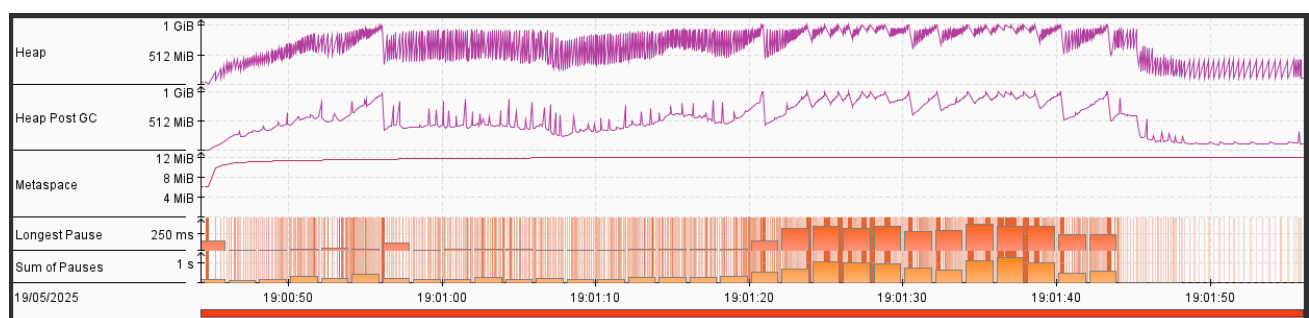


Figura 108: *Garbage Collector*. G1GC. Versão Atomic em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	842	GC Count	121
Average GC Time	11,050 ms	Average GC Time	379,249 ms
Maximum GC Time	151,827 ms	Maximum GC Time	1,417 s
Total GC Time	9,304 s	Total GC Time	45,889 s
All Collections Total Time		All Collections Pause Time	
GC Count	963	Average Pause	15,444 ms
Average GC Time	57,314 ms	Longest Pause	393,666 ms
Maximum GC Time	1,417 s	Sum of Pauses	15,861 s
Total GC Time	55,193 s		

Figura 109: *Garbage Collector Summary*. G1GC. Versão Atomic em Java

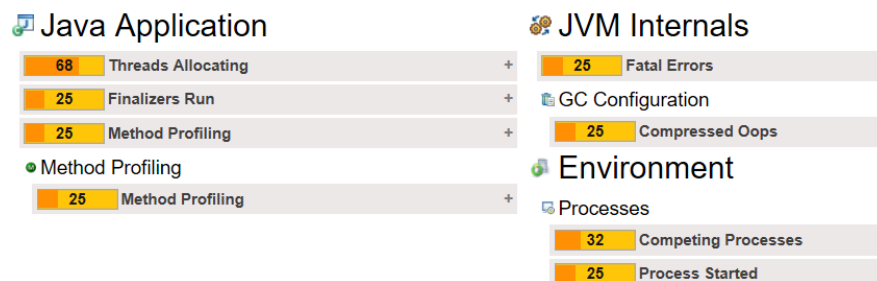


Figura 110: Resumo do JMC. ZGC. Versão Atomic em Java

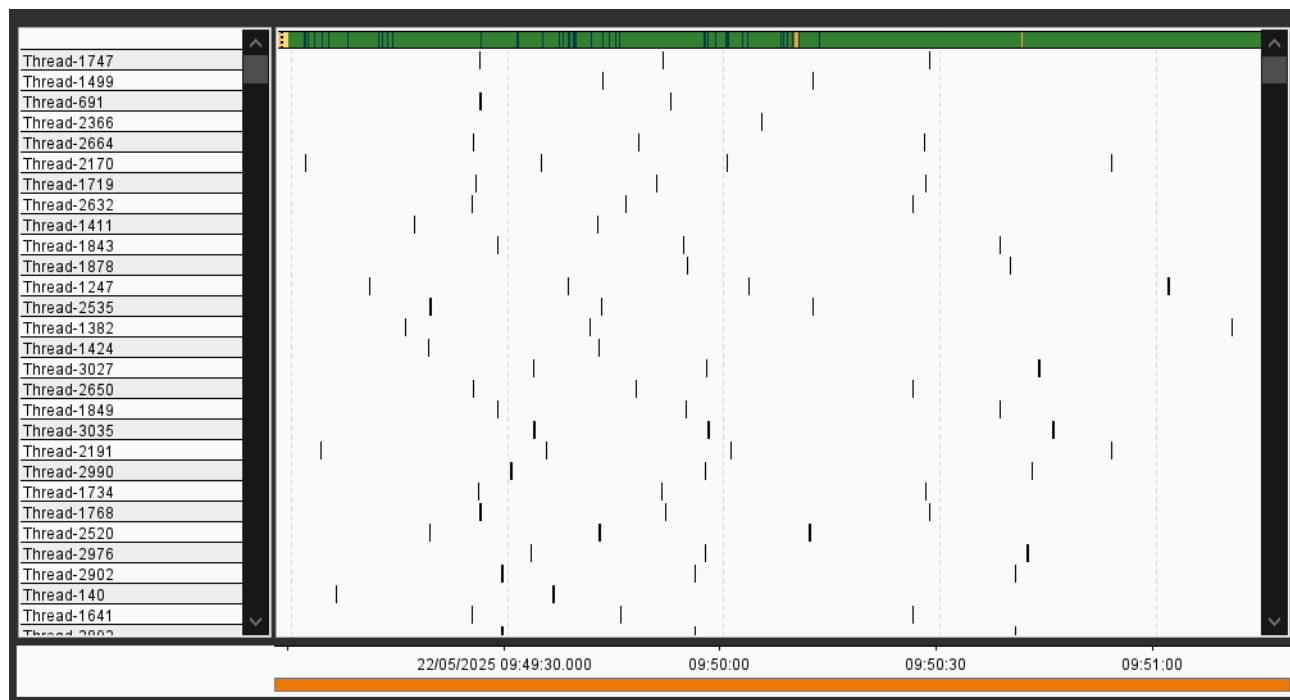


Figura 111: Threads. ZGC. Versão Atomic em Java

Monitor Class	Total Blocked Time	Distinct Threads	Count
java.lang.Object	525,925 ms	1	11
int[]	69,468 ms	1	2

Monitor Address	Total Blocked Time	Distinct Threads	Count
0x123DE6993C0	525,925 ms	1	11
0x123DE6EA120	69,468 ms	1	2

Thread	Total Blocked Time	Count
	595,393 ms	13

Figura 112: Contenção. ZGC. Versão Atomic em Java

Method	Count	Percentage
java.util.TreeMap.getCeilingEntry(Object)	875	10,6 %
java.io.FilterInputStream.read()	753	9,13 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	494	5,99 %
java.io.PushbackInputStream.read()	392	4,75 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	382	4,63 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	379	4,6 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	354	4,29 %
java.util.TreeMap.put(Object, Object, boolean)	317	3,85 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	285	3,46 %
org.apache.pdfbox.text.PDFTextStripper.processTextPosition(Text...	186	2,26 %
java.math.BigDecimal.<init>(char[], int, int, MathContext)	155	1,88 %

Figura 113: Hot methods. ZGC. Versão Atomic em Java

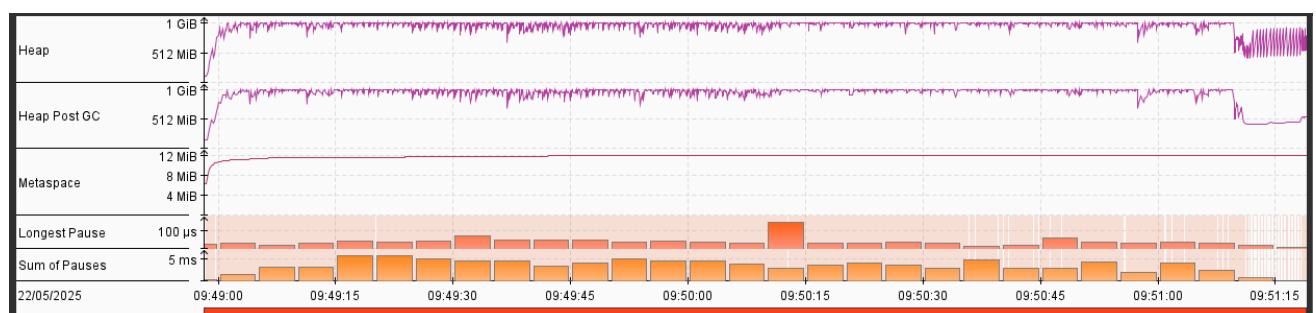


Figura 114: Garbage Collector. ZGC. Versão Atomic em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	0	GC Count	0
Average GC Time	N/A	Average GC Time	N/A
Maximum GC Time	N/A	Maximum GC Time	N/A
Total GC Time	N/A	Total GC Time	N/A
All Collections Total Time		All Collections Pause Time	
GC Count	2,613	Average Pause	12,588 μ s
Average GC Time	87,120 ms	Longest Pause	151,959 μ s
Maximum GC Time	1,139 s	Sum of Pauses	105,123 ms
Total GC Time	3 min 48 s		

Figura 115: *Garbage Collector Summary*. ZGC. Versão Atomic em Java

7.2. Haskell

A versão STM difere da versão Hybrid pelo uso de `TVars` no lugar de `MVars`. Esta mudança levou a algumas modificações em funções que lidavam com `MVars`. A Figura 116 mostra um exemplo desta troca. A maior diferença está no fato da necessidade de criação de funções para o bloqueamento, quando isto for necessário, pois esta funcionalidade já vinha embutida no uso das `MVar`'s, mas não em `TVar`'s. Tal implementação foi feita por meio de uma chamada recursiva a qual só terminava quando sinalizada para tal.

overview

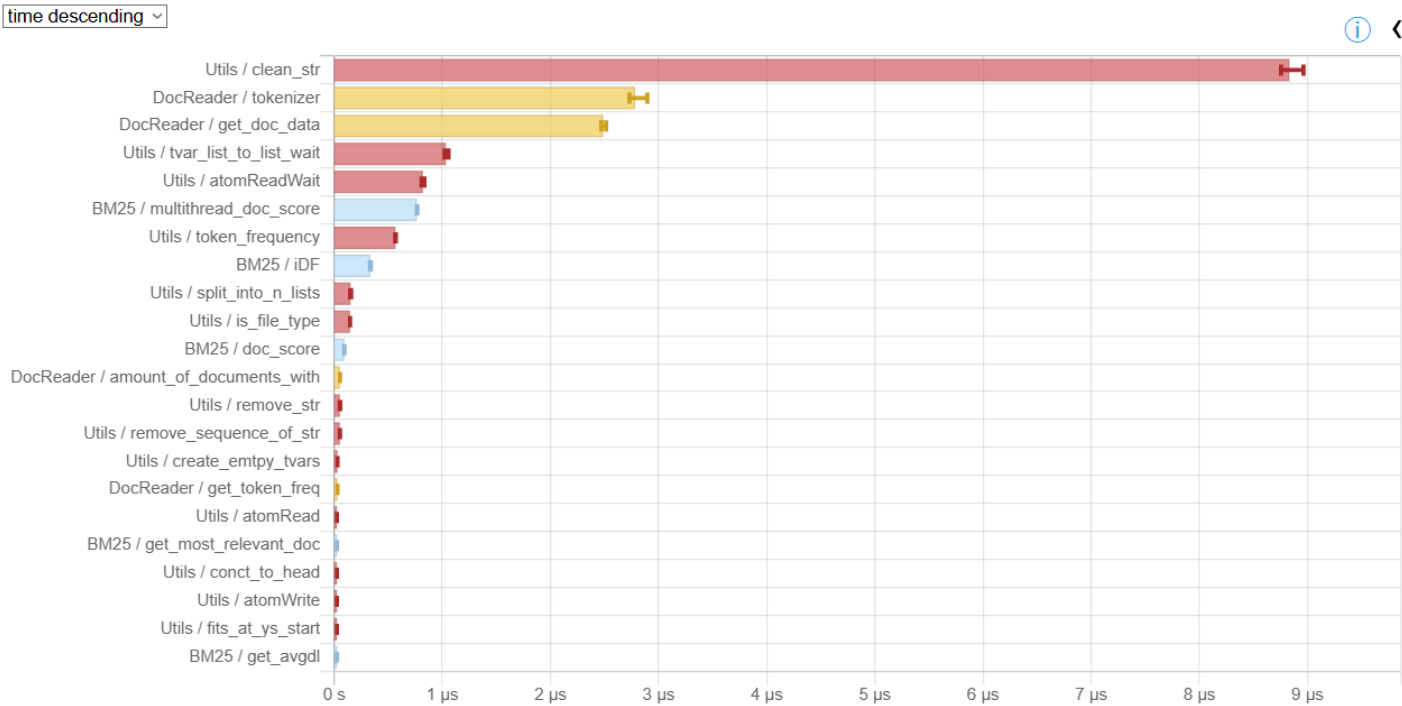


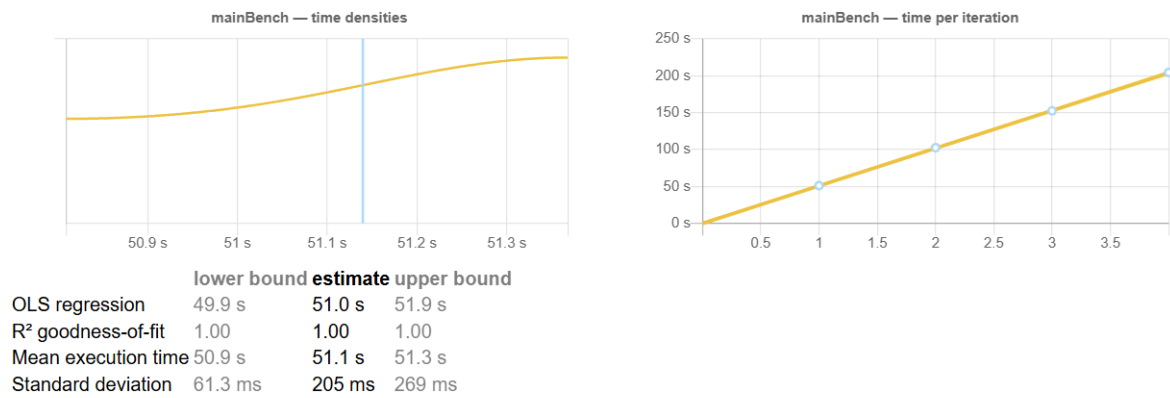
Figura 117: Resultados do Criterion para as funções da versão STM em Haskell.

7.2.3. Avaliação com Macrobenchmark

overview



mainBench



Outlying measurements have a moderate (18.8%) effect on estimated standard deviation.

Figura 118: Resultados do Criterion para main da versão STM em Haskell.

7.2.4. Avaliação de Profile

GC	31016684800ns	31.017s
USER EVENTS (user events are corrected for GC)		
CalculatingScore	18942790100ns	18.943s
ProcessingDocs	18938746500ns	18.939s
GatheringDocs	489200ns	0.000s
TOTAL	37882025800ns	37.882s
THREAD EVENTS		
1	267900ns	0.000s
IOManagerThread:2	66700ns	0.000s
3	19239567200ns	19.240s
4	166831300ns	0.167s
5	2397042700ns	2.397s
6	548188300ns	0.548s
7	2505590700ns	2.506s
8	681704700ns	0.682s
9	452614500ns	0.453s
10	531469400ns	0.531s
11	12236676100ns	12.237s
12	1078240500ns	1.078s
### tempos omitidos para visualização ###		
44	11900ns	0.000s
45	7800ns	0.000s
46	7300ns	0.000s
47	7100ns	0.000s
TOTAL	60265550500ns	60.266s

Figura 119: Resultados do ghc-events-analyze para versão STM em Haskell.

7.3. Discussão

Esta seção primeiro irá abordar as diferenças entre Java e Haskell, em seguida a diferença entre as execuções em Java com G1GC e ZGC.

Em primeiro lugar, com relação ao tempo de execução, observa-se que a média para a versão Haskell é de 51,1 segundos (Figura 118), já a versão em Java (G1GC), 45,5 segundos (Figura 100). Novamente, a versão em Java é mais rápida, e ao fazer uma extrapolação linear do tempo de execução de Haskell para o conjunto de dados de 1,1 GB, obtém-se aproximadamente 9 minutos.

Com relação a porcentagem do tempo de execução da versão em Haskell gasta pelo GC, na versão Atomic isto é 34%, a menor das versões até agora, mas ainda ineficiente. Em comparação, a versão em Java (G1GC) gastou 22,1% do seu tempo com GC (cálculo feito com base na Figura 109 e nos dados do *flight*), o que também não é desejado. Portanto, como nas versões anteriores, recomenda-se um melhor uso do GC para a melhoria do desempenho em ambas as linguagens, seguindo o que foi sugerido em 3.3.

Com relação à Microbenchmark, esta não apresentou resultados relevantes que já não foram discutidos em 3.3, exceto o fato de um novo método em Haskell, `printTVar`, ter um tempo de execução comparável à `tokenizeDoc`, como visto em 7.2.2, pois ambos realizam um `print` na tela.

Em suma, a versão de Java superou drasticamente a versão em Haskell na Avaliação de Profile e Macrobenchmark. Entretanto, com relação à Microbenchmark, Haskell se saiu melhor.

Em seguida, para as diferenças entre as execuções com G1GC e ZGC, novamente o ZGC obteve uma performance superior, entretanto com um tempo de execução maior: 45,5 segundos para G1GC e 46,1 segundos para ZGC (Figura 100).

- A execução com ZGC apresentou uma avaliação de profile menos problemática (Figuras 105 e 110).
- O uso de GC com o ZGC foi de apenas 0,07% (Figura 115, em comparação à 22,1% do G1GC (Figura 109)).
- O gráfico do uso da Heap do G1GC aparenta ter quedas maiores, com uma frequência parecida com o ZGC. (Figuras 108 e 114).

Com relação ao comportamento das threads (Figuras 106 e 111), ambas execuções parecem apresentar o mesmo comportamento e não há nada de relevante a se notar.

Quanto a contenção (Figura 112), apenas ocorreu na execução com o G1GC e de maneira não significativa.

Ambas versões apresentam *Hot methods* similares, com um resultado semelhante ao das versões seriais (3.3) e híbridas (4.3).

Por fim, quanto ao *Throughput* e *Response Time*, percebe-se que a execução com ZGC teve um desempenho pior, algo esperado devido a sua execução mais lenta. Dessa forma, como visto na Figura 104, o *Response Time* médio do G1GC é 49,4% daquele do ZGC e o *Throughput* o dobro.

8. Implementação Concorrente - Volatile

8.1. Java

A única diferença é a versão Hybrid e a Volatile é a declaração de certas variáveis como voláteis (Figuras 120 e 121). Vale notar que os modificadores `synchronized` ainda são utilizados para a resolução de conflitos.

```
108 @l public static class Counter { 4 usages 2 inheritors
109     private volatile int total = 0; 2 usages
110     protected final CountDownLatch controller; 3 usages
111 }
```

Figura 120: Classe Counter na versão Volatile em Java

```
98 @l public class Auction { 4 usages 2 inheritors
99     public DocumentData highest_bidder = null; 2 usages
100     protected volatile double highest_bidder_score = Double.NEGATIVE_INFINITY; 3 usages
101     protected final CountDownLatch controller; 3 usages
102 }
```

Figura 121: Classe Auction na versão Volatile em Java

8.1.1. Resultados da Implementação

Ver as observações em 3.1.1.

8.1.2. Avaliação de Condição de Corrida

All tests		
JCStressTests.BM25.JCStressTest.BM25_add_StressTest	10 ³	PASSED
JCStressTests.BM25.JCStressTest.BM25_challenge_highest_bidder_StressTest	10 ³	PASSED
JCStressTests.DocumentData.JCStressTest.DocumentData_get_token_frequency_StressTest	10 ⁸	PASSED

Figura 122: Resultados do JCStress para a versão Volatile em Java.

8.1.3. Avaliação com Microbenchmark

Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.BM25_BenchTest.test_IDF	thrpt	8	14,017 ±	0,127	ops/ms
JMHTests.BM25_BenchTest.test_add	thrpt	8	3,785 ±	0,045	ops/ms
JMHTests.BM25_BenchTest.test_auction_challenge_highest_bidder_doc	thrpt	8	3,347 ±	0,176	ops/ms
JMHTests.BM25_BenchTest.test_auction_spawn_thread	thrpt	8	0,122 ±	0,003	ops/ms
JMHTests.BM25_BenchTest.test_constructor	thrpt	8	4,771 ±	0,131	ops/ms
JMHTests.BM25_BenchTest.test_get_most_relevant_doc	thrpt	8	0,003 ±	0,001	ops/ms
JMHTests.BM25_BenchTest.test_score	thrpt	8	3,331 ±	0,010	ops/ms
JMHTests.BM25_BenchTest.test_update_IDF	thrpt	8	11,707 ±	0,057	ops/ms
JMHTests.DocumentData_BenchTest.test_cleanString	thrpt	8	238,792 ±	53,350	ops/s
JMHTests.DocumentData_BenchTest.test_constructor	thrpt	4	0,148 ±	0,044	ops/s
JMHTests.DocumentData_BenchTest.test_counter_spawn_thread	thrpt	8	77,360 ±	4,708	ops/s
JMHTests.DocumentData_BenchTest.test_get_token_frequency	thrpt	8	23,133 ±	0,480	ops/s
JMHTests.DocumentData_BenchTest.test_get_tokenizers	thrpt	8	54,956 ±	0,997	ops/ms
JMHTests.Query_BenchTest.test_constructor	thrpt	8	1,125 ±	0,039	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_Array	thrpt	8	2,597 ±	0,072	ops/ms
JMHTests.Utils_BenchTest.test_create_batch_ArrayList	thrpt	8	5,553 ±	0,285	ops/ms

Figura 123: Resultados do JMH para os métodos da versão Volatile em Java.

8.1.4. Avaliação com Macrobenchmark

-- G1GC					
Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.Main_BenchTest.test_main	avgt	8	47514,736 ±	3228,205	ms/op
-- ZGC					
Benchmark	Mode	Cnt	Score	Error	Units
JMHTests.Main_BenchTest.test_main	avgt	8	46505,874 ±	2083,741	ms/op

Figura 124: Tempo médio de execução . Versão Volatile em Java

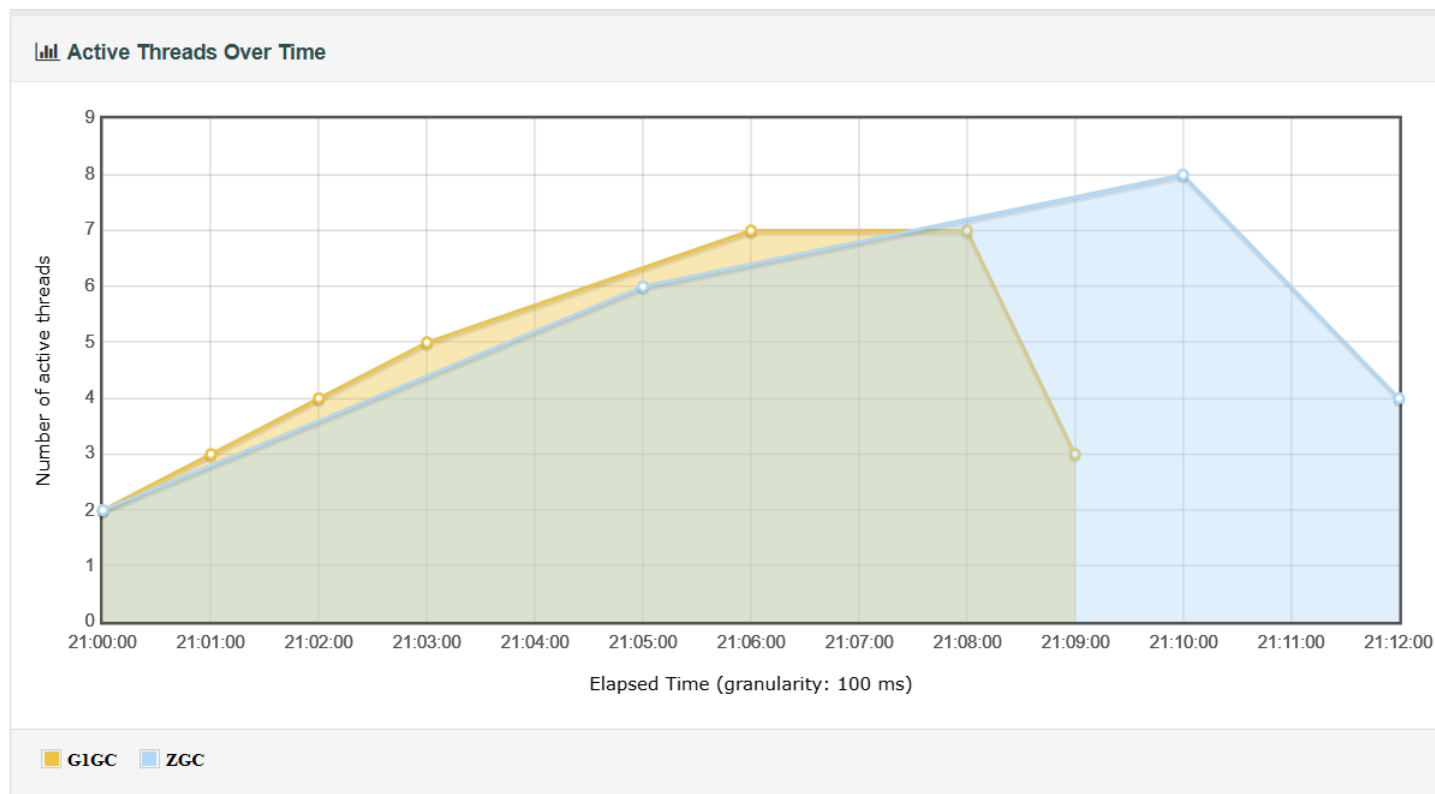


Figura 125: Número de threads ativas × tempo . Versão Volatile em Java

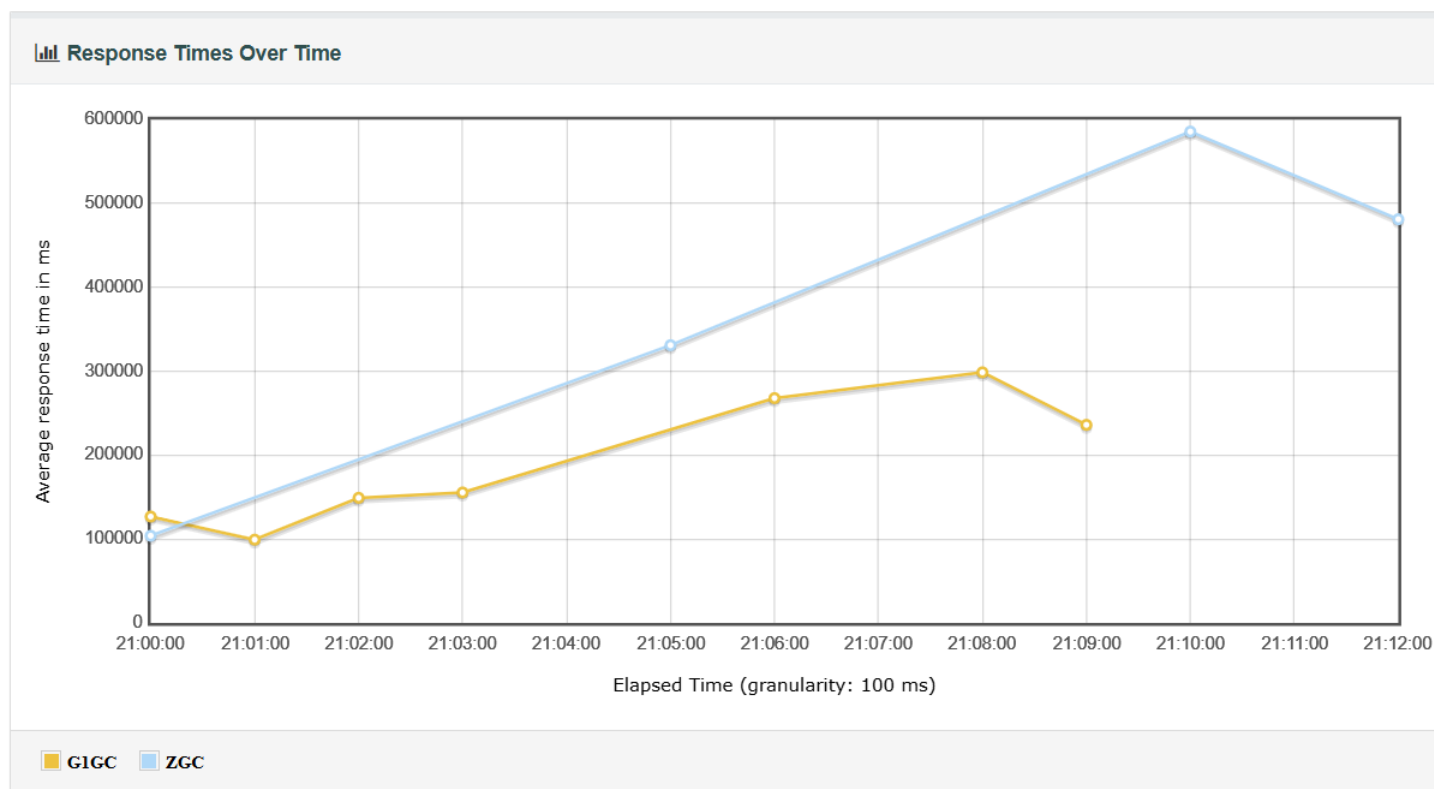


Figura 126: Tempo de resposta × tempo. Versão Volatile em Java

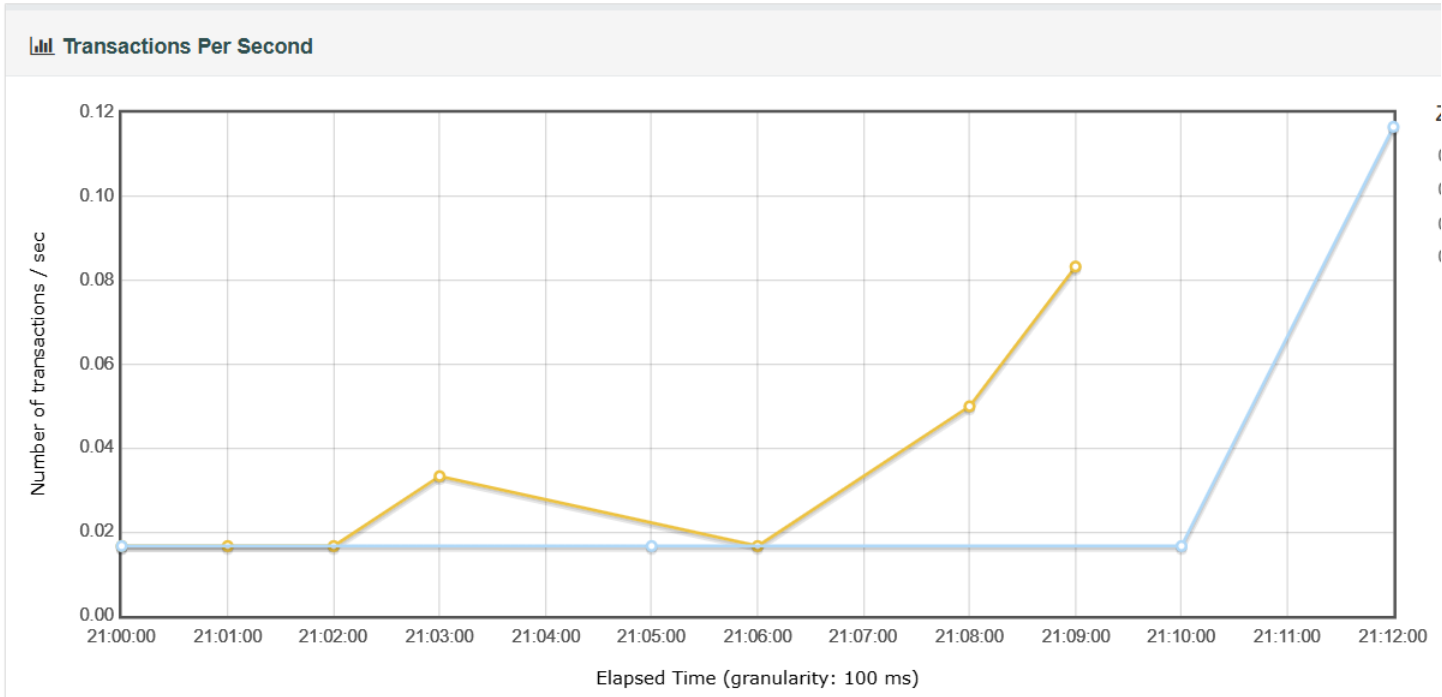


Figura 127: *Throughput* × tempo. Versão Volatile em Java

Statistics											G1GC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	14	0	0.00%	216543.07	99278	337973	189539.00	333744.50	337973.00	337973.00	0.02	0.00	0.00
Volatile Test Sample	14	0	0.00%	216543.07	99278	337973	189539.00	333744.50	337973.00	337973.00	0.02	0.00	0.00

Statistics											ZGC		
Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label ^	#Samples ^	FAIL ^	Error % ^	Average ^	Min ^	Max ^	Median ^	90th pct ^	95th pct ^	99th pct ^	Transactions/s ^	Received ^	Sent ^
Total	10	0	0.00%	439007.10	103978	659597	434072.00	654229.30	659597.00	659597.00	0.01	0.00	0.00
Volatile Test Sample	10	0	0.00%	439007.10	103978	659597	434072.00	654229.30	659597.00	659597.00	0.01	0.00	0.00

Figura 128: Estatísticas. Versão Volatile em Java

Observações:

-
- O número 21 no eixo x se refere ao dia em que a medição foi feita.

8.1.5. Avaliação de Profile

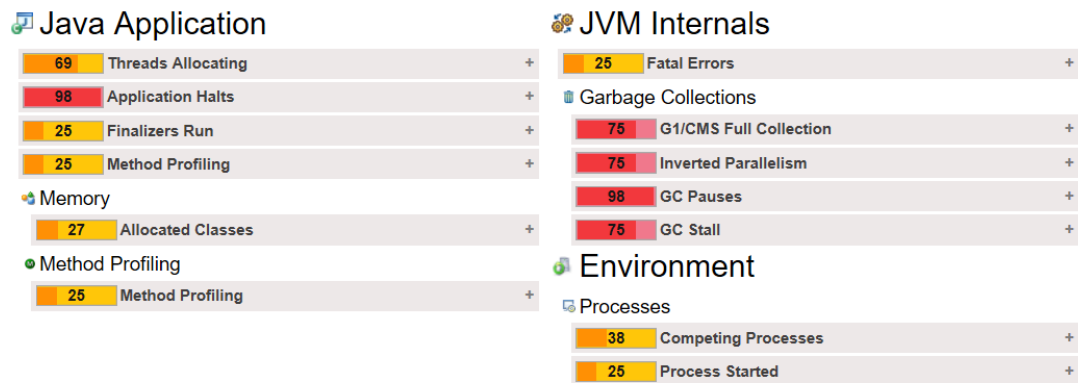


Figura 129: Resumo do JMC. G1GC. Versão Volatile em Java

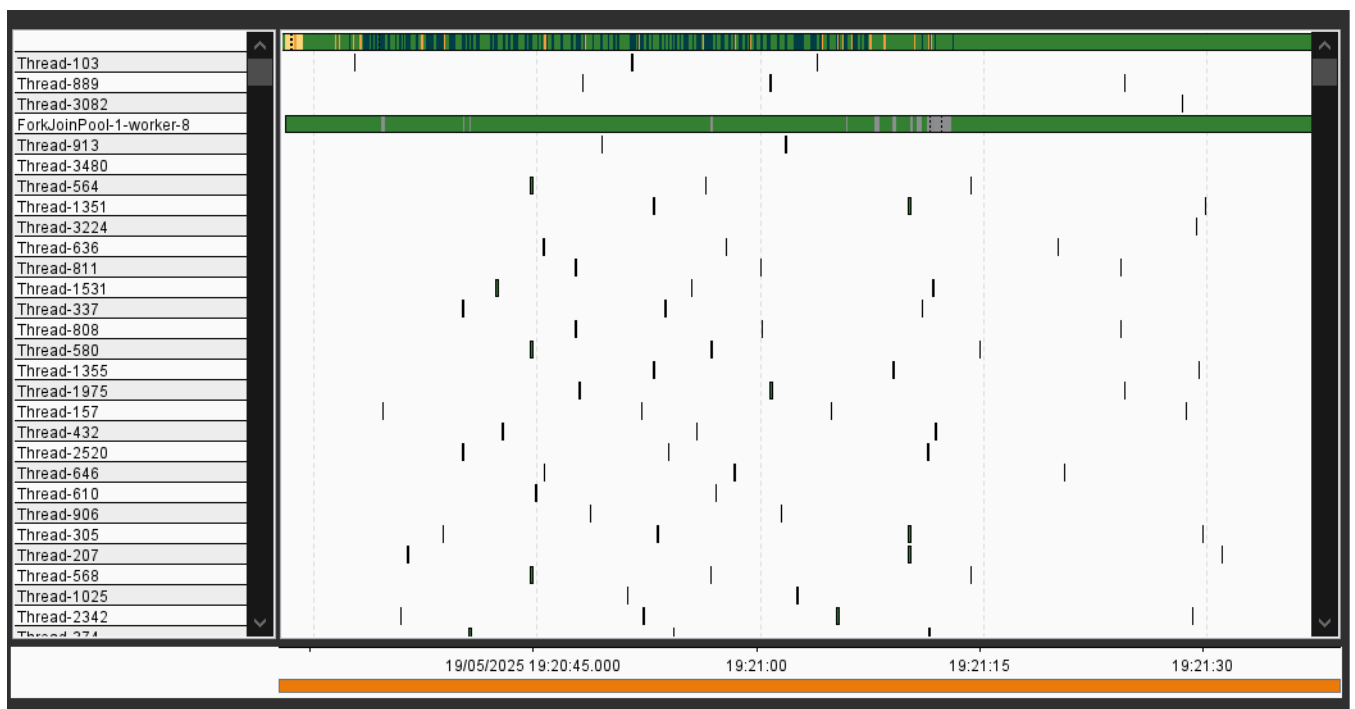


Figura 130: Threads. G1GC. Versão Volatile em Java

Monitor Class	total Blocked Time	distinct Threads	Count
int[]	43,725 ms	1	1
java.lang.Thread	24,624 ms	1	1

Monitor Address	total Blocked Time	distinct Threads	Count
0x2538B0EF470	43,725 ms	1	1
0x2538DA487C0	24,624 ms	1	1

Thread	total Blocked Time	Count
Thread-855	43,725 ms	1
	24,624 ms	1

Figura 131: Contenção. G1GC. Versão Volatile em Java

Method	Count	Percentage
java.util.TreeMap.getCeilingEntry(Object)	709	11,3 %
java.io.FilterInputStream.read()	535	8,49 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	425	6,75 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	284	4,51 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	279	4,43 %
java.io.PushbackInputStream.read()	267	4,24 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	228	3,62 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	208	3,3 %
java.util.HashMap\$TreeNode.find(int, Object, Class)	154	2,45 %
java.math.BigDecimal.<init>(char[], int, int, MathContext)	120	1,91 %
org.apache.pdfbox.pdfparser.PDFStreamParser.parseNextToken()	96	1,52 %

Figura 132: Hot methods. G1GC. Versão Volatile em Java

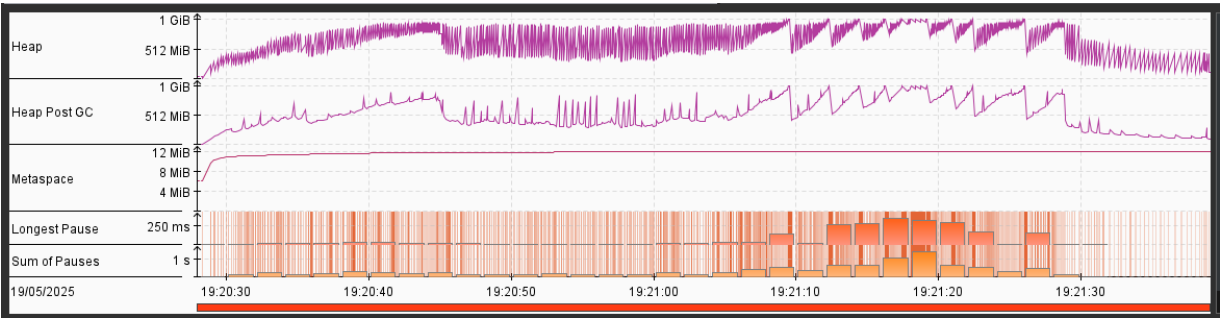


Figura 133: Garbage Collector. G1GC. Versão Volatile em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	724	GC Count	111
Average GC Time	12,468 ms	Average GC Time	359,881 ms
Maximum GC Time	50,223 ms	Maximum GC Time	1,277 s
Total GC Time	9,027 s	Total GC Time	39,947 s
All Collections Total Time		All Collections Pause Time	
GC Count	835	Average Pause	14,023 ms
Average GC Time	58,651 ms	Longest Pause	357,159 ms
Maximum GC Time	1,277 s	Sum of Pauses	12,845 s
Total GC Time	48,974 s		

Figura 134: *Garbage Collector Summary*. G1GC. Versão Volatile em Java

Java Application		JVM Internals	
68	Threads Allocating	25	Fatal Errors
25	Finalizers Run	GC Configuration	
25	Method Profiling	25	Compressed Oops
Method Profiling		Environment	
25	Method Profiling	Processes	
		31	Competing Processes
		25	Process Started

Figura 135: Resumo do JMC. ZGC. Versão Volatile em Java

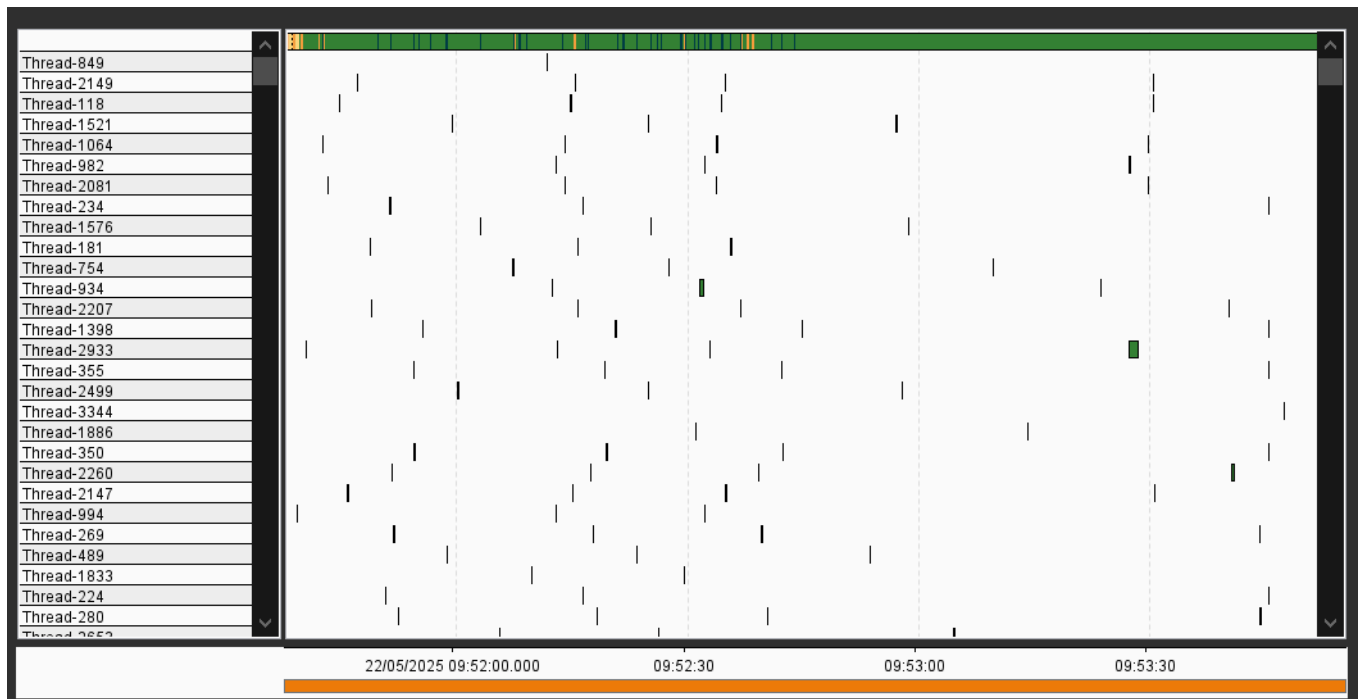


Figura 136: Threads. ZGC. Versão Volatile em Java

Monitor Class	Total Blocked Time	Distinct Threads	Count
 java.lang.Object	105,506 ms	1	5

Monitor Address	Total Blocked Time	Distinct Threads	Count
 0x1D81A725690	105,506 ms	1	5

Thread	Total Blocked Time	Count
	105,506 ms	5

Figura 137: Contenção. ZGC. Versão Volatile em Java

Method	Count	Percentage
java.io.FilterInputStream.read()	745	8,53 %
java.util.TreeMap.getCeilingEntry(Object)	533	6,1 %
java.util.HashMap\$TreeNode.getTreeNode(int, Object)	521	5,96 %
org.apache.pdfbox.io.ScratchFileBuffer.ensureAvailableBytesInPa...	482	5,52 %
org.apache.pdfbox.text.LegacyPDFStreamEngine.showGlyph(Matr...	432	4,95 %
java.io.PushbackInputStream.read()	389	4,45 %
jdk.internal.util.ArraysSupport.mismatch(byte[], int, byte[], int, int)	385	4,41 %
org.apache.pdfbox.contentstream.PDFStreamEngine.showText(by...	332	3,8 %
java.util.TreeMap.put(Object, Object, boolean)	317	3,63 %
java.lang.AbstractStringBuilder.ensureCapacityInternal(int)	179	2,05 %

Figura 138: *Hot methods*. ZGC. Versão Volatile em Java

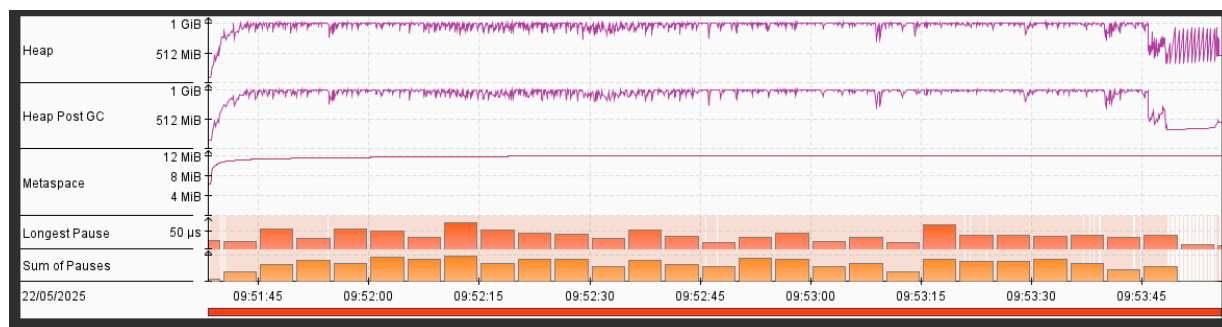


Figura 139: *Garbage Collector*. ZGC. Versão Volatile em Java

Young Collection Total Time		Old Collection Total Time	
GC Count	0	GC Count	0
Average GC Time	N/A	Average GC Time	N/A
Maximum GC Time	N/A	Maximum GC Time	N/A
Total GC Time	N/A	Total GC Time	N/A
All Collections Total Time		All Collections Pause Time	
GC Count	2.430	Average Pause	12,776 μs
Average GC Time	91,534 ms	Longest Pause	74,992 μs
Maximum GC Time	1,169 s	Sum of Pauses	99,598 ms
Total GC Time	3 min 42 s		

Figura 140: *Garbage Collector Summary*. ZGC. Versão Volatile em Java

8.2. Discussão

Como não há implementação equivalente em Haskell, nesta seção será somente discutido a diferença entre as execuções em Java com G1GC e ZGC.

Em primeiro lugar, novamente o ZGC obteve uma performance superior, inclusive com um tempo de execução menor, diferentemente do que acontece com as outras versões: 47,5 segundos para G1GC e 46,5 segundos para ZGC (Figura 124).

- A execução com ZGC apresentou uma avaliação de profile menos problemática (Figuras 129 e 135).
- O uso de GC com o G1GC foi de 18,0% (Figura 134), enquanto com o ZGC foi de apenas 0,07% (Figura 140).
- O gráfico do uso da Heap do G1GC aparenta ter quedas maiores, com uma frequência parecida com o ZGC. (Figuras 133 e 139).

Com relação ao comportamento das threads (Figuras 130 e 136), ambas execuções parecem apresentar o mesmo comportamento e não há nada de relevante a se notar.

Quanto a contenção (Figura 65), ocorreu em ambas de maneira não significativa.

Ambas versões apresentam *Hot methods* similares, com um resultado semelhante ao das versões híbridas em 4.3.

Por fim, quanto ao *Throughput* e *Response Time*, percebe-se que a execução com ZGC teve um desempenho pior, algo esperado devido a sua execução mais lenta. Dessa forma, como visto na Figura 104, o *Response Time* médio do G1GC é 49,3% daquele do ZGC e o *Throughput* o dobro.

9. Discussão Final

9.1. Tempo de Execução

	A	B	C	D	E	F	G
1	Tempo de Exec.	Serial	Hybrid	Virtual	Platform	Atomic/STM	Volatile
2	G1GC	136,3 ± 3,4	45,2 ± 1,4	47,9 ± 4,7	47,9 ± 1,3	45,5 ± 1,8	47,5 ± 3,2
3	ZGC	138,6 ± 5,9	47,5 ± 2,6	48,2 ± 4,1	49,0 ± 6,9	46,1 ± 1,3	46,5 ± 2,0
4	Haskell	79,2 ± 0,7	49,2 ± 0,09	49,2 ± 0,5	49,1 ± 0,5	51,1 ± 0,2	
5	Haskell Previsão	871,2	541,2	541,2	540,1	562,1	
6							
7							

Figura 141: Tempo de execução em segundos para todas as versões

Na Figura 141, a entrada "Haskell Previsão" se refere ao tempo de execução em Haskell multiplicado por 11, visto que o conjunto de dados utilizado para esta linguagem foi 11 vezes menor (supõe-se que a complexidade do algoritmo é linear para esta previsão).

Pela Figura 141, nota-se que **a melhor versão em geral foi a Hybrid em Java com G1GC**, a qual utiliza *synchronized* para resolução de condições de corrida. Esta obteve um tempo de execução 67% menor que a versão Serial na mesma linguagem, com

o mesmo GC, uma melhora significativa. Entretanto, vale notar que todos os tempos de execução em Java foram parecidos e considerando o erro, não fica claro nominar um vencedor.

Com relação à diferença entre Hybrid e Atomic, ambas com G1GC, a segunda teve um tempo de execução 0,6% maior que a primeira. Ainda em Java (G1GC), com relação às versões Virtual e Platform, ambas obtiveram resultados – extremamente parecidos – piores que Hybrid e Atomic. Ademais, a versão Volatile se saiu melhor que Virtual e Platform, porém pior que Atomic e Hybrid.

Com relação ao GC, o G1GC tem um tempo de execução levemente mais rápido para todas as versões exceto a Volatile, onde o ZGC se sai melhor. Suponho que isto seja devido às threads do ZGC tomarem tempo do CPU, deixando a execução do programa como um todo mais lenta, entretanto isto não explica o caso da versão Volatile.

Para as versões em Haskell, a melhor foi a Platform, com um tempo de execução 38% menor que a versão Serial. Entretanto, nesta linguagem o erro também impede de afirmar isto com certeza. Com relação a versão STM, análoga à Atomic em Java, teve o pior desempenho dentre as concorrentes, com um tempo de execução 4% maior que a Platform.

9.2. Microbenchmark

1	test	iterations	score	error	score	error	score	error	score	error	score	error	score	error	score	error	unit
2	JMHTests.BM25_BenchTest.test_idf	8	14.64	0.185	15.109	0.109	15.157	0.061	14.976	0.29	15.152	0.103	14.017	0.127	ops/ms		
3	JMHTests.BM25_BenchTest.test_add	8	3.706	0.058	3.777	0.067	3.821	0.01	3.803	0.041	7.364	0.023	3.785	0.045	ops/ms		
4	JMHTests.BM25_BenchTest.test_auction_challenge_highest_bidder_doc	8	5.29	0.035	3.395	0.159	3.364	0.241	3.37	0.175	0	0	3.347	0.176	ops/ms		
5	JMHTests.BM25_BenchTest.test_auction_spawn_thread	8	0.111	0.014	0.122	0.001	0.11	0.001	0.123	0.001	0	0	0.122	0.003	ops/ms		
6	JMHTests.BM25_BenchTest.test_constructor	8	4.629	0.04	4.847	0.053	4.865	0.007	4.875	0.022	4.826	0.02	4.771	0.131	ops/ms		
7	JMHTests.BM25_BenchTest.test_get_most_relevant_doc	8	0.003	0.001	0.003	0.001	0.079	0.005	0.003	0.001	2.763	0.008	0.003	0.001	ops/ms		
8	JMHTests.BM25_BenchTest.test_score	8	3.261	0.017	3.041	0.277	3.36	0.01	3.366	0.016	3.382	0.016	3.331	0.01	ops/ms		
9	JMHTests.BM25_BenchTest.test_update_idf	8	11.479	0.2	9.575	1.394	11.837	0.094	11.809	0.08	10.607	3.543	11.707	0.057	ops/ms		
10	JMHTests.DocumentData_BenchTest.test_cleanString	8	243.236	3.126	247.883	0.703	242.138	0.754	244.367	3.362	242.38	4.164	238.792	53.35	ops/s		
11	JMHTests.DocumentData_BenchTest.test_constructor	4	0.149	0.002	0.162	0.002	0.178	0.009	0.153	0.013	0.178	0.002	0.148	0.044	ops/s		
12	JMHTests.DocumentData_BenchTest.test_counter_spawn_thread	8	77.008	13.46	85.044	4.068	18.375	0.106	85.975	0.865	0	0	77.36	4.708	ops/s		
13	JMHTests.DocumentData_BenchTest.test_get_token_frequency	8	23.862	0.681	25.272	0.277	47.188	0.465	24.221	1.28	54.125	1.322	23.133	0.48	ops/s		
14	JMHTests.DocumentData_BenchTest.test_get_tokenizers	8	54.584	0.476	57.426	0.303	65.332	0.186	64.262	0.689	0	0	54.956	0.997	ops/ms		
15	JMHTests.Query_BenchTest.test_constructor	8	1.096	0.095	1.127	0.01	1.289	0.017	1.288	0.017	1.253	0.02	1.125	0.039	ops/ms		
16	JMHTests.Utils_BenchTest.test_create_batch_Array	8	2.694	0.096	2.67	0.114	2.746	0.155	2.831	0.074	0	0	2.597	0.072	ops/ms		
17	JMHTests.Utils_BenchTest.test_create_batch_ArrayList	8	5.902	0.22	5.928	0.117	6.233	0.039	6.327	0.078	0	0	5.553	0.285	ops/ms		
18			Atomic		Hybrid		Virtual		Platform		Serial		Volatile				

Figura 142: Comparação entre as Microbenchmarks de cada versão em Java

Na Figura 142, as entradas para os métodos concorrentes foram substituídas por 0 na versão Serial.

Primeiramente, nota-se que para os métodos os quais não foram aplicados conceitos de programação concorrente, como `cleanString`, não há variação relevante entre as versões, como esperado.

Para o caso do método `test_counter_spawn_thread`, o qual cria threads para a contagem de tokens de um documento, observa-se que este ficou bem mais eficiente na versão Virtual, com uma execução 78% mais curta que a Hybrid. Além disso, as versões Atomic e Volatile também tiveram um melhor aproveitamento para este método, com uma execução 9,4% mais curta que as versões Hybrid e Platform.

Entretanto, no método `test_get_token_frequency` a execução da versão Virtual parece ter sido a mais longa, com um tempo 88% maior que a Hybrid.

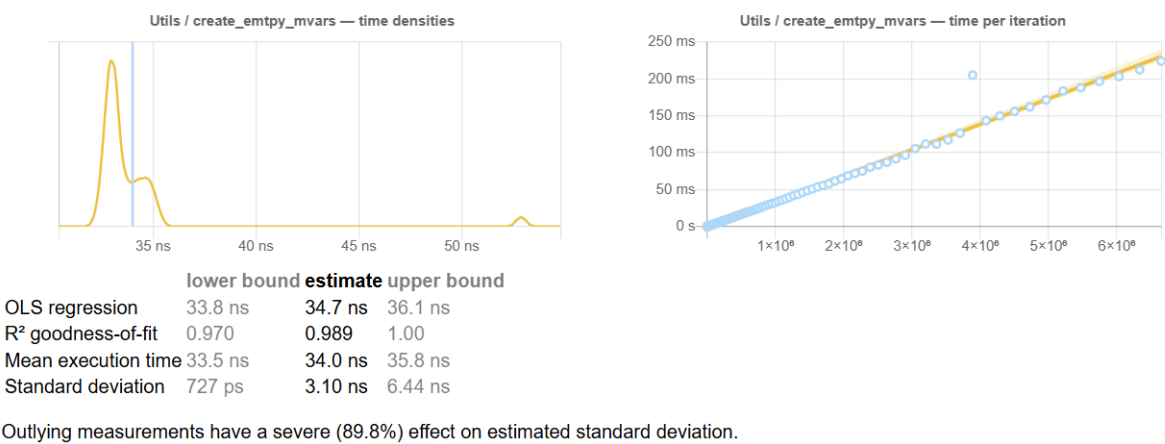
Para o caso do método `test_auction_spawn_thread`, o qual utiliza threads para determinar o documento de maior score, percebe-se uma pequena diminuição

no tempo de execução nas versões Atomic e Virtual em relação às outras. Bem como no método `test_counter_spawn_thread`, o uso de *Platform threads* e *synchronized* não parece ser a melhor escolha.

Ademais, nota-se uma tendência geral à métodos influenciados por partes concorrentes terem um tempo de execução menor que aqueles na versão Serial.

Com relação às versões em Haskell, nesta seção, somente serão comparadas as Hybrid e Atomic, visto que as outras não possuem diferenças à versão Hybrid quanto a Microbenchmark.

Utils / create_empty_mvars



Utils / create_empty_tvvars

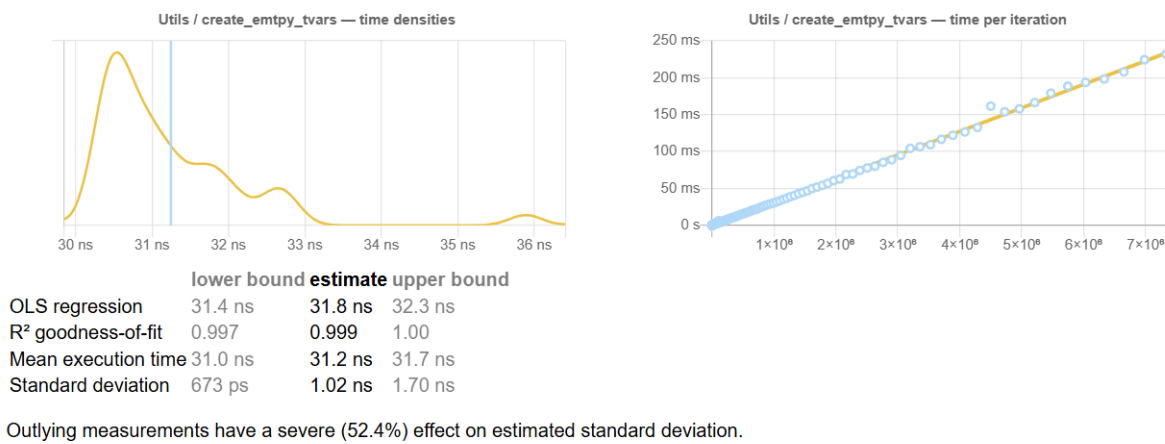
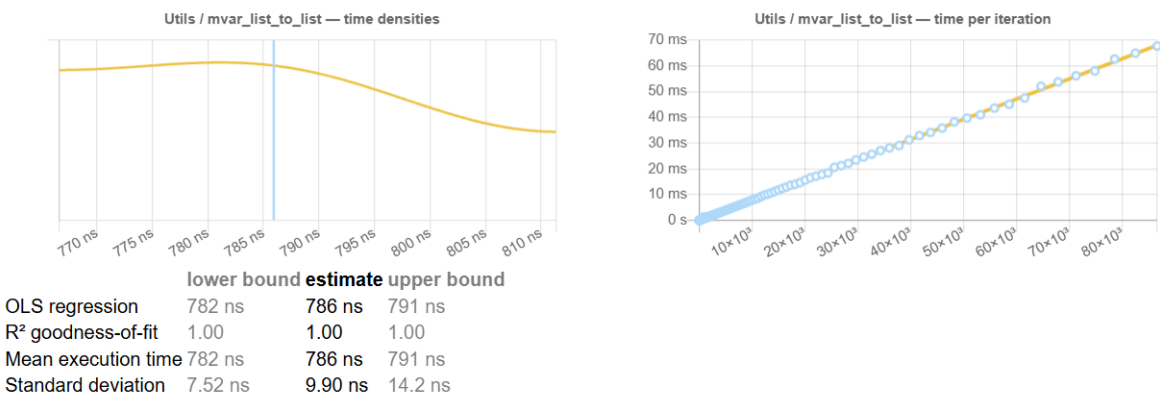


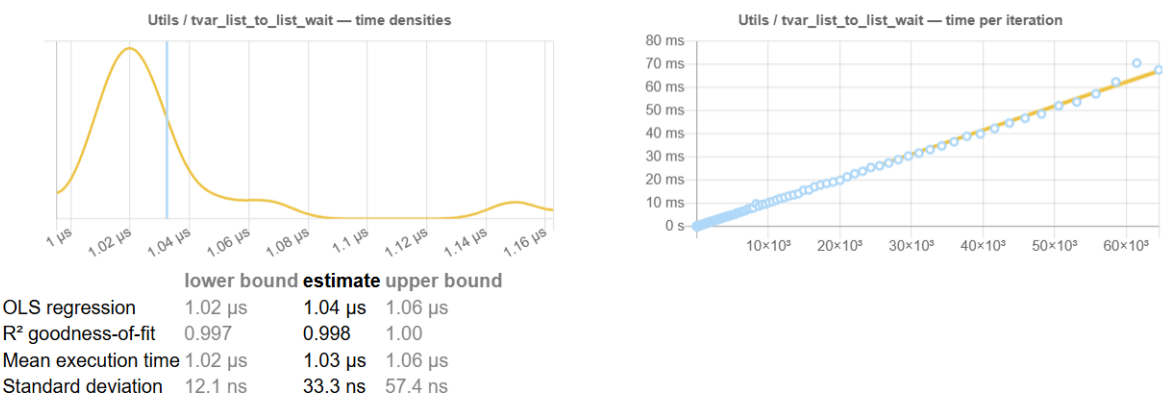
Figura 143: Comparação entre funções "create_empty" das versões Hybrid e STM em Haskell, respectivamente.

Utils / mvar_list_to_list



Outlying measurements have a slight (9.91%) effect on estimated standard deviation.

Utils / tvar_list_to_list_wait



Outlying measurements have a moderate (41.2%) effect on estimated standard deviation.

Figura 144: Comparação entre funções "list_to_list" das versões Hybrid e STM em Haskell, respectivamente.

Com base nas Figuras 143 e 144, observa-se que a versão com `MVar's` é mais rápida. No caso das funções `"list_to_list"`, há um pouco a mais de overhead naquela utilizando `TVar's` devido a necessidade de um mecanismo de bloqueio de execução de thread até que uma condição seja satisfeita.

9.3. Macrobenchmark

Nesta seção, será discutida a Macrobenchmark feita pelo JMeter para as versões em Java, visto que aquela das versões em Haskell já foi discutida em 9.1.

Com relação ao `Response Time` (Figuras 104, 12, 40, 62, 81 e 128), há uma tendência clara das versões com `G1GC` serem mais rápidas que aquelas com `ZGC`.

Com relação ao `Throughput` (Figuras 103, 11, 39, 61, 80 e 127), a mesma tendência pode ser observada, entretanto de uma maneira menos drástica. No geral, o `Throughput` médio da execução com `G1GC` parece sempre ser metade daquela com `ZGC`. Ademais, devido ao baixo número de threads acumuladas pelo tempo de execução, não possível obter gráficos clássicos de `Throughput` com `Knee Capacity` e `Usable Capacity`.

9.4. Avaliação de Profile

	A	B	C	D	E	F	G
1	Uso de GC	Serial	Hybrid	Virtual	Platform	Atomic/STM	Volatile
2	G1GC	3,61%	17,5%	21,8%	18,2%	22,1%	18,0%
3	ZGC	0,01%	0,07%	0,08%	0,06%	0,07%	0,07%
4	Haskell	56%	42%	42%	42%	34%	

Figura 145: Comparação entre o uso de GC entre as versões

Em primeiro lugar, nota-se que as versões em Haskell tem uma grande parte do seu tempo de execução gasto pelo GC. Como discutido nas seções de cada versão, isto é algo ineficiente e não caracteriza um bom código.

Em segundo lugar, com relação às versões Java, observa-se que aquela com o menor uso de GC é a Serial e, dentre as concorrentes, a Hybrid (`G1GC`). Bem como em Haskell, o tempo gasto pelo GC (`G1GC`) nas versões concorrentes não é ideal. Entretanto, isto não parece ser o caso para as execuções com `ZGC`, as quais tem porcentagem de tempo gasta pelo GC extremamente baixas. Como já mencionado neste relatório, isto se deve a natureza concorrente deste GC, e, como já visto nesta seção de discussão final, não necessariamente significa uma execução de código mais rápida.

Com relação aos `Hot Methods` (Figuras 107, 14, 43, 66, 85, 132, 113, 18, 49, 71, 91, 138), percebe-se que em todas as versões os métodos mais utilizados são aqueles da biblioteca `pdfbox`, como discutido anteriormente nas seções de cada versão. Logo, a experimentação com outras bibliotecas de mesma função é recomendado para uma melhora em performance.

Com relação à contenção, foi relativamente baixa em todas as versões, exceto na Platform, aonde foi alta. Suponho que isto se deve ao fato das `Virtual Threads` não serem consideradas bloqueadas da mesma maneira que `Platform Threads` no JMC.

Com relação às threads, somente vale destacar a versão Virtual, a qual apresenta o comportamento descrito no parágrafo acima.

9.5. Lições aprendidas e Opiniões

Neste trabalho, tive a oportunidade de desenvolver meu conhecimentos e habilidades de programação concorrente. Dentre as principais lições aprendidas, estão: Há um número ideal de threads para cada aplicação, levando em conta cada máquina, normalmente se aproximando do número de núcleos disponíveis; A depender do que se vai implementar de maneira concorrente, usar a estrutura adequada é importante, pois apesar de estruturas diferentes serem capazes de resolver um mesmo problema, suas performances podem variar drasticamente; Um foco em diminuir o tempo de GC em uma linguagem com esta funcionalidade é algo que sempre deve se ter em mente ao pensar em performance.

Quanto aos pontos positivos e negativos de Java e Haskell, notados na elaboração deste trabalho:

- Pontos positivos em Java:
 - Há várias funcionalidades para uma aplicação concorrente: mutex, semáforo, variáveis atômicas, variáveis voláteis, *Reentrant Lock*, *Virtual e Platform Threads*, *CountDownLatch* para sincronização, estruturas de dados feitas para aplicações concorrentes e etc.
 - Modelo de memória bem definido.
 - Várias ferramentas de avaliação.
- Pontos negativos em Java:
 - Não parece ser simples conseguir um uso baixo de GC, isto considerando o G1GC.
 - Há muitas maneiras de fazer uma mesma coisa, sem estar claro qual é a maneira correta. Por exemplo, inicialmente não sabia ao certo como colocar um ponto no código onde se devia esperar a execução de todas as threads. Tentei criar uma classe para isto e usar bibliotecas que achei online, mas com nenhum sucesso. No final, bastava usar um `join`.
 - Otimizações do JIT podem trazer comportamentos inesperados.
- Pontos positivos em Haskell:
 - Modelo de memória simples.
 - Pontos do código com `Side-effects` são explícitos devido ao `Monad IO`.
 - Implementação de uma versão concorrente de um código prática.
- Pontos negativos em Haskell:
 - `Lazy Evaluation`. Este foi o meu maior, e praticamente único, obstáculo ao lidar com programação concorrente em Haskell. Devido a esta, não fica claro em que parte do código esta sendo executado cada trabalho. No geral, talvez por não estar familiar com ela, tornou a depuração do código mais difícil.

Como um último comentário, pelo que tenho pesquisado online, Haskell é uma linguagem perfeitamente preparada para programação concorrente, passível de obter performances excelentes como em outras linguagens modernas. Entretanto, isto não foi o resultado obtido neste relatório, possivelmente pela minha inexperiência com a linguagem, já que aprendi ela somente para a disciplina. Em geral, gostei de Haskell e espero utilizar ela mais no futuro.

Referências

- [Architecture 2012] Architecture, G. (2012). In Brown, A. and Wilson, G., editors, *Architecture of Open Source Applications, Volume II*. The Architecture of Open Source Applications Project. Accessed: 2025-05-27.
- [BM25 Wikipédia 2025] BM25 Wikipédia (2025). Okapi bm25 — wikipédia, a enciclopédia livre. Acesso em: 27 de maio de 2025.
- [Criterion 2024] Criterion (2024). A powerful but simple library for measuring the performance of haskell code. <https://github.com/haskell/criterion>. Accessed: 2025-05-27.
- [dejafu 2025] dejafu (2025). dejafu: Systematic testing for concurrent haskell programs. <https://github.com/barrucadu/dejafu>. Accessed: 2025-05-27.
- [Eduardo 2025] Eduardo, C. (2025). Progconc. <https://github.com/CinquilCinquil/ProgConc>.
- [GHC 2025] GHC (2025). The glasgow haskell compiler (ghc). Accessed: 2025-05-27.
- [ghc-events analyze 2014] ghc-events analyze (2014). Performance profiling with ghc-events-analyze. Accessed: 2025-05-27.
- [GHC Profiling 2025] GHC Profiling (2025). Ghc user’s guide: Profiling. Accessed: 2025-05-27.
- [Gyan 2014] Gyan (2014). Why page tree obtained after creating pdf using pdfbox different? Stack Overflow. Accessed: 2025-05-27.
- [Haskell Concurrent 2025] Haskell Concurrent (2025). Control.concurrent – haskell base library (base-4.21.0.0). Accessed: 2025-05-27.
- [Haskell Parallel 2025] Haskell Parallel (2025). Control.parallel – haskell parallel programming library (parallel-3.2.2.0). Accessed: 2025-05-27.
- [Haskell Wiki 2025] Haskell Wiki (2025). Ghc/memory management. Accessed: 2025-05-27.
- [Löb 2022] Löb, A. (2022). Violating memory safety with haskell’s value restriction. Accessed: 2025-05-27.
- [Multiverse 2012] Multiverse (2012). Multiverse: Software transactional memory for java. <https://github.com/pveentjer/Multiverse>. Accessed: 2025-05-27.
- [Nelson 2025] Nelson, T. (2025). Technically-oriented pdf collection (papers, specs, decks, manuals, etc). <https://github.com/tpn/pdfs>. Accessed: 2025-05-27.
- [Robertson and Zaragoza 2009] Robertson, S. and Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*, volume 3 of *Foundations and Trends in Information Retrieval*. Now Publishers Inc.
- [Yang 2014] Yang, E. Z. (2014). So you want to add a new concurrency primitive to ghc? Accessed: 2025-05-27.